EJERCICIOS RESUELTOS DEL TEMA 2

Ejercicio 2.1. Desarrolla un programa que sume 543 y 653.

```
package SegundoTema;

public class Sumando {

	public static void main(String[] args) {
		int x = 543;
		int y = 653;
		System.out.println("Suma: " + (x + y));
	}
}
```

Ejercicio 2.2. Desarrolla un programa que divida 658 entre 251 con y sin decimales.

```
package SegundoTema;

public class DividiendoSinDecimales {

	public static void main(String[] args) {
		int x = 658;
		int y = 251;
		System.out.println("División: " + (x / y));
	}
}
```

```
package SegundoTema;

public class DividiendoConDecimales {

	public static void main(String[] args) {
		double x = 658;
		double y = 251;
		System.out.println("División: " + (x / y));
	}
}
```

Ejercicio 2.3. Desarrolla un programa que calcule el resto de la división 41 entre 3.

```
package SegundoTema;

public class RestoDeLaDivision {

	public static void main(String[] args) {
		int x = 41;
		int y = 3;
		System.out.println("División: " + (x % y));
	}
}
```

Ejercicio 2.4. Escribe un programa que le pida al usuario su nombre (texto), su edad (entero) y su peso en kilos (decimal). Luego mostrará por consola un mensaje como: Hola, Carla. Tienes 28 años y pesas 60.5 kg. Añade comentarios a tu código.

```
package SegundoTema;

import java.util.Scanner;

public class DatosCompletos {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce tu nombre: ");
      String nombre = teclado.nextLine(); // Lee una línea de texto

      System.out.print("Introduce tu edad: ");
      int edad = teclado.nextInt(); // Lee un número entero

      System.out.print("Introduce tu peso en kilogramos (ej. 70.5): ");
      double peso = teclado.nextDouble(); // Lee un número decimal

      // Mostramos un resumen
      System.out.println("Hola, " + nombre + ". Tienes " + edad + " años y pesas " + peso + "
kg.");

      teclado.close();
   }
}
```

Ejercicio 2.5. Desarrolla un programa que pida al usuario una cantidad de metros y los convierta a millas (1 milla = 1.609m). Usa los dos tipos de comentarios vistos a lo largo del código.

```
package SegundoTema;

import java.util.Scanner; // Importamos la clase Scanner para leer datos del usuario

public class ConversionMetrosAMillas {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in); // Creamos el objeto Scanner llamado
"teclado"

      /*
       * Solicitamos al usuario una cantidad de metros.
       * Vamos a convertir esa cantidad a millas usando la fórmula:
       * millas = metros / 1609
       */
```

```
        System.out.print("Introduce una cantidad de metros: "); // Pedimos los metros al usuario
        double metros = teclado.nextDouble(); // Leemos un número decimal y lo guardamos en la variable "metros"

        // Convertimos metros a millas dividiendo entre 1.609
        double millas = metros / 1609;

        // Mostramos el resultado en pantalla
        System.out.println(metros + " metros equivalen a " + millas + " millas.");

        teclado.close(); // Cerramos el objeto Scanner para liberar recursos
    }
}
```

Ejercicio 2.6. Desarrolla un programa que divida 4 entre 3 usando una variable de tipo entero, luego usando variables de tipo real de precisión simple y de precisión doble.

```
Package SegundoTema;

public class DivisionTipos {
    public static void main(String[] args) {
        // División con enteros
        int resultadoEntero = 4 / 3;
        System.out.println("División entera (int): 4 / 3 = " + resultadoEntero);

        // División con float (precisión simple)
        float resultadoFloat = 4f / 3f; // Operandos con sufijo 'f'
        System.out.println("División en coma flotante (float): 4 / 3 = " + resultadoFloat);

        // División con double (precisión doble)
        double resultadoDouble = 4.0 / 3.0;
        System.out.println("División en coma flotante (double): 4 / 3 = " + resultadoDouble);
    }
}
```

Ejercicio 2.7. Desarrolla un programa que calcule el área de un círculo (área = pi * radio al cuadrado). El radio se le pedirá al usuario.

```
package SegundoTema;

import java.util.Scanner;

public class AreaCirculo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Pedir el radio al usuario
        System.out.print("Introduce el radio del círculo: ");
```

```
        double radio = teclado.nextDouble();

        // Aproximación de PI
        double pi = 3.1416;

        // Calcular el área con multiplicación directa
        double area = pi * radio * radio;

        // Mostrar el resultado
        System.out.println("El área del círculo es: " + area);

        teclado.close();
    }
}
```

Ejercicio 2.8. Desarrolla un programa que calcule el volumen de una esfera (volumen = pi * radio al cubo * 4/3). El radio lo introducirá el usuario y se leerá con una variable de tipo double.

```
package SegundoTema;

import java.util.Scanner;

public class VolumenEsfera {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // Pedir al usuario el radio de la esfera
        System.out.print("Introduce el radio de la esfera: ");
        double radio = teclado.nextDouble();

        // Aproximación del valor de PI
        double pi = 3.1416;

        // Calcular el volumen: (4/3) * pi * radio^3
        double volumen = (4.0 / 3.0) * pi * radio * radio * radio;

        // Mostrar el resultado
        System.out.println("El volumen de la esfera es: " + volumen);

        teclado.close();
    }
}
```

Ejercicio 2.9. Desarrolla un programa que pida al usuario una distancia y una velocidad constante (double). El programa calculará el tiempo necesario para recorrer ese espacio en horas, minutos y segundos (int).

```
package SegundoTema;

import java.util.Scanner;
```

```
public class TiempoDeViaje {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedir al usuario la distancia en kilómetros
      System.out.print("Introduce la distancia en kilómetros: ");
      double distancia = teclado.nextDouble();

      // Pedir la velocidad en km/h
      System.out.print("Introduce la velocidad constante en km/h: ");
      double velocidad = teclado.nextDouble();

      // Calcular el tiempo total en horas (real)
      double tiempoHoras = distancia / velocidad;

      // Convertir a horas, minutos y segundos
      int horas = (int) tiempoHoras;
      int minutos = (int) ((tiempoHoras - horas) * 60);
      int segundos = (int) (((tiempoHoras - horas) * 60 - minutos) * 60);

      // Mostrar el resultado
      System.out.println("Tiempo estimado de viaje:");
      System.out.println(horas + " horas, " + minutos + " minutos y " + segundos + "
segundos.");

      teclado.close();
   }
}
```

Ejercicio 2.10. Desarrolla un programa que pida al usuario que introduzca dos números de doble precisión. El programa dividirá el primero entre el segundo y mostrará el resultado por consola utilizando tres cifras decimales.

```
package SegundoTema;

import java.util.Scanner;

public class DivisionConFormato {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedir los dos números de doble precisión
      System.out.print("Introduce el primer número: ");
      double num1 = teclado.nextDouble();

      System.out.print("Introduce el segundo número: ");
      double num2 = teclado.nextDouble();

      // Realizar la división y mostrar el resultado con tres cifras decimales
      double resultado = num1 / num2;
```

```
        System.out.printf("El resultado de la división es: %.3f\n", resultado);

        teclado.close();
    }
}
```

NOTA: Si introduces 0 como segundo número, el programa intentará dividir entre cero y mostrará un resultado como "Infinity" o un error. Esto es normal por ahora; aprenderemos a controlar este tipo de situaciones más adelante.

Ejercicio 2.11. Pide al usuario que introduzca dos números enteros y muestra la suma de ambos. Usa nextInt() y muestra el resultado con System.out.println().

```
package SegundoTema;

import java.util.Scanner;

public class SumaEnteros {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce el primer número entero: ");
        int num1 = teclado.nextInt();

        System.out.print("Introduce el segundo número entero: ");
        int num2 = teclado.nextInt();

        int suma = num1 + num2;
        System.out.println("La suma es: " + suma);

        teclado.close();
    }
}
```

Ejercicio 2.12. Pide al usuario que introduzca su nombre y apellidos (en líneas separadas) usando nextLine(). Luego muestra el nombre completo concatenado.

```
package SegundoTema;

import java.util.Scanner;

public class NombreCompleto {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.nextLine();

        System.out.print("Introduce tus apellidos: ");
```

```
        String apellidos = teclado.nextLine();

        String nombreCompleto = nombre + " " + apellidos;
        System.out.println("Nombre completo: " + nombreCompleto);

        teclado.close();
    }
}
```

Ejercicio 2.13. Pide al usuario una distancia en metros (tipo int) y conviértela a kilómetros (tipo double). Muestra el resultado con dos decimales.

```
package SegundoTema;

import java.util.Scanner;

public class MetrosAKilometros {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce la distancia en metros (entero): ");
        int metros = teclado.nextInt();

        double kilometros = metros / 1000.0;
        System.out.printf("La distancia en kilómetros es: %.2f\n", kilometros);

        teclado.close();
    }
}
```

Ejercicio 2.14. Pide al usuario un número entero y guárdalo en una variable int. Luego conviértelo a byte y short usando casting, y muestra ambos valores.

```
package SegundoTema;

import java.util.Scanner;

public class CastingEnteros {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce un número entero: ");
        int num = teclado.nextInt();

        byte b = (byte) num;
        short s = (short) num;

        System.out.println("Valor como byte: " + b);
        System.out.println("Valor como short: " + s);

        teclado.close();
```

```
    }
}
```

Ejercicio 2.15. Pide dos números double y muestra el resultado de la división con tres cifras decimales usando printf.

```
package SegundoTema;

import java.util.Scanner;

public class DivisionDouble {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce el primer número decimal: ");
        double num1 = teclado.nextDouble();

        System.out.print("Introduce el segundo número decimal: ");
        double num2 = teclado.nextDouble();

        double resultado = num1 / num2;
        System.out.printf("Resultado de la división: %.3f\n", resultado);

        teclado.close();
    }
}
```

Ejercicio 2.16. Declara una variable entera e inicialízala con un valor. Usa i++, i--, i += 5, i *= 2 y muestra el valor después de cada operación.

```
package SegundoTema;

public class OperacionesIncremento {
    public static void main(String[] args) {
        int i = 10; // Valor inicial
        System.out.println("Valor inicial: " + i);

        i++; // Incrementar en 1
        System.out.println("Después de i++: " + i);

        i--; // Decrementar en 1
        System.out.println("Después de i--: " + i);

        i += 5; // Sumar 5
        System.out.println("Después de i += 5: " + i);

        i *= 2; // Multiplicar por 2
        System.out.println("Después de i *= 2: " + i);
    }
}
```

Ejercicio 2.17. Desarrolla un programa que pida varias letras al usuario y luego las muestre por pantalla formando una única palabra.

```
package SegundoTema;

import java.util.Scanner;

public class FormarPalabra {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedir letras individuales al usuario
      System.out.print("Introduce la primera letra: ");
      char l1 = teclado.next().charAt(0);

      System.out.print("Introduce la segunda letra: ");
      char l2 = teclado.next().charAt(0);

      System.out.print("Introduce la tercera letra: ");
      char l3 = teclado.next().charAt(0);

      System.out.print("Introduce la cuarta letra: ");
      char l4 = teclado.next().charAt(0);

      System.out.print("Introduce la quinta letra: ");
      char l5 = teclado.next().charAt(0);

      // Concatenar las letras para formar una palabra
      String palabra = "" + l1 + l2 + l3 + l4 + l5;

      // Mostrar la palabra final
      System.out.println("La palabra formada es: " + palabra);

      teclado.close();
   }
}
```

Ejercicio 2.18. Desarrolla un programa que pida al usuario 5 palabras de una en una. Después se mostrarán en consola como una frase.

```
package SegundoTema;

import java.util.Scanner;

public class FormarFrase {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce la primera palabra: ");
      String p1 = teclado.nextLine();
```

```
        System.out.print("Introduce la segunda palabra: ");
        String p2 = teclado.nextLine();

        System.out.print("Introduce la tercera palabra: ");
        String p3 = teclado.nextLine();

        System.out.print("Introduce la cuarta palabra: ");
        String p4 = teclado.nextLine();

        System.out.print("Introduce la quinta palabra: ");
        String p5 = teclado.nextLine();

        // Concatenar las palabras separadas por espacios para formar la frase
        String frase = p1 + " " + p2 + " " + p3 + " " + p4 + " " + p5;

        // Mostrar la frase completa
        System.out.println("La frase formada es: " + frase);

        teclado.close();
    }
}
```

Ejercicio 2.19. Pide al usuario una temperatura en grados Celsius y conviértela a Fahrenheit usando la fórmula: F = (C * 9/5) + 32. Muestra el resultado con dos decimales.

```
package SegundoTema;

import java.util.Scanner;

public class CelsiusAFahrenheit {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce la temperatura en grados Celsius: ");
        double celsius = teclado.nextDouble();

        double fahrenheit = (celsius * 9 / 5) + 32;
        System.out.printf("La temperatura en Fahrenheit es: %.2f\n", fahrenheit);

        teclado.close();
    }
}
```

Ejercicio 2.20. Pide al usuario tres números double y muestra el promedio con dos decimales.

```
package SegundoTema;

import java.util.Scanner;

public class PromedioTresNumeros {
    public static void main(String[] args) {
```

```
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce el primer número: ");
        double num1 = teclado.nextDouble();

        System.out.print("Introduce el segundo número: ");
        double num2 = teclado.nextDouble();

        System.out.print("Introduce el tercer número: ");
        double num3 = teclado.nextDouble();

        double promedio = (num1 + num2 + num3) / 3;
        System.out.printf("El promedio es: %.2f\n", promedio);

        teclado.close();
    }
}
```

Ejercicio 2.21. Pide un número de días y muestra cuántas horas, minutos y segundos hay en esa cantidad.

```
package SegundoTema;

import java.util.Scanner;

public class DiasAHorasMinutosSegundos {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce el número de días: ");
        int dias = teclado.nextInt();

        int horas = dias * 24;
        int minutos = horas * 60;
        int segundos = minutos * 60;

        System.out.println(dias + " días equivalen a:");
        System.out.println(horas + " horas");
        System.out.println(minutos + " minutos");
        System.out.println(segundos + " segundos");

        teclado.close();
    }
}
```

Ejercicio 2.22. Pide nombre y apellido y forma una dirección de email en formato:

nombre.apellido@ejemplo.com.

```
package SegundoTema;
```

```
import java.util.Scanner;

public class FormarEmail {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce tu nombre: ");
      String nombre = teclado.nextLine();

      System.out.print("Introduce tu apellido: ");
      String apellido = teclado.nextLine();

      String email = nombre + "." + apellido + "@ejemplo.com";
      System.out.println("Tu email es: " + email);

      teclado.close();
   }
}
```

Ejercicio 2.23. Pide base y altura, luego calcula y muestra perímetro y área.

```
package SegundoTema;

import java.util.Scanner;

public class PerimetroAreaRectangulo {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce la base del rectángulo: ");
      double base = teclado.nextDouble();

      System.out.print("Introduce la altura del rectángulo: ");
      double altura = teclado.nextDouble();

      double perimetro = 2 * (base + altura);
      double area = base * altura;

      System.out.printf("El perímetro es: %.2f\n", perimetro);
      System.out.printf("El área es: %.2f\n", area);

      teclado.close();
   }
}
```

EJERCICIOS RESUELTOS DEL TEMA 3

Ejercicio 3.1. Desarrolla un programa que pida al usuario dos números reales de simple precisión (float) y diga cuál es el mayor de ellos.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_1 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce el primer número (float): ");
      float num1 = teclado.nextFloat();

      System.out.print("Introduce el segundo número (float): ");
      float num2 = teclado.nextFloat();

      if (num1 > num2) {
         System.out.println("El mayor es: " + num1);
      } else if (num2 > num1) {
         System.out.println("El mayor es: " + num2);
      } else {
         System.out.println("Ambos números son iguales.");
      }

      teclado.close();
   }
}
```

Ejercicio 3.2. Desarrolla un programa que pida al usuario un número entero corto (short). Si es impar pedirá un segundo número entero, pero en esta ocasión largo (long). Ahora mostrará por pantalla si el número largo es impar.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_2 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un número entero corto (short): ");
      short num1 = (short) teclado.nextInt();

      if (num1 % 2 != 0) {
         System.out.print("El número es impar. Introduce ahora un número largo (long): ");
         long num2 = teclado.nextLong();

         if (num2 % 2 != 0) {
```

```
            System.out.println("El número largo también es impar.");
         } else {
            System.out.println("El número largo es par.");
         }
      } else {
         System.out.println("El número introducido es par. No se pide otro número.");
      }

      teclado.close();
   }
}
```

Ejercicio 3.3. Desarrolla un programa que pida un número entero corto (short) al usuario y muestre por pantalla si es múltiplo de 2 o de 5. Después, pedirá un nuevo número y lo restará al primero.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_3 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un número entero corto (short): ");
      short num1 = (short) teclado.nextInt();

      if (num1 % 2 == 0 || num1 % 5 == 0) {
         System.out.println("Es múltiplo de 2 o de 5.");
      } else {
         System.out.println("No es múltiplo de 2 ni de 5.");
      }

      System.out.print("Introduce otro número para restar al primero: ");
      int num2 = teclado.nextInt();

      int resultado = num1 - num2;
      System.out.println("El resultado de la resta es: " + resultado);

      teclado.close();
   }
}
```

Ejercicio 3.4. Desarrolla un programa que pida al usuario un número entero de un byte y que diga si es múltiplo de 2 y de 5.

```
package TercerTema;

import java.util.Scanner;
```

```
public class Ejercicio3_4 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un número entero de tipo byte: ");
      byte numero = (byte) teclado.nextInt();

      if (numero % 2 == 0 && numero % 5 == 0) {
         System.out.println("El número es múltiplo de 2 y de 5.");
      } else {
         System.out.println("El número NO es múltiplo de 2 y de 5 a la vez.");
      }

      teclado.close();
   }
}
```

Ejercicio 3.5. Usa el operador condicional para calcular el menor de dos números reales de doble precisión introducidos por el usuario.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_5 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce el primer número (double): ");
      double num1 = teclado.nextDouble();

      System.out.print("Introduce el segundo número (double): ");
      double num2 = teclado.nextDouble();

      // Usamos el operador condicional para determinar el menor
      double menor = (num1 < num2) ? num1 : num2;

      System.out.println("El número menor es: " + menor);

      teclado.close();
   }
}
```

Ejercicio 3.6. Desarrolla un programa que pida dos números enteros cortos (short) al usuario. Después usará el operador ternario (?) para asignar a una variable booleana el valor “true” si los dos números son iguales, o “false” si no los son.

```
package TercerTema;

import java.util.Scanner;
```

```
public class Ejercicio3_6 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce el primer número (short): ");
      short num1 = (short) teclado.nextInt();

      System.out.print("Introduce el segundo número (short): ");
      short num2 = (short) teclado.nextInt();

      // Usamos el operador condicional para comparar los dos números
      boolean sonIguales = (num1 == num2) ? true : false;

      System.out.println("¿Los números son iguales?: " + sonIguales);

      teclado.close();
   }
}
```

Ejercicio 3.7. Desarrolla un programa que pida al usuario que introduzca un carácter. Después dirá si se trata de un signo de puntuación (. , ; :), un número u otro carácter.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_7 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce un carácter: ");
      char c = teclado.nextLine().charAt(0);

      switch (c) {
         case '.':
         case ',':
         case ';':
         case ':':
            System.out.println("Es un signo de puntuación.");
            break;

         case '0':
         case '1':
         case '2':
         case '3':
         case '4':
         case '5':
         case '6':
         case '7':
         case '8':
```

```
        case '9':
            System.out.println("Es un número.");
            break;

        default:
            System.out.println("Es otro tipo de carácter.");
    }

    teclado.close();
  }
}
```

Ejercicio 3.8. Desarrolla un programa que pida al usuario un número del 1 al 7. El programa mostrará el nombre del día correspondiente (1 = lunes, 2 = martes, ..., 7 = domingo).

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_8 {
  public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);

    System.out.print("Introduce un número del 1 al 7: ");
    int dia = teclado.nextInt();

    switch (dia) {
        case 1:
            System.out.println("Lunes");
            break;
        case 2:
            System.out.println("Martes");
            break;
        case 3:
            System.out.println("Miércoles");
            break;
        case 4:
            System.out.println("Jueves");
            break;
        case 5:
            System.out.println("Viernes");
            break;
        case 6:
            System.out.println("Sábado");
            break;
        case 7:
            System.out.println("Domingo");
            break;
        default:
            System.out.println("Número fuera de rango.");
    }
```

```
        teclado.close();
    }
}
```

Ejercicio 3.9. Desarrolla un programa que pida al usuario una contraseña. La contraseña correcta es 1234. Mientras no acierte la contraseña permanecerá el programa dentro del bucle pidiendo que vuelva a introducir la contraseña. Al final del programa se le dará *feedback* al usuario diciendo: "Contraseña acertada."

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_9 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String contraseña = "";

        while (!contraseña.equals("1234")) {
            System.out.print("Introduce la contraseña: ");
            contraseña = teclado.nextLine();
        }

        System.out.println("Contraseña acertada.");
        teclado.close();
    }
}
```

Ejercicio 3.10. Desarrolla un programa que calcule el cubo de un número entero corto introducido por el usuario. Esto lo hará de manera repetitiva hasta que el usuario introduzca un 0. Al final del programa se le dará *feedback* al usuario diciendo: "Fin del programa."

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_10 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        short numero = -1; // cualquier valor distinto de 0 para entrar en el bucle

        while (numero != 0) {
            System.out.print("Introduce un número (0 para salir): ");
            numero = teclado.nextShort();

            if (numero != 0) {
                int cubo = numero * numero * numero;
                System.out.println("El cubo de " + numero + " es " + cubo);
            }
```

```
        }

        System.out.println("Fin del programa.");
        teclado.close();
    }
}
```

Ejercicio 3.11. Vuelve a hacer el ejercicio 3.10, pero ahora con la estructura do-while.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_11 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        short numero;

        do {
            System.out.print("Introduce un número (0 para salir): ");
            numero = teclado.nextShort();

            if (numero != 0) {
                int cubo = numero * numero * numero;
                System.out.println("El cubo de " + numero + " es " + cubo);
            }

        } while (numero != 0);

        System.out.println("Fin del programa.");
        teclado.close();
    }
}
```

Ejercicio 3.12. Desarrolla un programa que pida al usuario que introduzca números enteros positivos y los sume. Cuando el usuario introduzca un número negativo, el programa debe terminar y mostrar la suma total de los números introducidos (sin contar el negativo).

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_12 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int numero;
        int suma = 0;

        do {
            System.out.print("Introduce un número positivo (negativo para salir): ");
            numero = teclado.nextInt();
```

```
        if (numero >= 0) {
            suma += numero;
        }
    } while (numero >= 0);

    System.out.println("La suma total es: " + suma);
    teclado.close();
  }
}
```

Ejercicio 3.13. Desarrolla un programa que muestre por consola los números del 100 al 200 seguidos y separados por un espacio.

```
package TercerTema;

public class Ejercicio3_13 {
  public static void main(String[] args) {
    for (int i = 100; i <= 200; i++) {
        System.out.print(i + " ");
    }
    System.out.println(); // salto de línea final
  }
}
```

Ejercicio 3.14. Desarrolla un programa que muestre por consola los números del 50 al 90 que sean divisibles entre 5 y entre 3.

```
package TercerTema;

public class Ejercicio3_14 {
  public static void main(String[] args) {
    for (int i = 50; i <= 90; i++) {
        if (i % 3 == 0 && i % 5 == 0) {
            System.out.print(i + " ");
        }
    }
    System.out.println();
  }
}
```

Ejercicio 3.15. Desarrolla un programa que muestre por consola los números pares comprendidos entre el 21 y el 37.

```
package TercerTema;

public class Ejercicio3_15 {
  public static void main(String[] args) {
    for (int i = 22; i <= 36; i += 2) {
        System.out.print(i + " ");
```

```
        }
        System.out.println();
    }
}
```

Ejercicio 3.16. Desarrolla un programa que muestre por consola los números del 15 al 5, es decir, en lugar de incrementar, deberás decrementar la variable de 1 en 1.

```
package TercerTema;

public class Ejercicio3_16 {
    public static void main(String[] args) {
        for (int i = 15; i >= 5; i--) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

Ejercicio 3.17. Muestra una tabla de multiplicar del 1 al 5.

```
package TercerTema;

public class Ejercicio3_19 {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) { // Tabla del 1 al 5
            for (int j = 1; j <= 10; j++) {
                System.out.println(i + " x " + j + " = " + (i * j));
            }
            System.out.println(); // Línea en blanco entre tablas
        }
    }
}
```

Ejercicio 3.18. Dibuja un rectángulo de asteriscos de 4 filas y 6 columnas.

```
package TercerTema;

public class Ejercicio3_20 {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int col = 1; col <= 6; col++) {
                System.out.print("* ");
            }
            System.out.println(); // Salto de línea al terminar cada fila
        }
    }
}
```

Ejercicio 3.19. Desarrolla un programa que pida al usuario palabras por teclado. Si el usuario escribe “salir”, el programa terminará con el mensaje “Programa finalizado”. (Usa un while infinito y break).

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_17 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      String palabra;

      while (true) {
         System.out.print("Escribe una palabra (o 'salir' para terminar): ");
         palabra = teclado.nextLine();

         if (palabra.equals("salir")) {
            break; // Salimos del bucle si escribe "salir"
         }
      }

      System.out.println("Programa finalizado.");
      teclado.close();
   }
}
```

Ejercicio 3.20. Desarrolla un programa que muestre los números del 1 al 10, excepto el 4 y el 7. (Usa for y continue).

```
package TercerTema;

public class Ejercicio3_18 {
   public static void main(String[] args) {
      for (int i = 1; i <= 10; i++) {
         if (i == 4 || i == 7) {
            continue; // Saltamos el 4 y el 7
         }
         System.out.print(i + " ");
      }
      System.out.println(); // Salto de línea final
   }
}
```

Ejercicio 3.21. Desarrolla un programa que muestre por consola los números impares del 39 al 17 usando while.

```
package TercerTema;

public class Ejercicio3_21 {
   public static void main(String[] args) {
```

```
        int i = 39;
        while (i >= 17) {
            if (i % 2 != 0) {
                System.out.print(i + " ");
            }
            i--;
        }
    }
}
```

Ejercicio 3.22. Desarrolla un programa que muestre por consola los números pares del 86 al 60, usando “do...while”.

```
package TercerTema;

public class Ejercicio3_22 {
    public static void main(String[] args) {
        int i = 86;
        do {
            if (i % 2 == 0) {
                System.out.print(i + " ");
            }
            i--;
        } while (i >= 60);
    }
}
```

Ejercicio 3.23. Desarrolla un programa que escriba indefinidamente “no hay fin” en pantalla sin avanzar de línea.

```
package TercerTema;

public class Ejercicio3_23 {
    public static void main(String[] args) {
        while (true) {
            System.out.print("no hay fin ");
        }
    }
}
```

Ejercicio 3.24. Desarrolla un programa que muestre indefinidamente en pantalla los números pares empezando por el 2.

```
package TercerTema;

public class Ejercicio3_24 {
    public static void main(String[] args) {
        int i = 2;
        while (true) {
            System.out.print(i + " ");
```

```
            i += 2;
        }
    }
}
```

Ejercicio 3.25. Desarrolla un programa que muestre 4 veces los números pares del 10 al 20, en una misma línea.

```
package TercerTema;

public class Ejercicio3_25 {
    public static void main(String[] args) {
        for (int rep = 1; rep <= 4; rep++) {
            for (int i = 10; i <= 20; i++) {
                if (i % 2 == 0) {
                    System.out.print(i + " ");
                }
            }
        }
    }
}
```

Ejercicio 3.26. Desarrolla un programa que muestre por consola los números comprendidos entre el 100 y el 130 que sean divisibles entre 5, 6 o 7.

```
package TercerTema;

public class Ejercicio3_26 {
    public static void main(String[] args) {
        for (int i = 100; i <= 130; i++) {
            if (i % 5 == 0 || i % 6 == 0 || i % 7 == 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Ejercicio 3.27. Desarrolla un programa que dibuje un cuadrado hueco de arrobas de lado 3. Así:

```
@@@
@   @
@@@
```

```
package TercerTema;

public class Ejercicio3_27 {
    public static void main(String[] args) {
        int lado = 3;
        for (int i = 1; i <= lado; i++) {
```

```
        for (int j = 1; j <= lado; j++) {
            if (i == 1 || i == lado || j == 1 || j == lado) {
                System.out.print("@");
            } else {
                System.out.print(" ");
            }
        }
        System.out.println();
    }
  }
}
```

Ejercicio 3.28. Desarrolla un programa que pida un número entero de tamaño byte al usuario y muestre por consola los múltiplos de 7 que haya entre 1 y el número introducido.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_28 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Introduce un número (byte): ");
        byte num = (byte) teclado.nextInt();

        for (int i = 1; i <= num; i++) {
            if (i % 7 == 0) {
                System.out.print(i + " ");
            }
        }

        teclado.close();
    }
}
```

Ejercicio 3.29. Desarrolla un programa que pida al usuario dos números enteros de 32 bits y busque su MCM (mínimo común múltiplo), es decir, que busque el número más bajo que al dividirlo entre los números que introduce el usuario dé de resto 0. Interrumpe la búsqueda cuando lo hayas encontrado.

```
package TercerTema;

import java.util.Scanner;

public class Ejercicio3_29 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce el primer número entero: ");
        int num1 = teclado.nextInt();
```

```
        System.out.print("Introduce el segundo número entero: ");
        int num2 = teclado.nextInt();

        // Determinar el mayor de los dos números sin usar Math.max()
        int mayor;
        if (num1 > num2) {
            mayor = num1;
        } else {
            mayor = num2;
        }

        int mcm = mayor;

        // Bucle para buscar el MCM
        while (true) {
            if (mcm % num1 == 0 && mcm % num2 == 0) {
                break;  // MCM encontrado, salimos del bucle
            }
            mcm++;
        }

        System.out.println("El MCM de " + num1 + " y " + num2 + " es: " + mcm);

        teclado.close();
    }
}
```

Ejercicio 3.30. Desarrolla un programa que muestre por consola los números impares del 85 al 17, excepto los múltiplos de 5. Hazlo con y sin la orden “continue”.

Con continue:

```
package TercerTema;

public class Ejercicio3_30_ConContinue {
    public static void main(String[] args) {
        for (int i = 85; i >= 17; i--) {
            // Si es múltiplo de 5, saltamos a la siguiente iteración
            if (i % 5 == 0) {
                continue;
            }

            // Si es impar, lo mostramos
            if (i % 2 != 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Sin continue:

```
package TercerTema;

public class Ejercicio3_30_SinContinue {
   public static void main(String[] args) {
      for (int i = 85; i >= 17; i--) {
         // Comprobamos si es impar y no múltiplo de 5
         if (i % 2 != 0) {
            if (i % 5 != 0) {
               System.out.print(i + " ");
            }
         }
      }
   }
}
```

EJERCICIOS RESUELTOS DEL TEMA 4

Ejercicio 4.1. Desarrolla un programa que declare un array de tipo entero de 32 bits. Después, pedirá al usuario 4 números y los almacenará dentro del array. Terminará mostrando por consola la media aritmética de los números introducidos.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_1 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      int[] numeros = new int[4];

      System.out.print("Introduce el número 1: ");
      numeros[0] = teclado.nextInt();
      System.out.print("Introduce el número 2: ");
      numeros[1] = teclado.nextInt();
      System.out.print("Introduce el número 3: ");
      numeros[2] = teclado.nextInt();
      System.out.print("Introduce el número 4: ");
      numeros[3] = teclado.nextInt();

      int suma = numeros[0] + numeros[1] + numeros[2] + numeros[3];
      double media = (double) suma / 4;

      System.out.printf("La media aritmética es: %.2f\n", media);
      teclado.close();
   }
}
```

Ejercicio 4.2. Desarrolla un programa que declare un array de números reales de simple precisión (float). Después, pedirá al usuario 5 números y los almacenará dentro del array. Terminará mostrando por consola los números en orden inverso al que se introdujeron.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_2 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      float[] numeros = new float[5];

      System.out.print("Introduce un número real (1): ");
      numeros[0] = teclado.nextFloat();
      System.out.print("Introduce un número real (2): ");
      numeros[1] = teclado.nextFloat();
      System.out.print("Introduce un número real (3): ");
      numeros[2] = teclado.nextFloat();
```

```
        System.out.print("Introduce un número real (4): ");
        numeros[3] = teclado.nextFloat();
        System.out.print("Introduce un número real (5): ");
        numeros[4] = teclado.nextFloat();

        System.out.println("Números en orden inverso:");
        System.out.println(numeros[4]);
        System.out.println(numeros[3]);
        System.out.println(numeros[2]);
        System.out.println(numeros[1]);
        System.out.println(numeros[0]);

        teclado.close();
    }
}
```

Ejercicio 4.3. Desarrolla un programa que declare un array de números enteros de tamaño byte con el número de días que tiene cada mes. Después, mostrará por consola los datos almacenados.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_3 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        byte[] diasDelMes = new byte[12];

        System.out.println("Introduce el número de días de cada mes:");

        diasDelMes[0] = (byte) teclado.nextInt(); // Enero
        diasDelMes[1] = (byte) teclado.nextInt(); // Febrero
        diasDelMes[2] = (byte) teclado.nextInt(); // Marzo
        diasDelMes[3] = (byte) teclado.nextInt(); // Abril
        diasDelMes[4] = (byte) teclado.nextInt(); // Mayo
        diasDelMes[5] = (byte) teclado.nextInt(); // Junio
        diasDelMes[6] = (byte) teclado.nextInt(); // Julio
        diasDelMes[7] = (byte) teclado.nextInt(); // Agosto
        diasDelMes[8] = (byte) teclado.nextInt(); // Septiembre
        diasDelMes[9] = (byte) teclado.nextInt(); // Octubre
        diasDelMes[10] = (byte) teclado.nextInt(); // Noviembre
        diasDelMes[11] = (byte) teclado.nextInt(); // Diciembre

        System.out.println("Días introducidos por mes:");
        System.out.println("Enero: " + diasDelMes[0]);
        System.out.println("Febrero: " + diasDelMes[1]);
        System.out.println("Marzo: " + diasDelMes[2]);
        System.out.println("Abril: " + diasDelMes[3]);
        System.out.println("Mayo: " + diasDelMes[4]);
        System.out.println("Junio: " + diasDelMes[5]);
```

```
        System.out.println("Julio: " + diasDelMes[6]);
        System.out.println("Agosto: " + diasDelMes[7]);
        System.out.println("Septiembre: " + diasDelMes[8]);
        System.out.println("Octubre: " + diasDelMes[9]);
        System.out.println("Noviembre: " + diasDelMes[10]);
        System.out.println("Diciembre: " + diasDelMes[11]);

        teclado.close();
    }
}
```

Ejercicio 4.4. Realiza el ejercicio 4.1 utilizando bucles for y la propiedad length.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_4 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int[] numeros = new int[4];
        int suma = 0;

        for (int i = 0; i < numeros.length; i++) {
            System.out.print("Introduce un número entero: ");
            numeros[i] = teclado.nextInt();
            suma += numeros[i];
        }

        double media = (double) suma / numeros.length;
        System.out.println("La media es: " + media);

        teclado.close();
    }
}
```

Ejercicio 4.5. Realiza el ejercicio 4.2. utilizando bucles for y la propiedad length.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_5 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        float[] numeros = new float[5];

        for (int i = 0; i < numeros.length; i++) {
            System.out.print("Introduce un número real (float): ");
            numeros[i] = teclado.nextFloat();
        }
```

```
        System.out.println("Números en orden inverso:");
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.println(numeros[i]);
        }
        teclado.close();
    }
}
```

Ejercicio 4.6. Realiza el ejercicio 4.3. utilizando bucles for y la propiedad length.

```
package CuartoTema;

public class Ejercicio4_6 {
    public static void main(String[] args) {
        byte[] diasDelMes = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        for (int i = 0; i < diasDelMes.length; i++) {
            System.out.println("Mes " + (i + 1) + " tiene " + diasDelMes[i] + " días.");
        }
    }
}
```

Ejercicio 4.7. Desarrolla un programa que pida al usuario 6 números enteros largos y luego los muestre por consola en orden inverso al que se introdujeron.

```
package CuartoTema;
import java.util.Scanner;

public class Ejercicio4_7 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        long[] numeros = new long[6];

        for (int i = 0; i < numeros.length; i++) {
            System.out.print("Introduce un número largo (" + (i + 1) + "/6): ");
            numeros[i] = teclado.nextLong();
        }

        System.out.println("Números en orden inverso:");
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.println(numeros[i]);
        }
    }
}
```

Ejercicio 4.8. Desarrolla un programa que pida al usuario la longitud de un array de float (es decir, cuántos números quiere introducir). Después, recorrerá el array para calcular la media y mostrarla por consola.

```
package CuartoTema;
import java.util.Scanner;

public class Ejercicio4_8 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      System.out.print("¿Cuántos números quieres introducir?: ");
      int cantidad = teclado.nextInt();

      float[] numeros = new float[cantidad];
      float suma = 0;

      for (int i = 0; i < cantidad; i++) {
         System.out.print("Introduce número " + (i + 1) + ": ");
         numeros[i] = teclado.nextFloat();
         suma += numeros[i];
      }

      float media = suma / cantidad;
      System.out.printf("La media es: %.2f\n", media);
   }
}
```

Ejercicio 4.9. Desarrolla un programa que pida al usuario 10 números reales de doble precisión. Se guardarán esos valores en un array, calculará la media y luego mostrará por consola todos los valores que estén por encima de la media.

```
package CuartoTema;
import java.util.Scanner;

public class Ejercicio4_9 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      double[] valores = new double[10];
      double suma = 0;

      for (int i = 0; i < 10; i++) {
         System.out.print("Introduce número " + (i + 1) + ": ");
         valores[i] = teclado.nextDouble();
         suma += valores[i];
      }

      double media = suma / 10;
      System.out.printf("Media: %.2f\n", media);
      System.out.println("Números por encima de la media:");

      for (int i = 0; i < 10; i++) {
         if (valores[i] > media) {
            System.out.println(valores[i]);
         }
      }
```

```
    }
}
```

Ejercicio 4.10. Desarrolla un programa que declare un array para almacenar 100 nombres. Se pedirá al usuario que introduzca nombres de manera repetitiva hasta que se pulse la tecla "intro" (se representa con una cadena vacía). Después, el programa mostrará los nombres que se han introducido hasta ese momento.

```
package CuartoTema;
import java.util.Scanner;

public class Ejercicio4_10 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String[] nombres = new String[100];
        int contador = 0;

        while (contador < nombres.length) {
            System.out.print("Introduce un nombre (enter para terminar): ");
            String nombre = teclado.nextLine();

            if (nombre.equals("")) {
                break;
            }

            nombres[contador] = nombre;
            contador++;
        }

        System.out.println("Nombres introducidos:");
        for (int i = 0; i < contador; i++) {
            System.out.println(nombres[i]);
        }
    }
}
```

Ejercicio 4.11. Desarrolla un programa que declare un array que guarde 5 palabras. Después pedirá al usuario una palabra que quiera buscar dentro del array. El programa avisará si el dato está o no y, además, en el caso de encontrarse en el array, dirá cuántas veces aparece.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_11 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String[] palabras = { "sol", "luna", "estrella", "nube", "sol" };

        System.out.print("Introduce una palabra para buscar: ");
```

```
        String buscar = teclado.nextLine();

        int contador = 0;
        for (int i = 0; i < palabras.length; i++) {
            if (palabras[i].equalsIgnoreCase(buscar)) {
                contador++;
            }
        }

        if (contador > 0) {
            System.out.println("La palabra está en el array y aparece " + contador + " vez/veces.");
        } else {
            System.out.println("La palabra no se encuentra en el array.");
        }
    }
}
```

Ejercicio 4.12. Desarrolla un programa que declare un array de 5 números enteros cortos. El array estará vacío y le pedirá al usuario que introduzca un número para añadirlo al array.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_12 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        short[] numeros = new short[5];

        for (int i = 0; i < numeros.length; i++) {
            System.out.print("Introduce un número (posición " + i + "): ");
            numeros[i] = (short) teclado.nextInt();
        }

        System.out.println("Contenido final del array:");
        for (short n : numeros) {
            System.out.print(n + " ");
        }
    }
}
```

Ejercicio 4.13. Desarrolla un programa que declare un array para guardar 5 caracteres (char). El array tendrá de inicio 3 posiciones ocupadas con los caracteres ‘r’, ‘s’ y ‘d’. Después le pedirá al usuario que introduzca un nuevo carácter y la posición en la que lo quiere insertar. El programa insertará el nuevo carácter y mostrará el resultado final por consola.

```
package CuartoTema;

import java.util.Scanner;
```

```
public class Ejercicio4_13 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      char[] caracteres = new char[5];
      caracteres[0] = 'r';
      caracteres[1] = 's';
      caracteres[2] = 'd';
      int elementos = 3; // posiciones ocupadas

      System.out.print("Introduce un nuevo carácter: ");
      char nuevo = teclado.nextLine().charAt(0);

      System.out.print("Introduce la posición donde insertarlo (0 a " + elementos + "): ");
      int posicion = teclado.nextInt();

      // Mover los elementos a la derecha desde el final hasta la posición
      for (int i = elementos; i > posicion; i--) {
         caracteres[i] = caracteres[i - 1];
      }

      caracteres[posicion] = nuevo;
      elementos++;

      System.out.println("Array final:");
      for (int i = 0; i < elementos; i++) {
         System.out.print(caracteres[i] + " ");
      }
   }
}
```

Ejercicio 4.14. Desarrolla un programa que declare un array de 5 números enteros de simple precisión. El array tendrá de inicio 4 posiciones ocupadas con los valores que quieras. Después preguntará al usuario qué posición del array quiere borrar. El programa, tras borrar la posición, mostrará el contenido del array por consola.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_14 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      int[] numeros = { 10, 20, 30, 40, 0 }; // el último está libre
      int elementos = 4;

      System.out.print("¿Qué posición deseas borrar? (0 a " + (elementos - 1) + ": ");
      int posicion = teclado.nextInt();

      // Mover los elementos hacia la izquierda desde la posición borrada
      for (int i = posicion; i < elementos - 1; i++) {
         numeros[i] = numeros[i + 1];
```

```
        }

        numeros[elementos - 1] = 0; // opcional, "limpiar" el hueco
        elementos--;

        System.out.println("Contenido del array tras borrar:");
        for (int i = 0; i < elementos; i++) {
            System.out.print(numeros[i] + " ");
        }
    }
}
```

Ejercicio 4.15. Desarrolla un programa que tenga un array de 4 notas (números reales de tipo float). El programa recorrerá el array usando un bucle foreach para **calcular la suma de todas las notas**. Finalmente, mostrará por consola el resultado de la suma.

```
package CuartoTema;

public class Ejercicio4_15 {
    public static void main(String[] args) {
        float[] notas = { 8.5f, 7.2f, 9.0f, 6.4f };
        float suma = 0;

        for (float nota : notas) {
            suma += nota;
        }

        System.out.println("Suma de notas: " + suma);
    }
}
```

Ejercicio 4.16. Desarrolla un programa que tenga un array con 4 nombres de colores (String). Después, se pedirá al usuario que escriba un color. Usando un bucle foreach, el programa buscará si ese color se encuentra en el array (ignorando mayúsculas y minúsculas). Si lo encuentra, dirá **"Color encontrado"**; si no, dirá **"Color no encontrado"**.

```
package CuartoTema;

public class Ejercicio4_16 {
    public static void main(String[] args) {
        String[] colores = { "rojo", "verde", "azul", "amarillo" };
        String buscado = "azul";
        boolean encontrado = false;

        for (String color : colores) {
            if (color.equalsIgnoreCase(buscado)) {
                encontrado = true;
                break;
            }
        }
```

```
        if (encontrado) {
            System.out.println("Color encontrado.");
        } else {
            System.out.println("Color no encontrado.");
        }
    }
}
```

Ejercicio 4.17. Desarrolla un programa que declare un array bidimensional de 2 bloques de 10 números enteros. Después, pedirá al usuario datos para rellenar el array. El programa terminará mostrando el mayor dato que se ha introducido.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_15 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int[][] numeros = new int[2][10]; // 2 bloques, cada uno con 10 números
        int mayor = Integer.MIN_VALUE;

        // Rellenamos el array con datos introducidos por el usuario
        for (int i = 0; i < numeros.length; i++) { // para cada bloque
            for (int j = 0; j < numeros[i].length; j++) { // para cada número en el bloque
                System.out.print("Introduce un número para bloque " + (i + 1) + ", posición " + (j + 1)
+ ": ");
                numeros[i][j] = teclado.nextInt();

                // Comprobamos si es el mayor número hasta el momento
                if (numeros[i][j] > mayor) {
                    mayor = numeros[i][j];
                }
            }
        }

        // Mostramos el mayor número introducido
        System.out.println("El número mayor introducido es: " + mayor);
    }
}
```

Ejercicio 4.18. Desarrolla un programa que pida al usuario 2 números enteros de tamaño byte. Luego declarará un array de 2 arrays con la longitud de los números que introdujo el usuario. Más tarde, se le pedirá al usuario que rellene ambos bloques de números. Finalmente deberá mostrar la media de cada uno de los bloques.

```
package CuartoTema;
```

```
import java.util.Scanner;

public class Ejercicio4_16 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Paso 1: Pedir al usuario las longitudes de cada bloque (dos valores byte)
      System.out.print("Introduce la longitud del primer bloque: ");
      byte longitud1 = (byte) teclado.nextInt();

      System.out.print("Introduce la longitud del segundo bloque: ");
      byte longitud2 = (byte) teclado.nextInt();

      // Paso 2: Declarar un array de arrays con esas longitudes
      int[][] bloques = new int[2][];
      bloques[0] = new int[longitud1];
      bloques[1] = new int[longitud2];

      // Paso 3: Pedir al usuario que rellene ambos bloques
      for (int i = 0; i < bloques.length; i++) {
         System.out.println("Introduce los números del bloque " + (i + 1) + ":");
         for (int j = 0; j < bloques[i].length; j++) {
            System.out.print("Elemento " + (j + 1) + ": ");
            bloques[i][j] = teclado.nextInt();
         }
      }

      // Paso 4: Calcular y mostrar la media de cada bloque
      for (int i = 0; i < bloques.length; i++) {
         int suma = 0;
         for (int j = 0; j < bloques[i].length; j++) {
            suma += bloques[i][j];
         }

         float media = (float) suma / bloques[i].length;
         System.out.println("Media del bloque " + (i + 1) + ": " + media);
      }

      teclado.close();
   }
}
```

Ejercicio 4.19. Declara un array de 6 números enteros. Pide al usuario que introduzca los valores. Después, usando un bucle foreach, muestra el número más pequeño del array.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_19 {
   public static void main(String[] args) {
```

```
        Scanner teclado = new Scanner(System.in);
        int[] numeros = new int[6];

        // Pedir los valores al usuario
        for (int i = 0; i < numeros.length; i++) {
            System.out.print("Introduce el número " + (i + 1) + ": ");
            numeros[i] = teclado.nextInt();
        }

        // Encontrar el menor usando foreach
        int menor = numeros[0];
        for (int numero : numeros) {
            if (numero < menor) {
                menor = numero;
            }
        }

        System.out.println("El número más pequeño es: " + menor);
    }
}
```

Ejercicio 4.20. Declara un array de 5 palabras (String). Pide al usuario que las introduzca una a una. Después, muestra todas las palabras en orden inverso.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_20 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String[] palabras = new String[5];

        // Pedir las palabras
        for (int i = 0; i < palabras.length; i++) {
            System.out.print("Introduce la palabra " + (i + 1) + ": ");
            palabras[i] = teclado.nextLine();
        }

        // Mostrar en orden inverso
        System.out.println("Palabras en orden inverso:");
        for (int i = palabras.length - 1; i >= 0; i--) {
            System.out.println(palabras[i]);
        }
    }
}
```

Ejercicio 4.21. Declara un array bidimensional de 2 filas y 3 columnas con números enteros. Pide al usuario que rellene el array. Después, muestra por consola la suma de cada fila por separado.

```
package CuartoTema;
```

```
import java.util.Scanner;

public class Ejercicio4_21 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      int[][] datos = new int[2][3];

      // Rellenar el array
      for (int fila = 0; fila < 2; fila++) {
         for (int col = 0; col < 3; col++) {
            System.out.print("Introduce el número para fila " + fila + ", columna " + col + ": ");
            datos[fila][col] = teclado.nextInt();
         }
      }

      // Calcular y mostrar suma de cada fila
      for (int fila = 0; fila < 2; fila++) {
         int suma = 0;
         for (int col = 0; col < 3; col++) {
            suma += datos[fila][col];
         }
         System.out.println("Suma de la fila " + fila + ": " + suma);
      }
   }
}
```

Ejercicio 4.22. Declara un array de arrays de tipo float, donde haya 2 bloques de 4 elementos cada uno. Pide al usuario que introduzca los datos. Calcula y muestra la media de cada bloque utilizando un bucle foreach.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_22 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      float[][] datos = new float[2][4];

      // Pedimos los datos
      for (int i = 0; i < datos.length; i++) {
         System.out.println("Introduce los 4 números del bloque " + (i + 1) + ":");
         for (int j = 0; j < datos[i].length; j++) {
            System.out.print("Elemento " + (j + 1) + ": ");
            datos[i][j] = teclado.nextFloat();
         }
      }

      // Calculamos la media de cada bloque usando foreach
      for (int i = 0; i < datos.length; i++) {
```

```
            float suma = 0;
            for (float num : datos[i]) {
                suma += num;
            }
            float media = suma / datos[i].length;
            System.out.println("Media del bloque " + (i + 1) + ": " + media);
        }
    }
}
```

Ejercicio 4.23. Declara un array bidimensional de String (3x2) con nombres de frutas. Pide al usuario que introduzca una palabra. Usando un bucle foreach, di si esa palabra se encuentra en el array (sin importar mayúsculas o minúsculas).

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_23 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String[][] frutas = {
            {"manzana", "pera"},
            {"kiwi", "uva"},
            {"mango", "plátano"}
        };

        System.out.print("Introduce una fruta para buscar: ");
        String buscada = teclado.nextLine();

        boolean encontrada = false;
        for (String[] fila : frutas) {
            for (String fruta : fila) {
                if (fruta.equalsIgnoreCase(buscada)) {
                    encontrada = true;
                }
            }
        }

        if (encontrada) {
            System.out.println("La fruta está en el array.");
        } else {
            System.out.println("La fruta no se encuentra.");
        }
    }
}
```

Ejercicio 4.24. Crea un array de enteros con los valores {10, 20, 30, 40, 50}. Recorre el array primero con for y luego con foreach, mostrando todos los valores por consola en ambos casos.

```
package CuartoTema;

public class Ejercicio4_24 {
   public static void main(String[] args) {
      int[] numeros = {10, 20, 30, 40, 50};

      System.out.println("Con for:");
      for (int i = 0; i < numeros.length; i++) {
         System.out.println(numeros[i]);
      }

      System.out.println("Con foreach:");
      for (int num : numeros) {
         System.out.println(num);
      }
   }
}
```

Ejercicio 4.25. Declara un array de 10 enteros. Pide al usuario que lo rellene. Después, pide un número y muestra cuántas veces aparece ese número en el array.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_25 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      int[] numeros = new int[10];

      // Leer valores
      for (int i = 0; i < numeros.length; i++) {
         System.out.print("Número " + (i + 1) + ": ");
         numeros[i] = teclado.nextInt();
      }

      System.out.print("Número a buscar: ");
      int buscado = teclado.nextInt();

      int contador = 0;
      for (int num : numeros) {
         if (num == buscado) {
            contador++;
         }
      }

      System.out.println("Aparece " + contador + " veces.");
   }
}
```

Ejercicio 4.26. Declara un array de 5 caracteres (char) con los valores 'a', 'e', 'i', 'o', 'u'. Pide al usuario un carácter y una posición (de 0 a 4) y reemplaza el carácter en esa posición. Muestra el array resultante con foreach.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_26 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      char[] vocales = {'a', 'e', 'i', 'o', 'u'};

      System.out.print("Introduce un nuevo carácter: ");
      char nuevo = teclado.next().charAt(0);

      System.out.print("Introduce la posición (0-4): ");
      int posicion = teclado.nextInt();

      if (posicion >= 0 && posicion < vocales.length) {
         vocales[posicion] = nuevo;

         System.out.println("Array actualizado:");
         for (char c : vocales) {
            System.out.print(c + " ");
         }
      } else {
         System.out.println("Posición fuera de rango.");
      }
   }
}
```

Ejercicio 4.27. Declara dos arrays de 4 enteros cada uno. Pide al usuario que introduzca los valores de ambos arrays. El programa debe decir cuántos elementos coinciden en la misma posición.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_27 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      int[] a = new int[4];
      int[] b = new int[4];

      // Leer valores del primer array
      System.out.println("Valores del primer array:");
      for (int i = 0; i < 4; i++) {
         System.out.print("a[" + i + "]: ");
         a[i] = teclado.nextInt();
```

```
        }

        // Leer valores del segundo array
        System.out.println("Valores del segundo array:");
        for (int i = 0; i < 4; i++) {
            System.out.print("b[" + i + "]: ");
            b[i] = teclado.nextInt();
        }

        // Comparar coincidencias
        int coincidencias = 0;
        for (int i = 0; i < 4; i++) {
            if (a[i] == b[i]) {
                coincidencias++;
            }
        }

        System.out.println("Coinciden en la misma posición: " + coincidencias + " elementos.");
    }
}
```

Ejercicio 4.28. Declara un array bidimensional de 3 filas y 10 columnas. Cada fila contendrá la tabla de multiplicar del 2, 3 y 4 respectivamente. Después, muestra las tablas por consola usando foreach.

```
package CuartoTema;

public class Ejercicio4_28 {
    public static void main(String[] args) {
        int[][] tablas = new int[3][10];

        // Llenamos el array con las tablas del 2, 3 y 4
        for (int fila = 0; fila < 3; fila++) {
            for (int col = 0; col < 10; col++) {
                tablas[fila][col] = (fila + 2) * (col + 1);
            }
        }

        // Mostrar las tablas
        int tabla = 2;
        for (int[] fila : tablas) {
            System.out.println("Tabla del " + tabla + ":");
            for (int valor : fila) {
                System.out.print(valor + " ");
            }
            System.out.println();
            tabla++;
        }
    }
}
```

Ejercicio 4.29. Crea un array de 5 números reales que representen temperaturas en grados Celsius. Transforma todas a Fahrenheit y muestra el resultado. Fórmula: F = C × 9/5 + 32.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_29 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      double[] celsius = new double[5];

      // Leer temperaturas
      for (int i = 0; i < celsius.length; i++) {
         System.out.print("Temperatura en Celsius " + (i + 1) + ": ");
         celsius[i] = teclado.nextDouble();
      }

      System.out.println("Temperaturas en Fahrenheit:");
      for (double temp : celsius) {
         double fahrenheit = temp * 9 / 5 + 32;
         System.out.println(fahrenheit);
      }
   }
}
```

Ejercicio 4.30. Declara un array de 6 palabras (String). Pide al usuario que introduzca las palabras. Luego, usando foreach, muestra la palabra más larga de todas.

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_30 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
      String[] palabras = new String[6];

      // Leer palabras
      for (int i = 0; i < palabras.length; i++) {
         System.out.print("Palabra " + (i + 1) + ": ");
         palabras[i] = teclado.nextLine();
      }

      // Buscar la palabra más larga
      String masLarga = palabras[0];
      for (String palabra : palabras) {
         if (palabra.length() > masLarga.length()) {
            masLarga = palabra;
         }
      }
```

```
        System.out.println("La palabra más larga es: " + masLarga);
    }
}
```

Ejercicio 4.31. Desarrolla un programa que muestre un menú con las siguientes opciones:

1. Añadir un valor al array
2. Insertar un valor en una posición
3. Buscar un valor
4. Borrar un valor
5. Mostrar todos los valores
6. Salir

El array tendrá un tamaño fijo de 10 elementos (inicialmente vacío). El programa debe gestionar correctamente los espacios disponibles y no permitir operaciones incorrectas (como insertar fuera de rango o añadir si el array está lleno).

```
package CuartoTema;

import java.util.Scanner;

public class Ejercicio4_31 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int[] datos = new int[10];     // Array fijo de tamaño 10
        int elementos = 0;             // Número actual de elementos válidos

        int opcion;
        do {
            // Menú principal
            System.out.println("\nMENÚ:");
            System.out.println("1. Añadir un valor");
            System.out.println("2. Insertar un valor en una posición");
            System.out.println("3. Buscar un valor");
            System.out.println("4. Borrar un valor");
            System.out.println("5. Mostrar todos los valores");
            System.out.println("6. Salir");
            System.out.print("Elige una opción: ");
            opcion = teclado.nextInt();

            switch (opcion) {
                case 1: // Añadir valor al final
                    if (elementos < datos.length) {
                        System.out.print("Introduce un número para añadir: ");
                        datos[elementos] = teclado.nextInt();
                        elementos++;
```

```
            } else {
                System.out.println("El array está lleno. No se puede añadir más.");
            }
            break;

        case 2: // Insertar valor en una posición
            if (elementos < datos.length) {
                System.out.print("Introduce el valor: ");
                int valor = teclado.nextInt();
                System.out.print("Introduce la posición (0 a " + elementos + "): ");
                int pos = teclado.nextInt();
                if (pos >= 0 && pos <= elementos) {
                    // Desplazar a la derecha
                    for (int i = elementos; i > pos; i--) {
                        datos[i] = datos[i - 1];
                    }
                    datos[pos] = valor;
                    elementos++;
                } else {
                    System.out.println("Posición no válida.");
                }
            } else {
                System.out.println("El array está lleno. No se puede insertar.");
            }
            break;

        case 3: // Buscar un valor
            System.out.print("Introduce el valor a buscar: ");
            int buscado = teclado.nextInt();
            boolean encontrado = false;
            for (int i = 0; i < elementos; i++) {
                if (datos[i] == buscado) {
                    System.out.println("Valor encontrado en la posición: " + i);
                    encontrado = true;
                }
            }
            if (!encontrado) {
                System.out.println("Valor no encontrado.");
            }
            break;

        case 4: // Borrar un valor (la primera aparición)
            System.out.print("Introduce el valor a borrar: ");
            int borrar = teclado.nextInt();
            boolean borrado = false;
            for (int i = 0; i < elementos; i++) {
                if (datos[i] == borrar) {
                    // Desplazar a la izquierda
                    for (int j = i; j < elementos - 1; j++) {
                        datos[j] = datos[j + 1];
                    }
                    elementos--;
```

```
                    borrado = true;
                    System.out.println("Valor borrado.");
                    break;
                }
            }
            if (!borrado) {
                System.out.println("Valor no encontrado. No se ha borrado nada.");
            }
            break;

        case 5: // Mostrar datos
            if (elementos == 0) {
                System.out.println("El array está vacío.");
            } else {
                System.out.print("Contenido del array: ");
                for (int i = 0; i < elementos; i++) {
                    System.out.print(datos[i] + " ");
                }
                System.out.println();
            }
            break;

        case 6:
            System.out.println("Saliendo del programa...");
            break;

        default:
            System.out.println("Opción no válida.");
            break;
        }

    } while (opcion != 6);
  }
}
```

EJERCICIOS RESUELTOS DEL TEMA 5

Ejercicio 5.1. Desarrolla un programa que pida una palabra al usuario y después la muestre en consola separando los caracteres.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_1 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedimos una palabra al usuario
      System.out.print("Introduce una palabra: ");
      String palabra = teclado.nextLine();

      // Recorremos la cadena carácter por carácter
      for (int i = 0; i < palabra.length(); i++) {
         System.out.print(palabra.charAt(i) + " ");
      }

      teclado.close();
   }
}
```

Ejercicio 5.2. Desarrolla un programa que pida una frase al usuario y después la muestre por consola en orden inverso.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_2 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedimos una frase al usuario
      System.out.print("Introduce una frase: ");
      String frase = teclado.nextLine();

      // Mostramos la frase al revés
      for (int i = frase.length() - 1; i >= 0; i--) {
         System.out.print(frase.charAt(i));
      }

      teclado.close();
   }
}
```

Ejercicio 5.3. Desarrolla un programa que pida al usuario su nombre y sus apellidos. Después realizará las siguientes operaciones:

1. Eliminará espacios al principio y al final de cada dato si los hubiera.
2. Unirá el nombre y los apellidos en una sola cadena utilizando concat().
3. Mostrará el nombre completo en mayúsculas y en minúsculas.
4. Mostrará uno a uno los caracteres del nombre completo usando toCharArray().

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_3 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Pedimos nombre y apellidos
      System.out.print("Introduce tu nombre: ");
      String nombre = teclado.nextLine();

      System.out.print("Introduce tus apellidos: ");
      String apellidos = teclado.nextLine();

      // Eliminamos espacios sobrantes
      nombre = nombre.trim();
      apellidos = apellidos.trim();

      // Unimos nombre y apellidos
      String nombreCompleto = nombre.concat(" ").concat(apellidos);

      // Mostramos resultados
      System.out.println("\nNombre completo: " + nombreCompleto);
      System.out.println("En mayúsculas: " + nombreCompleto.toUpperCase());
      System.out.println("En minúsculas: " + nombreCompleto.toLowerCase());

      // Convertimos a array de caracteres y mostramos uno a uno
      System.out.println("Caracteres del nombre completo:");
      char[] caracteres = nombreCompleto.toCharArray();
      for (char c : caracteres) {
         System.out.println(c);
      }
   }
}
```

Ejercicio 5.4. Desarrolla un programa que pida al usuario una frase y una palabra. Después mostrará:

1. Si la frase contiene esa palabra.
2. En qué posición empieza esa palabra (si está).

3. Si la frase comienza con esa palabra.
4. Además, mostrará desde el inicio de la palabra hasta el final de la frase (substring()).

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_4 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      // Entrada de datos
      System.out.print("Escribe una frase: ");
      String frase = teclado.nextLine();

      System.out.print("Escribe una palabra a buscar: ");
      String palabra = teclado.nextLine();

      // Comprobaciones
      System.out.println("\n¿Contiene la frase la palabra? " + frase.contains(palabra));

      int posicion = frase.indexOf(palabra);
      if (posicion != -1) {
         System.out.println("La palabra aparece en la posición: " + posicion);
      } else {
         System.out.println("La palabra no se encuentra en la frase.");
      }

      System.out.println("¿Empieza la frase con esa palabra? " + frase.startsWith(palabra));

      // Substring desde la posición de la palabra
      if (posicion != -1 && posicion < frase.length()) {
         System.out.println("Desde esa posición hasta el final: " + frase.substring(posicion));
      }
   }
}
```

Ejercicio 5.5. Desarrolla un programa que pida al usuario una frase. Luego, aplicará las siguientes operaciones y mostrará el resultado por consola:

1. Elimina los espacios al principio y al final.
2. Convierte toda la frase a mayúsculas.
3. Convierte toda la frase a minúsculas.
4. Muestra el primer carácter.
5. Muestra los primeros 5 caracteres.
6. Cuenta cuántas veces aparece la letra 'a'.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_5 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Escribe una frase: ");
      String frase = teclado.nextLine();

      String fraseSinEspacios = frase.trim();
      String mayusculas = fraseSinEspacios.toUpperCase();
      String minusculas = fraseSinEspacios.toLowerCase();

      System.out.println("\nFrase sin espacios al inicio y al final: " + fraseSinEspacios);
      System.out.println("En mayúsculas: " + mayusculas);
      System.out.println("En minúsculas: " + minusculas);

      if (fraseSinEspacios.length() > 0) {
         System.out.println("Primer carácter: " + fraseSinEspacios.charAt(0));
      }

      if (fraseSinEspacios.length() >= 5) {
         System.out.println("Primeros 5 caracteres: " + fraseSinEspacios.substring(0, 5));
      } else {
         System.out.println("La frase tiene menos de 5 caracteres.");
      }

      // Contar cuántas veces aparece la 'a'
      int contador = 0;
      for (int i = 0; i < fraseSinEspacios.length(); i++) {
         if (fraseSinEspacios.charAt(i) == 'a' || fraseSinEspacios.charAt(i) == 'A') {
            contador++;
         }
      }
      System.out.println("Número de veces que aparece la letra 'a': " + contador);
   }
}
```

Ejercicio 5.6. Desarrolla un programa que pida al usuario 5 frases que se guardarán en un array. Luego pedirá cadenas y las buscará por todo el array para ver si se encuentran. No es necesario saber la posición. El programa terminará cuando el usuario escriba “fin”.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_6 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);
```

```
        String[] frases = new String[5];

        // Guardar 5 frases
        System.out.println("Introduce 5 frases:");
        for (int i = 0; i < frases.length; i++) {
            System.out.print("Frase " + (i + 1) + ": ");
            frases[i] = teclado.nextLine();
        }

        // Búsqueda de cadenas
        String busqueda;
        while (true) {
            System.out.print("\nIntroduce una cadena para buscar (o 'fin' para salir): ");
            busqueda = teclado.nextLine();

            if (busqueda.equalsIgnoreCase("fin")) {
                System.out.println("Programa terminado.");
                break;
            }

            boolean encontrada = false;

            for (String frase : frases) {
                if (frase.toLowerCase().contains(busqueda.toLowerCase())) {
                    encontrada = true;
                    break;
                }
            }

            if (encontrada) {
                System.out.println("La cadena se encuentra en alguna de las frases.");
            } else {
                System.out.println("La cadena NO se encuentra.");
            }
        }
    }
}
```

Ejercicio 5.7. Desarrolla una versión del ejercicio anterior en la que el programa, en caso de encontrar la cadena en el array, avise si se encuentra en la primera posición de alguna de las frases.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_7 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String[] frases = new String[5];
```

```
        // Pedimos 5 frases al usuario
        System.out.println("Introduce 5 frases:");
        for (int i = 0; i < frases.length; i++) {
            System.out.print("Frase " + (i + 1) + ": ");
            frases[i] = teclado.nextLine();
        }

        // Búsqueda de cadenas
        while (true) {
            System.out.print("\nIntroduce una cadena para buscar (o 'fin' para terminar): ");
            String busqueda = teclado.nextLine();

            if (busqueda.equalsIgnoreCase("fin")) {
                System.out.println("Programa finalizado.");
                break;
            }

            boolean encontrada = false;
            boolean alInicio = false;

            for (String frase : frases) {
                String fraseMinuscula = frase.toLowerCase();
                String busquedaMinuscula = busqueda.toLowerCase();

                int posicion = fraseMinuscula.indexOf(busquedaMinuscula);

                if (posicion == 0) {
                    encontrada = true;
                    alInicio = true;
                    break; // Ya hemos encontrado una coincidencia al principio, no necesitamos
seguir
                } else if (posicion > 0) {
                    encontrada = true;
                }
            }

            if (encontrada) {
                if (alInicio) {
                    System.out.println("La cadena se encuentra al inicio de alguna frase.");
                } else {
                    System.out.println("La cadena se encuentra en alguna frase, pero NO al inicio.");
                }
            } else {
                System.out.println("La cadena no se encuentra en ninguna frase.");
            }
        }

        teclado.close();
    }
}
```

Ejercicio 5.8. Pide al usuario una frase y muestra cuántas palabras tiene. Utiliza split(" ").

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_8 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Introduce una frase: ");
      String frase = sc.nextLine();

      // Separamos las palabras por los espacios
      String[] palabras = frase.trim().split(" ");

      System.out.println("Número de palabras: " + palabras.length);
   }
}
```

Ejercicio 5.9. Desarrolla un programa que pida al usuario una frase y la descomponga en palabras. El delimitador será el espacio en blanco. Después, mostrará por consola los fragmentos extraídos en orden inverso.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_9 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Introduce una frase: ");
      String frase = sc.nextLine();

      String[] palabras = frase.trim().split(" ");

      System.out.println("Palabras en orden inverso:");
      for (int i = palabras.length - 1; i >= 0; i--) {
         System.out.println(palabras[i]);
      }
   }
}
```

Ejercicio 5.10. Desarrolla un programa que pida al usuario varios números enteros separados por espacios en blanco (recuerda que los números son caracteres salvo que los conviertas a números). Después se fragmentará la cadena cuando se encuentre un espacio y se guardará cada uno de los números enteros en un array. El programa terminará mostrando por consola la suma de todos los números del array (recuerda que si el número entero no se convierte al tipo int o cualquier otro no podrás operar con él).

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_10 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Introduce varios números separados por espacios: ");
      String entrada = sc.nextLine();

      String[] partes = entrada.trim().split(" ");
      int[] numeros = new int[partes.length];
      int suma = 0;

      // Convertimos cada fragmento a entero y lo sumamos
      for (int i = 0; i < partes.length; i++) {
         numeros[i] = Integer.parseInt(partes[i]);
         suma += numeros[i];
      }

      System.out.println("Suma total: " + suma);
   }
}
```

Ejercicio 5.11. Desarrolla un programa que pida al usuario una frase y sustituya todas las “a” que contenga la cadena por “o”.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_11 {

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Pedimos la frase al usuario
      System.out.print("Introduce una frase: ");
      String frase = sc.nextLine();

      // Creamos un StringBuilder a partir de la frase
      StringBuilder sb = new StringBuilder(frase);

      // Recorremos todos los caracteres
      for (int i = 0; i < sb.length(); i++) {
         // Si el carácter actual es 'a', lo sustituimos por 'o'
         if (sb.charAt(i) == 'a') {
            sb.setCharAt(i, 'o');
         }
      }
```

```
        // Mostramos el resultado
        System.out.println("Frase modificada: " + sb);

        sc.close();
    }
}
```

Ejercicio 5.12. Desarrolla un programa que pida al usuario una palabra. Luego realizará las siguientes operaciones usando la clase StringBuilder:

1. Insertar un asterisco * al principio y al final de la palabra.
2. Reemplazar la primera letra por una #.
3. Eliminar el tercer carácter (posición 2).
4. Mostrar el texto final.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_12 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce una palabra: ");
        String palabra = teclado.nextLine();

        StringBuilder sb = new StringBuilder(palabra);

        // Insertar * al principio y al final
        sb.insert(0, '*');
        sb.insert(sb.length(), '*');

        // Reemplazar la primera letra original (ahora en la posición 1) por #
        sb.replace(1, 2, "#");

        // Eliminar el tercer carácter (posición 2)
        if (sb.length() > 2) {
            sb.delete(2, 3);  // delete elimina desde el índice 2 hasta antes del índice 3
        }

        // Mostrar el resultado
        System.out.println("Resultado final: " + sb.toString());

        teclado.close();
    }
}
```

Ejercicio 5.13. Pide una frase al usuario. Elimina todos los espacios con replaceAll() y muestra la frase sin espacios.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_13 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce una frase: ");
      String frase = teclado.nextLine();

      // Eliminamos todos los espacios
      String sinEspacios = frase.replaceAll(" ", "");

      System.out.println("Frase sin espacios: " + sinEspacios);
   }
}
```

Ejercicio 5.14. Pide al usuario un nombre de archivo. Si termina en ".jpg" o ".png", muestra “Es una imagen”. Usa endsWith().

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_14 {
   public static void main(String[] args) {
      Scanner teclado = new Scanner(System.in);

      System.out.print("Introduce el nombre de un archivo: ");
      String archivo = teclado.nextLine();

      if (archivo.endsWith(".jpg") || archivo.endsWith(".png")) {
         System.out.println("Es una imagen.");
      } else {
         System.out.println("No es una imagen.");
      }
   }
}
```

Ejercicio 5.15. Pide una frase al usuario. Usa StringBuilder para eliminar el primer carácter y el último usando deleteCharAt().

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_15 {
```

```
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
        String frase = teclado.nextLine();

        StringBuilder sb = new StringBuilder(frase);

        if (sb.length() >= 2) {
            sb.deleteCharAt(0); // elimina el primer carácter
            sb.deleteCharAt(sb.length() - 1); // elimina el último carácter
            System.out.println("Frase sin el primer y último carácter: " + sb);
        } else {
            System.out.println("La frase es demasiado corta.");
        }
    }
}
```

Ejercicio 5.16. Pide una frase al usuario. Luego cambia el carácter del medio por un asterisco usando setCharAt() (si es par, cambia el de la izquierda).

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_16 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
        String frase = teclado.nextLine();

        StringBuilder sb = new StringBuilder(frase);

        if (sb.length() > 0) {
            int posicion = (sb.length() % 2 == 0) ? (sb.length() / 2 - 1) : (sb.length() / 2);
            sb.setCharAt(posicion, '*');
            System.out.println("Frase con asterisco en el centro: " + sb);
        } else {
            System.out.println("La frase está vacía.");
        }
    }
}
```

Ejercicio 5.17. Pide una frase al usuario y muestra su versión invertida utilizando reverse().

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_17 {
```

```
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
        String frase = teclado.nextLine();

        StringBuilder sb = new StringBuilder(frase);
        sb.reverse(); // invierte la frase

        System.out.println("Frase invertida: " + sb);
    }
}
```

Ejercicio 5.18. Pide una frase al usuario. Muestra la misma frase en mayúsculas, minúsculas y sin espacios al principio ni al final.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_18 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
        String frase = sc.nextLine();

        System.out.println("En mayúsculas: " + frase.toUpperCase());
        System.out.println("En minúsculas: " + frase.toLowerCase());
        System.out.println("Sin espacios al principio y al final: '" + frase.trim() + "'");

        sc.close();
    }
}
```

Ejercicio 5.19. Pide una palabra y muestra si comienza con “pre” y si termina con “ción”.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_19 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce una palabra: ");
        String palabra = sc.nextLine();

        if (palabra.startsWith("pre")) {
            System.out.println("Comienza con 'pre'");
        } else {
```

```
            System.out.println("No comienza con 'pre'");
        }

        if (palabra.endsWith("ción")) {
            System.out.println("Termina con 'ción'");
        } else {
            System.out.println("No termina con 'ción'");
        }

        sc.close();
    }
}
```

Ejercicio 5.20. Pide una frase y muestra las 3 primeras letras y las 3 últimas.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
        String frase = sc.nextLine();

        if (frase.length() >= 3) {
            String primeras = frase.substring(0, 3);
            String ultimas = frase.substring(frase.length() - 3);
            System.out.println("Primeras 3 letras: " + primeras);
            System.out.println("Últimas 3 letras: " + ultimas);
        } else {
            System.out.println("La frase es demasiado corta.");
        }

        sc.close();
    }
}
```

Ejercicio 5.21. Pide una frase y una palabra. Muestra si la palabra aparece en la frase, sin importar mayúsculas o minúsculas.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_21 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce una frase: ");
```

```
        String frase = sc.nextLine();

        System.out.print("Introduce una palabra: ");
        String palabra = sc.nextLine();

        if (frase.toLowerCase().contains(palabra.toLowerCase())) {
            System.out.println("La palabra está en la frase.");
        } else {
            System.out.println("La palabra NO está en la frase.");
        }

        sc.close();
    }
}
```

Ejercicio 5.22. Desarrolla un programa que simule el juego del ahorcado:

1. El programa tendrá una **palabra oculta** que el jugador debe adivinar.
2. Se mostrará al jugador una versión oculta de la palabra con guiones bajos (_) por cada letra.
3. El jugador irá introduciendo letras, una por una.
4. Si la letra está en la palabra, se revelará en su(s) posición(es) correspondiente(s).
5. Si no está, se contará un fallo.
6. El jugador tendrá un máximo de **6 intentos fallidos**.
7. El juego termina cuando se adivina la palabra o si se superan los 6 fallos.

```
package QuintoTema;

import java.util.Scanner;

public class Ejercicio5_22 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String palabraSecreta = "programacion";
        StringBuilder adivinada = new StringBuilder();

        for (int i = 0; i < palabraSecreta.length(); i++) {
            adivinada.append("_");
        }

        int fallos = 0;
        int intentosMaximos = 6;

        while (fallos < intentosMaximos && adivinada.indexOf("_") != -1) {
            System.out.println("\nPalabra: " + adivinada);
            System.out.print("Introduce una letra: ");
```

```
            String entrada = sc.nextLine();

            char letra = entrada.toLowerCase().charAt(0);
            boolean acierto = false;

            for (int i = 0; i < palabraSecreta.length(); i++) {
                if (palabraSecreta.charAt(i) == letra) {
                    adivinada.setCharAt(i, letra);
                    acierto = true;
                }
            }

            if (!acierto) {
                fallos++;
                System.out.println("Fallaste. Intentos restantes: " + (intentosMaximos - fallos));
            } else {
                System.out.println("¡Has acertado una letra!");
            }
        }

        if (adivinada.indexOf("_") == -1) {
            System.out.println("\n¡Ganaste! La palabra era: " + palabraSecreta);
        } else {
            System.out.println("\n¡Perdiste! La palabra era: " + palabraSecreta);
        }

        sc.close();
    }
}
```

EJERCICIOS RESUELTOS DEL TEMA 6

Ejercicio 6.1. Desarrolla un programa que pida al usuario dos números de doble precisión (tipo double). Después los dividirá y mostrará el resultado por consola. Piensa en los posibles errores que pueden aparecer en el programa y contrólalos con varios bloques "catch".

```
package SextoTema;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejercicio6_1 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         // Pedimos al usuario el primer número
         System.out.print("Introduce el primer número (double): ");
         double num1 = sc.nextDouble();

         // Pedimos el segundo número
         System.out.print("Introduce el segundo número (double): ");
         double num2 = sc.nextDouble();

         // Realizamos la división
         double resultado = num1 / num2;

         // Mostramos el resultado
         System.out.println("Resultado: " + resultado);

      } catch (InputMismatchException e) {
         // Se lanza si el usuario introduce algo que no sea un número
         System.out.println("Error: Debes introducir un número válido.");

      } catch (ArithmeticException e) {
         // Aunque con tipo double no lanza ArithmeticException por división entre 0,
         // se incluye por buenas prácticas si cambiamos el tipo a int en el futuro
         System.out.println("Error: División no válida (quizás por cero).");

      } catch (Exception e) {
         // Captura cualquier otro error no previsto
         System.out.println("Ha ocurrido un error inesperado: " + e.getMessage());
      }

      sc.close();
   }
}
```

Ejercicio 6.2. Pide al usuario un número entero y muéstralo por consola. Si introduce algo que no sea un número, muestra un mensaje de error. Usa un solo bloque catch.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_2 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce un número entero: ");
         int numero = sc.nextInt();
         System.out.println("Has introducido: " + numero);
      } catch (InputMismatchException e) {
         System.out.println("Error: No has introducido un número entero.");
      }

      sc.close();
   }
}
```

Ejercicio 6.3. Pide al usuario un número entero. Después, muestra la tabla de multiplicar de ese número del 1 al 10. Controla que no introduzca letras ni otros símbolos. Usa un solo bloque catch.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_3 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce un número entero: ");
         int num = sc.nextInt();

         for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
         }

      } catch (InputMismatchException e) {
         System.out.println("Error: Debes introducir un número válido.");
      }

      sc.close();
   }
}
```

Ejercicio 6.4. Crea un array de 4 elementos. Pide al usuario un índice (posición) y muestra el valor del array en esa posición. Controla posibles errores. Usa varios bloques catch para diferenciar InputMismatchException e IndexOutOfBoundsException.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_4 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int[] datos = {10, 20, 30, 40};

      try {
         System.out.print("Introduce una posición del array (0 a 3): ");
         int indice = sc.nextInt();
         System.out.println("Valor en la posición " + indice + ": " + datos[indice]);
      } catch (InputMismatchException e) {
         System.out.println("Error: Debes introducir un número entero.");
      } catch (IndexOutOfBoundsException e) {
         System.out.println("Error: Índice fuera de rango.");
      }

      sc.close();
   }
}
```

Ejercicio 6.5. Pide al usuario dos números enteros. Divide el primero entre el segundo. Controla la división por cero y errores de entrada. Usa varios bloques catch y un finally que muestre un mensaje final de agradecimiento.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_5 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce el numerador: ");
         double a = sc.nextDouble();

         System.out.print("Introduce el denominador: ");
         double b = sc.nextDouble();

         double resultado = a / b;
         System.out.println("Resultado: " + resultado);

      } catch (InputMismatchException e) {
         System.out.println("Error: Entrada inválida. Debes introducir números.");
```

```
        } catch (ArithmeticException e) {
            System.out.println("Error: División por cero no permitida.");
        } finally {
            System.out.println("Gracias por usar el programa.");
            sc.close();
        }
    }
}
```

Ejercicio 6.6. Crea un programa que siempre intente ejecutar una operación de suma. Usa el bloque finally para mostrar por consola “Operación completada”, pase lo que pase.

```
package SextoTema;

public class Ejercicio6_6 {
    public static void main(String[] args) {
        try {
            int suma = 10 + 5;
            System.out.println("Resultado de la suma: " + suma);
        } catch (Exception e) {
            System.out.println("Ocurrió un error.");
        } finally {
            System.out.println("Operación completada.");
        }
    }
}
```

Ejercicio 6.7. Declara una variable de tipo String como nula. Intenta usar length() sobre ella y captura el error adecuadamente. Usa un solo bloque catch.

```
package SextoTema;

public class Ejercicio6_7 {
    public static void main(String[] args) {
        String texto = null;

        try {
            System.out.println("Longitud del texto: " + texto.length());
        } catch (NullPointerException e) {
            System.out.println("Error: Estás intentando usar un objeto nulo.");
        }
    }
}
```

Ejercicio 6.8. Pide al usuario que introduzca 5 números separados por espacios. Luego, muestra la suma de esos números. Controla errores de entrada. Usa split, conversión con Integer.parseInt() y control de errores con catch múltiple.

```
package SextoTema;
import java.util.Scanner;
```

```
public class Ejercicio6_8 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Introduce varios números separados por espacios: ");
      String entrada = sc.nextLine();

      try {
         String[] partes = entrada.split(" ");
         int suma = 0;

         for (String p : partes) {
            int n = Integer.parseInt(p);
            suma += n;
         }

         System.out.println("Suma total: " + suma);

      } catch (NumberFormatException e) {
         System.out.println("Error: Uno de los elementos no es un número válido.");
      }

      sc.close();
   }
}
```

Ejercicio 6.9. Desarrolla un programa que pida al usuario dos números enteros. Después los dividirá y mostrará el resultado. Si el usuario introduce un valor no numérico o intenta dividir entre cero, el programa mostrará un mensaje de error y la traza de la excepción.

```
package SextoTema;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejercicio6_9 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce el primer número entero: ");
         int num1 = sc.nextInt();

         System.out.print("Introduce el segundo número entero: ");
         int num2 = sc.nextInt();

         int resultado = num1 / num2;
         System.out.println("Resultado: " + resultado);

      } catch (ArithmeticException e) {
         System.out.println("Error: no se puede dividir entre cero.");
         System.out.println("Mensaje técnico: " + e.getMessage());
```

```
            e.printStackTrace();
        } catch (InputMismatchException e) {
            System.out.println("Error: debes introducir un número entero válido.");
            System.out.println("Mensaje técnico: " + e.getMessage());
            e.printStackTrace();
        }

        sc.close();
    }
}
```

Ejercicio 6.10. Crea un programa que declare un array de 5 enteros. Luego pide al usuario una posición para mostrar el valor almacenado en esa posición. Si el índice está fuera de rango, el programa debe mostrar el mensaje del error y la traza correspondiente.

```
package SextoTema;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejercicio6_10 {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30, 40, 50};
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Introduce una posición: ");
            int indice = sc.nextInt();

            System.out.println("El valor en esa posición es: " + numeros[indice]);

        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: posición fuera del rango válido.");
            System.out.println("Mensaje técnico: " + e.getMessage());
            e.printStackTrace();
        } catch (InputMismatchException e) {
            System.out.println("Error: debes escribir un número entero.");
            System.out.println("Mensaje técnico: " + e.getMessage());
            e.printStackTrace();
        }

        sc.close();
    }
}
```

Ejercicio 6.11. Desarrolla un programa que pida dos números enteros: el dividendo y el divisor. El programa debe asegurarse de que el divisor no sea cero y que ambos valores sean números válidos. Si hay algún error, volverá a pedir ambos valores.

```
package SextoTema;
import java.util.InputMismatchException;
```

```
import java.util.Scanner;

public class Ejercicio6_11 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int dividendo = 0;
      int divisor = 0;
      boolean valido = false;

      while (!valido) {
         try {
            System.out.print("Introduce el dividendo: ");
            dividendo = sc.nextInt();

            System.out.print("Introduce el divisor: ");
            divisor = sc.nextInt();

            int resultado = dividendo / divisor;
            System.out.println("Resultado: " + resultado);
            valido = true;
         } catch (InputMismatchException e) {
            System.out.println("Error: Debes introducir números válidos.");
            sc.nextLine(); // Consumimos la entrada incorrecta (limpiamos el buffer)
         } catch (ArithmeticException e) {
            System.out.println("Error: No se puede dividir entre cero.");
         }
      }

      sc.close();
   }
}
```

Ejercicio 6.12. Desarrolla un programa que tenga un array de nombres. El usuario debe introducir un índice, y el programa mostrará el nombre correspondiente. Si el índice es incorrecto (fuera de rango o no es un número), se mostrará un error y se volverá a pedir el índice.

```
package SextoTema;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejercicio6_12 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String[] nombres = {"Ana", "Luis", "Carlos", "María", "Elena"};
      int indice;
      boolean valido = false;

      while (!valido) {
         try {
            System.out.print("Introduce un índice: ");
```

```
            indice = sc.nextInt();
            System.out.println("Nombre en esa posición: " + nombres[indice]);
            valido = true;
         } catch (InputMismatchException e) {
            System.out.println("Error: Debes introducir un número válido.");
            sc.nextLine(); // Consumimos la entrada incorrecta (limpiamos el buffer)
         } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: El índice está fuera del rango permitido.");
         }
      }

      sc.close();
   }
}
```

Ejercicio 6.13. Desarrolla un programa que pida al usuario su edad. Si la edad no es un número válido o es menor que 0 o mayor que 110, el programa debe lanzar una excepción con throw new IllegalArgumentException("Edad no válida").

```
package SextoTema;

import java.util.Scanner;

public class Ejercicio6_13 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         // Pedimos la edad al usuario
         System.out.print("Introduce tu edad: ");
         String entrada = sc.nextLine(); // Leemos la entrada como texto

         // Intentamos convertir la entrada a número entero
         int edad = Integer.parseInt(entrada);

         // Comprobamos si la edad está en el rango válido
         if (edad < 0 || edad > 110) {
            throw new IllegalArgumentException("Edad no válida.");
         }

         // Si es válida, la mostramos
         System.out.println("Tu edad es: " + edad);

      } catch (NumberFormatException e) {
         // Captura si la conversión a número falla
         System.out.println("Error: Debes introducir un número válido.");
      } catch (IllegalArgumentException e) {
         // Captura si la edad está fuera del rango permitido
         System.out.println("Error: " + e.getMessage());
      } finally {
         // Este bloque se ejecuta siempre
```

```
        System.out.println("Fin del programa.");
    }

    sc.close();
  }
}
```

Ejercicio 6.14. Desarrolla un programa que pida al usuario una contraseña. La contraseña debe cumplir con los siguientes requisitos:

- Tener al menos 8 caracteres.
- Contener al menos un dígito.

Si no cumple con los requisitos, **lanza una excepción** con throw new SecurityException("Contraseña no segura").

```
package SextoTema;

import java.util.Scanner;

public class Ejercicio6_14 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
      // Pedimos una contraseña al usuario
      System.out.print("Introduce una contraseña: ");
      String contrasena = sc.nextLine();

      // Validamos si la longitud es al menos 8
      if (contrasena.length() < 8) {
        throw new SecurityException("Contraseña no segura: debe tener al menos 8
caracteres.");
      }

      // Validamos si contiene al menos un dígito
      boolean contieneDigito = false;
      for (int i = 0; i < contrasena.length(); i++) {
        if (Character.isDigit(contrasena.charAt(i))) {
          contieneDigito = true;
          break;
        }
      }

      if (!contieneDigito) {
        throw new SecurityException("Contraseña no segura: debe contener al menos un
número.");
      }

      // Si pasa todas las validaciones
      System.out.println("Contraseña aceptada.");
```

```
        } catch (SecurityException e) {
            // Captura de la excepción personalizada
            System.out.println("Error: " + e.getMessage());
        } finally {
            // Este bloque se ejecuta siempre
            System.out.println("Validación finalizada.");
        }

        sc.close();
    }
}
```

Ejercicio 6.15. Pide una palabra al usuario y luego una posición. Muestra el carácter en esa posición. Captura excepciones si el número no es válido o está fuera de los límites (StringIndexOutOfBoundsException).

```
package SextoTema;

import java.util.Scanner;

public class Ejercicio6_15 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduce una palabra: ");
            String palabra = sc.nextLine();

            System.out.print("Introduce una posición: ");
            int pos = Integer.parseInt(sc.nextLine());

            // Intentamos mostrar el carácter en esa posición
            char c = palabra.charAt(pos);
            System.out.println("Carácter en la posición " + pos + ": " + c);
        } catch (NumberFormatException e) {
            System.out.println("Error: La posición introducida no es un número válido.");
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Error: La posición está fuera de los límites de la palabra.");
        }
        sc.close();
    }
}
```

Ejercicio 6.16. Pide al usuario dos números enteros y muestra el resultado de su división. Si ocurre cualquier tipo de error (formato, división por cero, etc.), usa **un bloque catch genérico** para capturar cualquier excepción. Luego muestra e.getClass().getSimpleName() y e.getMessage().

```
package SextoTema;
```

```
import java.util.Scanner;

public class Ejercicio6_16 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      try {
         System.out.print("Introduce el primer número entero: ");
         int a = sc.nextInt();

         System.out.print("Introduce el segundo número entero: ");
         int b = sc.nextInt();

         int resultado = a / b;
         System.out.println("Resultado de la división: " + resultado);

      } catch (Exception e) {
         // Captura cualquier excepción y muestra el tipo y mensaje
         System.out.println("Excepción capturada: " + e.getClass().getSimpleName());
         System.out.println("Mensaje: " + e.getMessage());
      }
      sc.close();
   }
}
```

Ejercicio 6.17. Pide al usuario una cadena y luego un índice. Muestra la letra que se encuentra en esa posición de la cadena. Captura cualquier excepción que pueda producirse y muestra el tipo de excepción y su mensaje.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_17 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce una palabra: ");
         String palabra = sc.nextLine();

         System.out.print("Introduce un índice: ");
         int indice = sc.nextInt();

         char letra = palabra.charAt(indice);
         System.out.println("El carácter en la posición " + indice + " es: " + letra);

      } catch (StringIndexOutOfBoundsException e) {
         System.out.println("Índice fuera de los límites: " + e.getMessage());
      } catch (InputMismatchException e) {
         System.out.println("Entrada no válida: " + e.getMessage());
         sc.nextLine(); // Consumimos la entrada incorrecta (limpiamos el buffer)
```

```
      } catch (Exception e) {
         System.out.println("Excepción inesperada: " + e.getClass().getSimpleName() + " - " +
e.getMessage());
      }
   }
}
```

Ejercicio 6.18. Desarrolla un programa que pida al usuario una contraseña dos veces. Si las contraseñas no coinciden, lanza una excepción con throw new IllegalArgumentException("Las contraseñas no coinciden"). Usa un bloque finally para mostrar un mensaje de cierre.

```
package SextoTema;
import java.util.Scanner;

public class Ejercicio6_18 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Introduce la contraseña: ");
         String pass1 = sc.nextLine();

         System.out.print("Repite la contraseña: ");
         String pass2 = sc.nextLine();

         if (!pass1.equals(pass2)) {
            throw new IllegalArgumentException("Las contraseñas no coinciden.");
         }

         System.out.println("Contraseña confirmada correctamente.");

      } catch (IllegalArgumentException e) {
         System.out.println("Error: " + e.getMessage());
      } finally {
         System.out.println("Fin del proceso de validación.");
      }
   }
}
```

Ejercicio 6.19. Solicita al usuario un número entero positivo. Si el número es negativo, lanza una excepción personalizada con un mensaje adecuado. Si el número es correcto, calcula su raíz cuadrada y muéstrala. Controla cualquier error con bloques try-catch.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_19 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```

```
        try {
            System.out.print("Introduce un número entero positivo: ");
            int numero = sc.nextInt();

            if (numero < 0) {
                throw new IllegalArgumentException("El número debe ser positivo.");
            }

            double raiz = Math.sqrt(numero);
            System.out.println("La raíz cuadrada de " + numero + " es: " + raiz);

        } catch (InputMismatchException e) {
            System.out.println("No has introducido un número válido.");
            sc.nextLine(); // Consumimos la entrada incorrecta (limpiamos el buffer)
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Ejercicio 6.20. Pide al usuario un número decimal. Si la entrada no es un número válido, captura la excepción y vuelve a pedir el número hasta que el usuario lo introduzca correctamente. Utiliza un bucle while para repetir hasta tener una entrada válida.

```
package SextoTema;
import java.util.Scanner;
import java.util.InputMismatchException;

public class Ejercicio6_20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        float numero = 0;
        boolean valido = false;

        while (!valido) {
            try {
                System.out.print("Introduce un número decimal: ");
                numero = sc.nextFloat();
                valido = true;
            } catch (InputMismatchException e) {
                System.out.println("Entrada no válida. Intenta de nuevo.");
                sc.nextLine(); // Consumimos la entrada incorrecta (limpiamos el buffer)
            }
        }

        System.out.println("Número válido: " + numero);
    }
}
```

EJERCICIOS RESUELTOS DEL TEMA 7

Ejercicio 7.1. Crea una función llamada "tablaMultiplicar", que muestre la tabla de multiplicar del 5.

```
package SeptimoTema;

public class Ejercicio7_1 {

   public static void tablaMultiplicar() {
      int numero = 5;
      for (int i = 1; i <= 10; i++) {
         System.out.println(numero + " x " + i + " = " + (numero * i));
      }
   }

   public static void main(String[] args) {
      tablaMultiplicar();
   }
}
```

Ejercicio 7.2. Crea una función llamada "tablaMultiplicarUsuario", que muestre la tabla de multiplicar del número que indique el usuario.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_2 {

   public static void tablaMultiplicarUsuario() {
      Scanner scanner = new Scanner(System.in);
      System.out.print("Introduce un número para ver su tabla de multiplicar: ");
      int numero = scanner.nextInt();

      for (int i = 1; i <= 10; i++) {
         System.out.println(numero + " x " + i + " = " + (numero * i));
      }
   }

   public static void main(String[] args) {
      tablaMultiplicarUsuario();
   }
}
```

Ejercicio 7.3. Crea una función llamada "DibujarCuadrado3x3", que dibuje un cuadrado formato por 3 filas con 3 asteriscos cada una.

```
package SeptimoTema;

public class Ejercicio7_3 {
```

```
    public static void DibujarCuadrado3x3() {
        for (int fila = 1; fila <= 3; fila++) {
            for (int col = 1; col <= 3; col++) {
                System.out.print("* ");
            }
            System.out.println(); // Salto de línea al final de cada fila
        }
    }

    public static void main(String[] args) {
        DibujarCuadrado3x3();
    }
}
```

Ejercicio 7.4. Crea una función "dibujarRectanguloHueco" que dibuje en pantalla un rectángulo hueco del ancho y alto que se indiquen como parámetros, formado por una letra que también se indique como parámetro. Completa el programa con un "*main*" que pida esos datos al usuario y dibuje el rectángulo.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_4 {

    // Función que dibuja un rectángulo hueco con el ancho, alto y letra indicados
    public static void dibujarRectanguloHueco(int ancho, int alto, char letra) {
        for (int fila = 1; fila <= alto; fila++) {
            for (int col = 1; col <= ancho; col++) {
                // Dibujar la letra solo en el borde (primera o última fila, o primera o última columna)
                if (fila == 1 || fila == alto || col == 1 || col == ancho) {
                    System.out.print(letra + " ");
                } else {
                    System.out.print("  "); // Espacio en el interior
                }
            }
            System.out.println(); // Salto de línea al final de cada fila
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce el ancho del rectángulo: ");
        int ancho = scanner.nextInt();

        System.out.print("Introduce el alto del rectángulo: ");
        int alto = scanner.nextInt();

        System.out.print("Introduce la letra para dibujar el rectángulo: ");
```

```
        char letra = scanner.next().charAt(0); // Tomar el primer carácter de la entrada

        System.out.println("\nRectángulo hueco:");
        dibujarRectanguloHueco(ancho, alto, letra);
    }
}
```

Ejercicio 7.5. Crea una función "*calcularCubo*" que calcule el cubo de un número real (float) que se indique como parámetro. El resultado deberá ser otro número real. Prueba esta función para calcular el cubo de 3.2 y el de 5.

```
package SeptimoTema;

public class Ejercicio7_5 {

    public static float calcularCubo(float numero) {
        return numero * numero * numero;
    }

    public static void main(String[] args) {
        System.out.println("El cubo de 3.2 es: " + calcularCubo(3.2f));
        System.out.println("El cubo de 5 es: " + calcularCubo(5.0f));
    }
}
```

Ejercicio 7.6. Crea una función "*calcularMenor*" que calcule el menor de dos números enteros que recibirá como parámetros. El resultado será otro número entero.

```
package SeptimoTema;

public class Ejercicio7_6 {

    public static int calcularMenor(int a, int b) {
        return (a < b) ? a : b;
    }

    public static void main(String[] args) {
        int menor = calcularMenor(10, 3);
        System.out.println("El menor número es: " + menor);
    }
}
```

Ejercicio 7.7. Crea una función llamada "*obtenerSigno*", que reciba un número real, y devuelva un número entero con el valor: -1 si el número es negativo, 1 si es positivo o 0 si es cero.

```
package SeptimoTema;

public class Ejercicio7_7 {

    public static int obtenerSigno(double numero) {
```

```
        if (numero > 0) {
            return 1;
        } else if (numero < 0) {
            return -1;
        } else {
            return 0;
        }
    }

    public static void main(String[] args) {
        System.out.println("Signo de -5.7: " + obtenerSigno(-5.7));
        System.out.println("Signo de 0: " + obtenerSigno(0));
        System.out.println("Signo de 8.9: " + obtenerSigno(8.9));
    }
}
```

Ejercicio 7.8. Crea una función "*extraerInicial*", que devuelva la primera letra de una cadena de texto. Prueba esta función para calcular la primera letra de la frase "Hola".

```
package SeptimoTema;

public class Ejercicio7_8 {

    public static char extraerInicial(String texto) {
        return texto.charAt(0);
    }

    public static void main(String[] args) {
        System.out.println("Primera letra de 'Hola': " + extraerInicial("Hola"));
    }
}
```

Ejercicio 7.9. Crea una función "*ultimaLetra*", que devuelva la última letra de una cadena de texto. Prueba esta función para calcular la última letra de la frase "Hola".

```
package SeptimoTema;

public class Ejercicio7_9 {

    public static char ultimaLetra(String texto) {
        return texto.charAt(texto.length() - 1);
    }

    public static void main(String[] args) {
        System.out.println("Última letra de 'Hola': " + ultimaLetra("Hola"));
    }
}
```

Ejercicio 7.10. Crea una función "*mostrarPerimSuperfCuadrado*" que reciba un número entero y calcule y muestre en pantalla el valor del perímetro y de la superficie de un cuadrado que tenga como lado el número que se ha indicado como parámetro.

```
package SeptimoTema;

public class Ejercicio7_10 {

   public static void mostrarPerimSuperfCuadrado(int lado) {
      int perimetro = lado * 4;
      int superficie = lado * lado;
      System.out.println("Perímetro: " + perimetro);
      System.out.println("Superficie: " + superficie);
   }

   public static void main(String[] args) {
      mostrarPerimSuperfCuadrado(6);
   }
}
```

Ejercicio 7.11. Crea una función "saberSiEsPrimo", que reciba un número y devuelva el valor booleano "true" si es un número primo o "false" en caso contrario.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio711 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Introduce un número: ");
      int num = sc.nextInt();

      if (saberSiEsPrimo(num)) {
         System.out.println("El número es primo.");
      } else {
         System.out.println("El número no es primo.");
      }
   }

   public static boolean saberSiEsPrimo(int num) {
      int cont = 0;
      for (int i = 1; i <= num; i++) {
         if (num % i == 0) {
            cont++;
         }
      }
      return cont == 2;
   }
}
```

Ejercicio 7.12. Crea una función "contarLetra", que reciba una cadena y una letra, y devuelva la cantidad de veces que dicha letra aparece en la cadena. Por ejemplo, si la cadena es "Barcelona" y la letra es 'a', debería devolver 2 (porque la "a" aparece 2 veces).

```
package SeptimoTema;

public class Ejercicio7_12 {
   public static void main(String[] args) {
      String texto = "Barcelona";
      char letra = 'a';
      System.out.println("La letra '" + letra + "' aparece " + contarLetra(texto, letra) + " veces.");
   }

   public static int contarLetra(String cadena, char letra) {
      int contador = 0;
      for (int i = 0; i < cadena.length(); i++) {
         if (cadena.charAt(i) == letra) {
            contador++;
         }
      }
      return contador;
   }
}
```

Ejercicio 7.13. Crea una función "sumarCifras" que reciba un numero cualquiera y que devuelva como resultado la suma de sus dígitos. Por ejemplo, si el número fuera 123 la suma sería 6.

```
package SeptimoTema;

public class Ejercicio7_13 {
   public static void main(String[] args) {
      int numero = 123;
      System.out.println("La suma de sus cifras es: " + sumarCifras(numero));
   }

   public static int sumarCifras(int numero) {
      int suma = 0;
      while (numero != 0) {
         suma += numero % 10;
         numero /= 10;
      }
      return suma;
   }
}
```

Ejercicio 7.14. Crea una función "dibujarTriangulo" que reciba una letra y un número, y escriba un "triángulo" formado por esa letra, que tenga como anchura inicial la que se ha indicado. Por ejemplo, si la letra es * y la anchura es 4, debería escribir:

****

***

**

*

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_14 {
	public static void main(String args[]) {
		Scanner sc = new Scanner(System.in);
	  System.out.print("Introduce una letra: ");
	  String letra = sc.next();
	  System.out.print("Introduce un número: ");
	  int num = sc.nextInt();

	  dibujarTriangulo(letra, num);
    }

	public static void dibujarTriangulo(String letra, int num) {
		int unoMenos = num;
		for (int i=0; i<num; i++) {
			for (int j=0; j<unoMenos; j++) {
				System.out.print(letra);
			}
			System.out.println();
			unoMenos--;
		}
	}
}
```

Ejercicio 7.15. Crea una función "modificar", que modifique el valor de dos números enteros que se le indiquen como parámetro.

```
package SeptimoTema;

public class Ejercicio7_15 {
	public static int[] modificar(int n1, int n2, int[] modificados) {
		modificados[0] = n1 + 5;
		modificados[1] = n2 + 5;
		return modificados;
	}

	public static void main(String args[]) {
		int n1 = 5, n2 = 7;
		System.out.println("Los números originales son: " + n1 + " y " + n2);

		int[] modificados = new int[2];
		modificar(n1, n2, modificados);

		System.out.println("Los números modificados son: " + modificados[0] + " y "
+ modificados[1]);
	}
```

```
}
```

Ejercicio 7.16. Crea una función "extraerIniciales", que reciba una cadena como "Antonio Barba" y devuelva las letras A y B (primera letra, y letra situada tras el primer espacio). Utiliza substring().

```
package SeptimoTema;

public class Ejercicio7_16 {
	public static String[] extraerIniciales(String cadena, String[] iniciales) {
		String[] palabras = cadena.split(" ");
		iniciales[0] = palabras[0].substring(0, 1);
		iniciales[1] = palabras[1].substring(0, 1);
		return iniciales;
	}

	public static void main(String args[]) {
		String cadena = "Antonio Barba";
		System.out.println("La cadena es: " + cadena);

		String[] iniciales = new String[2];
		extraerIniciales(cadena, iniciales);

		System.out.println("Las iniciales son: " + iniciales[0] + " y " + iniciales[1]);
	}
}
```

Ejercicio 7.17. Crea una función que calcule el valor de elevar un número entero a otro número entero (por ejemplo, $5^3 = 5 \cdot 5 \cdot 5 = 125$). Esta función se debe crear de forma recursiva. Piensa cuál será el caso base (qué potencia se puede calcular de forma trivial) para salir de la recursión.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_17 {
	public static void main(String args[]) {
		Scanner sc = new Scanner(System.in);
	  System.out.print("Introduce la base: ");
	  int base = sc.nextInt();
	  System.out.print("Introduce el exponente: ");
	  int exp = sc.nextInt();

	  System.out.print("La potencia de " + base + " elevado a " + exp + "
es: " + calcularPotencia(base, exp));
	}

	public static int calcularPotencia(int base, int exp) {
		if (exp==0)
			return 1;
		else {
			return base*calcularPotencia(base,exp-1);
		}
```

```
        }
}
```

Ejercicio 7.18. Crea de forma recursiva una función que diga si una cadena de caracteres es palíndroma (se lee igual de izquierda a derecha que viceversa). Por ejemplo, "reconocer" o “luz azul”.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_18 {
        public static void main(String args[]) {
                Scanner sc = new Scanner(System.in);
          System.out.print("Escribe una frase: ");
          String frase = sc.next();

          System.out.print("¿La frase " + frase + " es un palíndromo?: " +
esPalindromo(frase));
        }

        public static boolean esPalindromo(String frase) {
                if (frase.length() <= 1)
                        return true;
                else {
                        if (frase.charAt(0) != frase.charAt(frase.length() -
1)) {
                                return false;
                        } else {
                                return esPalindromo(frase.substring(1,
frase.length() - 1));
                        }
                }
        }
}
```

Ejercicio 7.19. Crear un programa que encuentre el máximo común divisor de dos números usando el algoritmo de Euclides: Dados dos números enteros positivos m y n, tal que m > n, para encontrar su máximo común divisor, es decir, el mayor entero positivo que divide a ambos:

1. Dividir m por n para obtener el resto r (0 ≤ r< n);
2. Si r = 0, el MCD es n;
3. Si no, el máximo común divisor es MCD (n, r).

```
Package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_19 {
        public static void main(String args[]) {
                Scanner sc = new Scanner(System.in);
          System.out.print("Introduce un número: ");
          int num1 = sc.nextInt();
          System.out.print("Introduce otro número: ");
          int num2 = sc.nextInt();
```

```
        if (num1>num2) {
            System.out.print("El MCD de los números introducidos es: " +
calcularMCD(num1, num2));
        }else {
            System.out.print("El MCD de los números introducidos es: " +
calcularMCD(num2, num1));
        }
    }

    public static int calcularMCD(int mayor, int menor) {
        if (mayor % menor == 0)
            return menor;
        else {
            return calcularMCD(menor, mayor % menor);
        }
    }
}
```

Ejercicio 7.21. Crea una clase con una función que declare 2 variables de tipo entero y realice una división. Desde el método main, solo se llamará a esta función.

```
package SeptimoTema;

public class Ejercicio7_21 {
  public static void realizarDivision() {
    int num1 = 10;
    int num2 = 3;
    int resultado = num1 / num2;
    System.out.println("Resultado de la división: " + resultado);
  }
  public static void main(String[] args) {
    realizarDivision();
  }
}
```

Ejercicio 7.22. Crea una clase llamada OperacionesSimples. Implementa las siguientes funciones:

- sumar(): Declara dos variables enteras llamadas numero1 y numero2. Calcula la suma y muestra el resultado por la consola.
- restar(): Declara dos variables enteras llamadas numero1 y numero2. Calcula la resta y muestra el resultado por la consola.

```
package SeptimoTema;

public class Ejercicio7_22 {
  public static void sumar() {
    int numero1 = 5, numero2 = 7;
    int suma = numero1 + numero2;
    System.out.println("Suma: " + suma);
  }
  public static void restar() {
```

```
        int numero1 = 10, numero2 = 4;
        int resta = numero1 - numero2;
        System.out.println("Resta: " + resta);
    }
    public static void main(String[] args) {
        sumar();
        restar();
    }
}
```

Ejercicio 7.23. Crea una clase que contenga una función que salude por consola. Solicitarás el nombre del usuario desde la función main y lo pasarás como parámetro a esta función.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_23 {
    public static void saludar(String nombre) {
        System.out.println("¡Hola, " + nombre + "!");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce tu nombre: ");
        String nombre = sc.nextLine();
        saludar(nombre);
    }
}
```

Ejercicio 7.24. Crea una clase llamada FigurasGeometricas. Implementa las siguientes funciones:

- calcularAreaCirculo(double radio): Calcula el área de un círculo (π x radio^2) y muestra el resultado en la consola.
- calcularCircunferencia(double radio): Calcula la circunferencia de un círculo ($2 \times \pi \times$ radio) y muestra el resultado en la consola.
- calcularAreaRectangulo(double base, double altura): Calcula el área de un rectángulo y muestra el resultado en la consola.
- calcularPerimetroRectangulo(double base, double altura): Calcula el perímetro de un rectángulo y muestra el resultado en la consola.

Dentro de la función main, pide datos al usuario y envíalos como parámetros a las funciones.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_24 {
    public static void calcularAreaCirculo(double radio) {
        double area = Math.PI * radio * radio;
        System.out.println("Área del círculo: " + area);
    }
```

```
    public static void calcularCircunferencia(double radio) {
        double circunferencia = 2 * Math.PI * radio;
        System.out.println("Circunferencia del círculo: " + circunferencia);
    }
    public static void calcularAreaRectangulo(double base, double altura) {
        double area = base * altura;
        System.out.println("Área del rectángulo: " + area);
    }
    public static void calcularPerimetroRectangulo(double base, double altura) {
        double perimetro = 2 * (base + altura);
        System.out.println("Perímetro del rectángulo: " + perimetro);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce el radio del círculo: ");
        double radio = sc.nextDouble();
        calcularAreaCirculo(radio);
        calcularCircunferencia(radio);

        System.out.print("Introduce la base del rectángulo: ");
        double base = sc.nextDouble();
        System.out.print("Introduce la altura del rectángulo: ");
        double altura = sc.nextDouble();
        calcularAreaRectangulo(base, altura);
        calcularPerimetroRectangulo(base, altura);
    }
}
```

Ejercicio 7.25. Crea una clase llamada Calculadora. Implementa las siguientes funciones:

- sumar(int a, int b): Calcula la suma de dos números enteros y devuelve el resultado.
- restar(int a, int b): Calcula la resta de dos números enteros y devuelve el resultado.
- multiplicar(int a, int b): Calcula el producto de dos números enteros y devuelve el resultado.
- dividir(double a, double b): Calcula el cociente de dos números. Si el divisor es 0, muestra un mensaje de error.

En el método main, solicita al usuario dos números y elige la operación que desea realizar llamando a las funciones. Después, al recibir el resultado de la operación, lo mostrará por la consola.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_25 {
    public static int sumar(int a, int b) {
        return a + b;
    }
```

```
    public static int restar(int a, int b) {
        return a - b;
    }
    public static int multiplicar(int a, int b) {
        return a * b;
    }
    public static double dividir(double a, double b) {
        return a / b;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce el primer número: ");
        int num1 = sc.nextInt();
        System.out.print("Introduce el segundo número: ");
        int num2 = sc.nextInt();

        System.out.println("Elige operación: 1.Sumar 2.Restar 3.Multiplicar 4.Dividir");
        int opcion = sc.nextInt();

        if (opcion == 1) {
            System.out.println("Suma: " + sumar(num1, num2));
        } else if (opcion == 2) {
            System.out.println("Resta: " + restar(num1, num2));
        } else if (opcion == 3) {
            System.out.println("Multiplicación: " + multiplicar(num1, num2));
        } else if (opcion == 4) {
            System.out.println("División: " + dividir(num1, num2));
        } else {
            System.out.println("Opción inválida.");
        }
    }
}
```

Ejercicio 7.26. Crea una clase llamada ConversorUnidades. Implementa las siguientes funciones:

- celsiusAFahrenheit(double celsius): Convierte grados Celsius a Fahrenheit usando la fórmula: F=C×(9/5)+32. Después devolverá el resultado.
- fahrenheitACelsius(double fahrenheit): Convierte grados Fahrenheit a Celsius usando la fórmula: C=(F−32)×5/9. Después devolverá el resultado.
- kilometrosAMillas(double kilometros): Convierte kilómetros a millas usando la fórmula: millas=kilometros×0.621371. Después devolverá el resultado.
- millasAKilometros(double millas): Convierte millas a kilómetros usando la fórmula: kilometros=millas/0.621371. Después devolverá el resultado.
- kilogramosALibras(double kilogramos): Convierte kilogramos a libras usando la fórmula: libras=kilogramos×2.20462. Después devolverá el resultado.
- librasAKilogramos(double libras): Convierte libras a kilogramos usando la fórmula: kilogramos=libras/2.20462. Después devolverá el resultado.

En el método main, solicita al usuario que elija una conversión y un valor (que se pasará como parámetro). Después, mostrará el resultado por la consola.

```
package SeptimoTema;

import java.util.Scanner;

public class Ejercicio7_26 {

   public static double celsiusAFahrenheit(double celsius) {
      return celsius * 9 / 5 + 32;
   }

   public static double fahrenheitACelsius(double fahrenheit) {
      return (fahrenheit - 32) * 5 / 9;
   }

   public static double kilometrosAMillas(double kilometros) {
      return kilometros * 0.621371;
   }

   public static double millasAKilometros(double millas) {
      return millas / 0.621371;
   }

   public static double kilogramosALibras(double kilogramos) {
      return kilogramos * 2.20462;
   }

   public static double librasAKilogramos(double libras) {
      return libras / 2.20462;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.println("Elige la conversión:");
      System.out.println("1. Celsius a Fahrenheit");
      System.out.println("2. Fahrenheit a Celsius");
      System.out.println("3. Kilómetros a Millas");
      System.out.println("4. Millas a Kilómetros");
      System.out.println("5. Kilogramos a Libras");
      System.out.println("6. Libras a Kilogramos");
      System.out.print("Opción: ");
      int opcion = sc.nextInt();

      System.out.print("Introduce el valor a convertir: ");
      double valor = sc.nextDouble();

      double resultado = 0;

      if (opcion == 1) {
```

```
            resultado = celsiusAFahrenheit(valor);
        } else if (opcion == 2) {
            resultado = fahrenheitACelsius(valor);
        } else if (opcion == 3) {
            resultado = kilometrosAMillas(valor);
        } else if (opcion == 4) {
            resultado = millasAKilometros(valor);
        } else if (opcion == 5) {
            resultado = kilogramosALibras(valor);
        } else if (opcion == 6) {
            resultado = librasAKilogramos(valor);
        } else {
            System.out.println("Opción inválida.");
            return;
        }

        System.out.println("Resultado: " + resultado);
    }
}
```

EJERCICIOS RESUELTOS DEL TEMA 8

Ejercicio 8.1. Crea una clase llamada Persona. Esta clase deberá tener un atributo "nombre", de tipo String. También deberá tener un método "setNombre", de tipo void y con un parámetro String, que permita cambiar el valor del nombre. Finalmente, también tendrá un método "saludar", que escribirá en pantalla "Hola, soy " seguido de su nombre. Crea también una clase llamada PruebaPersona. Esta clase deberá contener solo la función main, que creará dos objetos de tipo Persona, le asignará un nombre a cada uno y les pedirá que saluden.

```
public class Persona {
	String nombre;

	public void setNombre(String nuevoNombre) {
		nombre=nuevoNombre;
	}

	public void saludar() {
		System.out.println("Hola, soy " + nombre);
	}
}
```

```
public class PruebaPersona {

	public static void main(String[] args) {
		Persona p1 = new Persona();
		Persona p2 = new Persona();

		p1.setNombre("Pepe");
		p2.setNombre("Juan");

		p1.saludar();
		p2.saludar();
	}
}
```

Ejercicio 8.2. Para guardar información sobre libros, vamos a comenzar por crear una clase llamada "Libro", que contendrá atributos "autor", "titulo", "ubicacion" (todos ellos String) y métodos get y set adecuados para leer su valor y cambiarlo. Prepara también un main en otra clase llamada “pruebaLibro”, que cree un objeto de la clase Libro, dé valores a sus tres atributos y luego los muestre.

```
public class PruebaLibro {

	public static void main(String[] args) {
		Libro l1 = new Libro();
		l1.setAutor("Miguel de Cervantes");
		l1.setTitulo("El Quijote");
		l1.setUbicacion("al fondo a la derecha");

		l1.mostrar();
	}
}
```

```
public class Libro {
```

```
	private String autor;
	private String titulo;
	private String ubicacion;

	String getAutor() {
		return autor;
	}
	void setAutor(String autor) {
		this.autor = autor;
	}
	String getTitulo() {
		return titulo;
	}
	void setTitulo(String titulo) {
		this.titulo = titulo;
	}
	String getUbicacion() {
		return ubicacion;
	}
	void setUbicacion(String ubicacion) {
		this.ubicacion = ubicacion;
	}

	void mostrar() {
		System.out.println("El libro: " + titulo + " de " + autor + "
se encuentra " + ubicacion);
	}
}
```

Ejercicio 8.3. Crea la clase Coche, que tendrá atributos "marca" (texto), "modelo" (texto), "cilindrada" (número entero) y potencia (número real). Crea también la clase PruebaDeCoche, que contendrá la función "main", que creará un objeto de tipo coche, pedirá al usuario su marca, modelo, cilindrada y potencia, y luego mostrará en pantalla el valor de esos datos.

```
import java.util.Scanner;

public class PruebaCoche {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Coche car = new Coche();

		System.out.println("Introduce la marca del coche: ");
		String marca = in.next();
		car.setMarca(marca);

		System.out.println("Introduce el modelo del coche: ");
		String modelo = in.next();
		car.setModelo(modelo);

		System.out.println("Introduce la cilindrada del coche: ");
		int cilindrada = in.nextInt();
		car.setCilindrada(cilindrada);

		System.out.println("Introduce la potencia del coche: ");
		double potencia = in.nextDouble();
		car.setPotencia(potencia);
```

```
            car.mostrar();
        }
}
```

```
public class Coche {
        String marca;
        String modelo;
        int cilindrada;
        double potencia;
        void setMarca(String marca) {
                this.marca = marca;
        }
        void setModelo(String modelo) {
                this.modelo = modelo;
        }
        void setCilindrada(int cilindrada) {
                this.cilindrada = cilindrada;
        }
        void setPotencia(double potencia) {
                this.potencia = potencia;
        }
        void mostrar() {
                System.out.println("La marca es " + marca + " y el modelo " +
modelo + ". Su cilindrada " + cilindrada + " y su potencia " + potencia);
        }
}
```

Ejercicio 8.4. Crea una variante ampliada del ejercicio 8.1. En ella, la clase Persona no cambia. Se creará una nueva clase PersonaInglesa. Esta clase deberá heredar las características de la clase "Persona", y añadir un método "TomarTe", de tipo void, que escribirá en pantalla "Estoy tomando té". Ahora, la clase PruebaPersona creará dos objetos de tipo Persona y uno de tipo PersonaInglesa, les asignará un nombre, les pedirá que saluden y pedirá a la persona inglesa que tome té.

```
public class PersonaInglesa extends Persona {
        public void tomarTe() {
                System.out.println("Estoy tomando té.");
        }
}
```

```
public class Persona {
        String nombre;

        public void setNombre(String nuevoNombre) {
                nombre=nuevoNombre;
        }

        public void saludar() {
                System.out.println("Hola, soy " + nombre);
        }
}
```

```
public class PruebaPersona {
```

```
	public static void main(String[] args) {
		Persona p1 = new Persona();
		Persona p2 = new Persona();
		PersonaInglesa pi = new PersonaInglesa();

		p1.setNombre("Pepe");
		p2.setNombre("Juan");
		pi.setNombre("Will");

		p1.saludar();
		p2.saludar();
		pi.saludar();
		pi.tomarTe();
	}
}
```

Ejercicio 8.5. Amplía el ejercicio 8.2. Crea una clase "Documento", de la que Libro heredará todos sus atributos y métodos. Ahora la clase Libro contendrá sólo un atributo "paginas", número entero, con sus correspondientes get y set.

```
public class Libro extends Documento{
	private int numPaginas;

	int getNumPaginas() {
		return numPaginas;
	}

	void setNumPaginas(int numPaginas) {
		this.numPaginas = numPaginas;
	}
}
```

```
public class Documento {
	private String autor;
	private String titulo;
	private String ubicacion;

	String getAutor() {
		return autor;
	}
	void setAutor(String autor) {
		this.autor = autor;
	}
	String getTitulo() {
		return titulo;
	}
	void setTitulo(String titulo) {
		this.titulo = titulo;
	}
	String getUbicacion() {
		return ubicacion;
	}
	void setUbicacion(String ubicacion) {
		this.ubicacion = ubicacion;
	}
```

```
	void mostrar() {
		System.out.println("El libro: " + titulo + " de " + autor + " 
se encuentra " + ubicacion);
	}
}
```

```
public class PruebaLibro {

	public static void main(String[] args) {
		Libro l1 = new Libro();
		l1.setAutor("Miguel de Cervantes");
		l1.setTitulo("El Quijote");
		l1.setUbicacion("al fondo a la derecha");

		l1.mostrar();
	}
}
```

Ejercicio 8.6. Amplía el proyecto de la clase Coche (ejercicio 8.3). Crea una clase "Vehiculo", de la que heredarán "Coche" y una nueva clase "Moto". La clase Vehiculo contendrá todos los atributos y métodos que antes estaban en Coche, y tanto Coche como Moto heredarán de ella.

```
public class Vehiculo {
	String marca;
	String modelo;
	int cilindrada;
	double potencia;
	void setMarca(String marca) {
		this.marca = marca;
	}
	void setModelo(String modelo) {
		this.modelo = modelo;
	}
	void setCilindrada(int cilindrada) {
		this.cilindrada = cilindrada;
	}
	void setPotencia(double potencia) {
		this.potencia = potencia;
	}
	void mostrar() {
		System.out.println("La marca es " + marca + " y el modelo " + 
modelo + ". Su cilindrada " + cilindrada + " y su potencia " + potencia);
	}
}
```

```
public class Coche extends Vehiculo{

}
```

```
public class Moto extends Vehiculo {

}
```

```
import java.util.Scanner;
```

```java
public class PruebaCoche {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Coche car = new Coche();

		System.out.println("Introduce la marca del coche: ");
		String marca = in.next();
		car.setMarca(marca);

		System.out.println("Introduce el modelo del coche: ");
		String modelo = in.next();
		car.setModelo(modelo);

		System.out.println("Introduce la cilindrada del coche: ");
		int cilindrada = in.nextInt();
		car.setCilindrada(cilindrada);

		System.out.println("Introduce la potencia del coche: ");
		double potencia = in.nextDouble();
		car.setPotencia(potencia);

		car.mostrar();
	}
}
```

Ejercicio 8.7. Desarrolla un programa con una superclase Profesor (clase base o padre) y una subclase ProfesorEmerito (clase hija o derivada). Crea el código para estas clases cumpliendo los siguientes requisitos:

- Como atributos de la superclase tendremos nombre (String), edad (int) y añosConsolidados (int) declarados como protected.
- En la subclase se trabajará con el campo adicional añosEmerito declarado como private.
- Un método de la subclase será double obtenerSalarioBase () que obtendrá el salario base como (925 + añosConsolidados * 33.25 + 47.80 * añosEmerito).

¿Puedes acceder al campo añosConsolidados desde la subclase? ¿Por qué? ¿Qué ocurre si el atributo en la superclase lo declaras private?

```java
public class Profesor {
	protected String nombre;
	protected int edad;
	protected int añosConsolidados;
}
```

```java
public class ProfesorEmerito extends Profesor {
	private int añosEmerito;
	double obtenerSalarioBase() {
		return (925 + (añosConsolidados * 33.25) + (47.8*añosEmerito));
	}
}
```

Ejercicio 8.8. Continua con el ejercicio de la clase Persona añadiendo las siguientes características:

- La clase Persona no cambia.
- La clase PersonaInglesa se modificará para que redefina el método "Saludar", para que escriba en pantalla "Hi, I am " seguido de su nombre.
- Se creará una nueva clase PersonaItaliana, que deberá heredar las características de la clase "Persona", pero redefinir el método "saludar", para que escriba en pantalla "Ciao".
- La clase PruebaPersona deberá contener solo la función main, y creará un objeto de tipo Persona, dos de tipo PersonaInglesa, uno de tipo PersonaItaliana, les asignará un nombre, les pedirá que saluden y pedirá a la persona inglesa que tome té.

```
public class PruebaPersona {

	public static void main(String[] args) {
		Persona p1 = new Persona();
		PersonaInglesa pi1 = new PersonaInglesa();
		PersonaInglesa pi2 = new PersonaInglesa();
		PersonaItaliana pit = new PersonaItaliana();

		p1.setNombre("Pepe");
		pi1.setNombre("John");
		pi2.setNombre("Will");
		pit.setNombre("Giancarlo");

		p1.saludar();
		pi1.saludar();
		pi2.saludar();
		pit.saludar();
		pi1.tomarTe();
		pi2.tomarTe();
	}
}
```

```
public class PersonaItaliana extends Persona{
	public void saludar() {
		System.out.println("Ciao, io sono " + nombre);
	}

}
```

```
public class PersonaInglesa extends Persona {
	public void tomarTe() {
		System.out.println("Estoy tomando té.");
	}
	public void saludar() {
		System.out.println("Hi, I am " + nombre);
	}
}
```

```
public class Persona {
	String nombre;
```

```
	public void setNombre(String nuevoNombre) {
		nombre=nuevoNombre;
	}

	public void saludar() {
		System.out.println("Hola, soy " + nombre);
	}
}
```

Ejercicio 8.9. Continua el ejercicio de la clase Libro. Haz que los atributos de la clase Documento y de la clase Libro sean "protegidos".

```
public class Documento {
	protected String autor;
	protected String titulo;
	protected String ubicacion;

	String getAutor() {
		return autor;
	}
	void setAutor(String autor) {
		this.autor = autor;
	}
	String getTitulo() {
		return titulo;
	}
	void setTitulo(String titulo) {
		this.titulo = titulo;
	}
	String getUbicacion() {
		return ubicacion;
	}
	void setUbicacion(String ubicacion) {
		this.ubicacion = ubicacion;
	}

	void mostrar() {
		System.out.println("El libro: " + titulo + " de " + autor + "
se encuentra " + ubicacion);
	}
}
```

```
public class Libro extends Documento{
	protected int numPaginas;

	int getNumPaginas() {
		return numPaginas;
	}

	void setNumPaginas(int numPaginas) {
		this.numPaginas = numPaginas;
	}
}
```

Ejercicio 8.10. Mejora el ejercicio de la clase Coche, donde todos los atributos serán "protegidos" y los métodos serán "públicos".

```
public class Vehiculo {
	protected String marca;
	protected String modelo;
	protected int cilindrada;
	protected double potencia;
	public void setMarca(String marca) {
		this.marca = marca;
	}
	public void setModelo(String modelo) {
		this.modelo = modelo;
	}
	public void setCilindrada(int cilindrada) {
		this.cilindrada = cilindrada;
	}
	public void setPotencia(double potencia) {
		this.potencia = potencia;
	}
	public void mostrar() {
		System.out.println("La marca es " + marca + " y el modelo " +
modelo + ". Su cilindrada " + cilindrada + " y su potencia " + potencia);
	}
}
```

Ejercicio 8.11. Continua el ejercicio de las clases Persona para que todas ellas contengan constructores. Los constructores de casi todas las clases estarán vacíos, excepto del de PersonaInglesa, que prefijará su nombre a "John". Crea también un constructor alternativo para esta clase que permita escoger cualquier otro nombre.

```
public class Persona {
	String nombre;

	public Persona() { // constructor vacío

	}

	public void setNombre(String nuevoNombre) {
		nombre=nuevoNombre;
	}

	public void saludar() {
		System.out.println("Hola, soy " + nombre);
	}
}
```

```
public class PruebaPersona {

	public static void main(String[] args) {
		Persona p1 = new Persona();
		PersonaInglesa pi1 = new PersonaInglesa();
		PersonaInglesa pi2 = new PersonaInglesa("Will");
		PersonaItaliana pit = new PersonaItaliana();

		p1.setNombre("Pepe");
		pit.setNombre("Giancarlo");

		p1.saludar();
		pi1.saludar();
```

```
            pi2.saludar();
            pit.saludar();
            pi1.tomarTe();
            pi2.tomarTe();
        }
}
```

```
public class PersonaItaliana extends Persona{
        public PersonaItaliana() {

        }
        public void saludar() {
                System.out.println("Ciao, io sono " + nombre);
        }
}
```

```
public class PersonaInglesa extends Persona {

        public PersonaInglesa() {
                nombre="John";
        }
        public PersonaInglesa(String nuevoNombre) {
                nombre = nuevoNombre;
        }
        public void tomarTe() {
                System.out.println("Estoy tomando té.");
        }
        public void saludar() {
                System.out.println("Hi, I am " + nombre);
        }
}
```

Ejercicio 8.12. Continua el ejercicio de la clase Libro. Ahora tendrá un constructor que permitirá dar valores al autor, el título y la ubicación.

```
public class Libro extends Documento{
        protected int numPaginas;

        public   Libro(String   nuevoAutor,   String   nuevoTitulo,   String
nuevaUbicacion) {
                autor = nuevoAutor;
                titulo = nuevoTitulo;
                ubicacion = nuevaUbicacion;
        }

        int getNumPaginas() {
                return numPaginas;
        }

        void setNumPaginas(int numPaginas) {
                this.numPaginas = numPaginas;
        }
}
```

```
public class PruebaLibro {
```

```
	public static void main(String[] args) {
		Libro l1 = new Libro("Miguel de Cervantes", "El Quijote", "al
fondo a la derecha");

		l1.mostrar();
	}
}
```

Ejercicio 8.13. Continua el ejercicio de la clase Coche. Añade un atributo "cantidadDeRuedas" a la clase Vehiculo, junto con sus get y set. El constructor de la clase Coche le dará el valor 4 y el constructor de la clase Moto le dará el valor 2.

```
public class Vehiculo {
	protected String marca;
	protected String modelo;
	protected int cilindrada;
	protected double potencia;
	protected int cantidadDeRuedas;
	public void setMarca(String marca) {
		this.marca = marca;
	}
	public void setModelo(String modelo) {
		this.modelo = modelo;
	}
	public void setCilindrada(int cilindrada) {
		this.cilindrada = cilindrada;
	}
	public void setPotencia(double potencia) {
		this.potencia = potencia;
	}
	public void mostrar() {
		System.out.println("La marca es " + marca + " y el modelo " +
modelo + ". Su cilindrada " + cilindrada + " y su potencia " + potencia);
	}
	public int getCantidadDeRuedas() {
		return cantidadDeRuedas;
	}
	public void setCantidadDeRuedas(int cantidadDeRuedas) {
		this.cantidadDeRuedas = cantidadDeRuedas;
	}
}
```

```
public class Moto extends Vehiculo {

	public Moto() {
		cantidadDeRuedas = 2;
	}
}
```

```
public class Coche extends Vehiculo{

	public Coche() {
		cantidadDeRuedas = 4;
	}
}
```

Ejercicio 8.14. Continúa con el ejercicio de la clase "Persona" y añade un nuevo método Saludar, que reciba un parámetro, que será el texto que debe decir esa persona cuando salude.

```
public class PruebaPersona {

	public static void main(String[] args) {
		Persona p1 = new Persona();
		PersonaInglesa pi1 = new PersonaInglesa();
		PersonaInglesa pi2 = new PersonaInglesa("Will");
		PersonaItaliana pit = new PersonaItaliana();

		p1.setNombre("Pepe");
		pit.setNombre("Giancarlo");

		p1.saludar();
		p1.saludar("¿Qué pasa tío?");
		pi1.saludar();
		pi1.saludar("What´s up dude?");
		pi2.saludar();
		pit.saludar();
		pi1.tomarTe();
		pi2.tomarTe();
	}
}
```

```
public class Persona {
	String nombre;

	public Persona() { // constructor vacío

	}

	public void setNombre(String nuevoNombre) {
		nombre=nuevoNombre;
	}

	public void saludar() {
		System.out.println("Hola, soy " + nombre);
	}

	public void saludar (String saludo) {
		System.out.println(saludo);
	}
}
```

NOTA: El resto de clases permanecen igual.

Ejercicio 8.15. Continúa el ejercicio de la clase Libro. Ahora, la clase Libro tendrá un segundo constructor que permita dar valores al autor y el título, pero no a la ubicación, que tomará el valor por defecto "No detallada". Crea un nuevo objeto para probar este constructor.

```
public class Libro extends Documento{
	protected int numPaginas;

	public	Libro(String	nuevoAutor,	String	nuevoTitulo,	String
nuevaUbicacion) {
		autor = nuevoAutor;
```

```
            titulo = nuevoTitulo;
            ubicacion = nuevaUbicacion;
        }

        public Libro(String nuevoAutor, String nuevoTitulo) {
            autor = nuevoAutor;
            titulo = nuevoTitulo;
            ubicacion = "No detallada";
        }

        int getNumPaginas() {
            return numPaginas;
        }

        void setNumPaginas(int numPaginas) {
            this.numPaginas = numPaginas;
        }
}
```

```
public class PruebaLibro {

        public static void main(String[] args) {
            Libro l1 = new Libro("Miguel de Cervantes", "El Quijote", "al
fondo a la derecha");
            Libro l2 = new Libro ("Bram Stoker", "Drácula");

            l1.mostrar();
            l2.mostrar();
        }
}
```

NOTA: El resto de clases permanecen igual.

Ejercicio 8.16. Continúa el ejercicio de la clase Coche. Crea dos nuevos métodos en la clase Vehiculo: uno llamado Circular, que fijará su "velocidad" (un nuevo atributo) a 50, y otro Circular(v), que fijará su velocidad al valor que se indique como parámetro.

```
import java.util.Scanner;

public class PruebaCoche {

        public static void main(String[] args) {
            Scanner in = new Scanner (System.in);
            Coche car = new Coche();
            Coche car2 = new Coche();

            car2.circular(70);
            car2.circular();

            System.out.println("Introduce la marca del coche: ");
            String marca = in.next();
            car.setMarca(marca);

            System.out.println("Introduce el modelo del coche: ");
            String modelo = in.next();
```

```
            car.setModelo(modelo);

            System.out.println("Introduce la cilindrada del coche: ");
            int cilindrada = in.nextInt();
            car.setCilindrada(cilindrada);

            System.out.println("Introduce la potencia del coche: ");
            double potencia = in.nextDouble();
            car.setPotencia(potencia);

            car.mostrar();
        }
}
```

```
public class Vehiculo {
        protected String marca;
        protected String modelo;
        protected int cilindrada;
        protected double potencia;
        protected int cantidadDeRuedas;
        protected int velocidad;

        public void setMarca(String marca) {
                this.marca = marca;
        }
        public void setModelo(String modelo) {
                this.modelo = modelo;
        }
        public void setCilindrada(int cilindrada) {
                this.cilindrada = cilindrada;
        }
        public void setPotencia(double potencia) {
                this.potencia = potencia;
        }
        public void mostrar() {
                System.out.println("La marca es " + marca + " y el modelo " +
modelo + ". Su cilindrada " + cilindrada + " y su potencia " + potencia);
        }
        public int getCantidadDeRuedas() {
                return cantidadDeRuedas;
        }
        public void setCantidadDeRuedas(int cantidadDeRuedas) {
                this.cantidadDeRuedas = cantidadDeRuedas;
        }
        protected int getVelocidad() {
                return velocidad;
        }
        protected void setVelocidad(int velocidad) {
                this.velocidad = velocidad;
        }
        public void circular() {
                System.out.println("El vehículo va circulando a " + velocidad +
" km/h.");
        }
        public void circular(int v) {
                System.out.println("El vehículo va circulando a " + v + "
km/h.");
        }
```

```
}
```

```
public class Coche extends Vehiculo{

	public Coche() {
		cantidadDeRuedas = 4;
		velocidad = 50;
	}
}
```

Ejercicio 8.17. Crea un proyecto que contenga:

- Una clase Trabajador, cuyo constructor escriba en pantalla "Soy un trabajador".
- Una clase Programador, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy programador".
- Una clase Analista, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy analista".
- Una clase Ingeniero, que derive de Trabajador, cuyo constructor escriba en pantalla "Soy ingeniero".
- Una clase IngenieroInformatico, que derive de Ingeniero, cuyo constructor escriba en pantalla "Soy ingeniero informático".
- Una clase "PruebaDeTrabajadores", que cree un objeto perteneciente a cada una de esas clases.

```
public class PruebaConstructores {

	public static void main(String[] args) {
		Trabajador trabajador = new Trabajador();
		System.out.println();
		Programador programador = new Programador();
		System.out.println();
		Analista analista = new Analista();
		System.out.println();
		Ingeniero ingeniero = new Ingeniero();
		System.out.println();
		IngenieroInformatico ii = new IngenieroInformatico();
	}
}
```

```
public class Trabajador {
	public Trabajador() {
		System.out.println("Soy un trabajador.");
	}
}
```

```
public class Programador extends Trabajador{
	public Programador() {
		System.out.println("Soy un programador.");
	}
}
```

```
public class Analista extends Trabajador{
	public Analista() {
		System.out.println("Soy un analista.");
	}
}
```

```
public class Ingeniero extends Trabajador{
	public Ingeniero() {
		System.out.println("Soy un ingeniero.");
	}
}
```

```
public class IngenieroInformatico extends Ingeniero{
	public IngenieroInformatico() {
		System.out.println("Soy un ingeniero informático.");
	}
}
```

Ejercicio 8.18. Continúa con el proyecto Libro en la que el constructor de Documento escriba en pantalla "Creando documento" y el constructor de Libro escriba en pantalla "Creando libro". Comprueba su funcionamiento.

```
public class PruebaLibro {

	public static void main(String[] args) {
		Libro l1 = new Libro("Miguel de Cervantes", "El Quijote", "al
fondo a la derecha");
		System.out.println();
		Libro l2 = new Libro ("Bram Stoker", "Drácula");

		//l1.mostrar();
		//l2.mostrar();
	}
}
```

```
public class Documento {
	protected String autor;
	protected String titulo;
	protected String ubicacion;

	public Documento() {
		System.out.println("Creando documento.");
	}

	String getAutor() {
		return autor;
	}
	void setAutor(String autor) {
		this.autor = autor;
	}
	String getTitulo() {
		return titulo;
	}
	void setTitulo(String titulo) {
		this.titulo = titulo;
```

```
	}
	String getUbicacion() {
		return ubicacion;
	}
	void setUbicacion(String ubicacion) {
		this.ubicacion = ubicacion;
	}

	void mostrar() {
		System.out.println("El libro: " + titulo + " de " + autor + " 
se encuentra " + ubicacion);
	}
}
```

```
public class Libro extends Documento{
	protected int numPaginas;

	public	Libro(String	nuevoAutor,	String	nuevoTitulo,	String 
nuevaUbicacion) {
		autor = nuevoAutor;
		titulo = nuevoTitulo;
		ubicacion = nuevaUbicacion;
		System.out.println("Creando libro.");
	}

	public Libro(String nuevoAutor, String nuevoTitulo) {
		autor = nuevoAutor;
		titulo = nuevoTitulo;
		ubicacion = "No detallada";
		System.out.println("Creando libro.");
	}

	int getNumPaginas() {
		return numPaginas;
	}

	void setNumPaginas(int numPaginas) {
		this.numPaginas = numPaginas;
	}
}
```

Ejercicio 8.19. Crea una versión alternativa de las clases Vehiculo, Coche y Moto, para que el programa te avise del momento en que se entra a cada constructor. Crea un programa de prueba que defina un coche y una moto, y comprueba su funcionamiento.

```
public class PruebaCoche {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Coche car = new Coche();
		System.out.println();
		Coche car2 = new Coche();
		System.out.println();
		Moto moto = new Moto();
	}
}
```

```java
public class Vehiculo {
	protected String marca;
	protected String modelo;
	protected int cilindrada;
	protected double potencia;
	protected int cantidadDeRuedas;
	protected int velocidad;

	public Vehiculo() {
		System.out.println("Estamos en la clase Vehículo.");
	}
}
```

```java
public class Moto extends Vehiculo {

	public Moto() {
		cantidadDeRuedas = 2;
		System.out.println("Estamos en la clase Moto.");
	}
}
```

```java
public class Coche extends Vehiculo{

	public Coche() {
		cantidadDeRuedas = 4;
		velocidad = 50;
		System.out.println("Estamos en la clase coche.");
	}
}
```

Ejercicio 8.20. Continúa con el ejercicio de la clase Trabajador. Ahora hay que instanciar 3 objetos de cada clase en arrays.

```java
public class PruebaConstructores {

	public static void main(String[] args) {
		Trabajador[] trabajador = new Trabajador[3];
		for (int i=0; i<3; i++) {
			trabajador[i]=new Trabajador();
		}
		System.out.println();
		Programador[] programador = new Programador[3];
		for (int i=0; i<3; i++) {
			programador[i]=new Programador();
		}
		System.out.println();
		Analista[] analista = new Analista[3];
		for (int i=0; i<3; i++) {
			analista[i]=new Analista();
		}
		System.out.println();
		Ingeniero[] ingeniero = new Ingeniero[3];
		for (int i=0; i<3; i++) {
			ingeniero[i]=new Ingeniero();
		}
		System.out.println();
```

```
		IngenieroInformatico[] ii = new IngenieroInformatico[3];
		for (int i=0; i<3; i++) {
			ii[i]=new IngenieroInformatico();
		}
	}
}
```

NOTA: El resto de clases permanecen igual.

Ejercicio 8.21. Amplía el proyecto Libro de modo que permita guardar hasta 1.000 libros. La función *main* mostrará un menú que permita añadir un nuevo libro o ver los datos de los ya existentes.

```
import java.util.Scanner;

public class PruebaLibro {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Libro[] libros = new Libro[1000];
		int contadorLibros = 0, opcion = 0;
		String titulo, autor;

		do {
			System.out.print("MENÚ:\n1)	Añadir\n2)	Mostrar\n3)
Salir\nElige una opción: ");
			opcion = in.nextInt();
			switch (opcion) {
			case 1:
				System.out.print("Introduce el autor: ");
				autor = in.next();
				System.out.print("Introduce el título: ");
				titulo = in.next();
				libros[contadorLibros]=new Libro(autor, titulo);
				contadorLibros++;
				break;
			case 2:
				for (int i=0; i<contadorLibros; i++) {
					libros[i].mostrar();
				}
				break;
			case 3:
				break;
			default:
				System.out.println("No es una opción válida.");
			}
		}
		while (opcion != 3);
		System.out.print("Fin del programa.");
	}
}
```

NOTA: El resto de clases permanecen igual.

Ejercicio 8.22. En el ejercicio de la clase Trabajador, crea un array de trabajadores en el que no sean todos de la misma clase.

```
public class PruebaConstructores {

	public static void main(String[] args) {
		Trabajador[] trabajador = new Trabajador[5];
		trabajador[0] = new Analista();
		for (int i=1; i<3; i++) {
			trabajador[i]=new Ingeniero();
		}
		trabajador[3]= new IngenieroInformatico();
		trabajador[4]=new Programador();
	}
}
```

NOTA: El resto de clases permanecen igual.

Ejercicio 8.23. Amplía el proyecto Libro de modo que permita guardar 1000 documentos de cualquier tipo. A la hora de añadir un documento, se preguntará al usuario si desea guardar un documento genérico o un libro, para usar el constructor adecuado.

```
import java.util.Scanner;

public class PruebaLibro {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Documento[] documentos = new Documento[1000];
		int contadorObjetos = 0, opcion = 0;
		String titulo, autor;

		do {
			System.out.print("MENÚ:\n1)	Añadir\n2)	Mostrar\n3)
Salir\nElige una opción: ");
			opcion = in.nextInt();
			switch (opcion) {
			case 1:
				System.out.print("¿Documento genérico (1) o libro
(2)?: ");
				int opcion2 = in.nextInt();
				if (opcion2==1) {
					documentos[contadorObjetos]=new
Documento();
					contadorObjetos++;
				} else {
					if (opcion2==2) {
						System.out.print("Introduce el autor:
");
						autor = in.next();
						System.out.print("Introduce	el
título: ");
						titulo = in.next();
						documentos[contadorObjetos]=new
Libro(autor, titulo);
						contadorObjetos++;
					} else {
						System.out.print("No	es	una	opción
válida.");
					}
				}
```

```
                        break;
                case 2:
                        for (int i=0; i<contadorObjetos; i++) {
                                documentos[i].mostrar();
                        }
                        break;
                case 3:
                        break;
                default:
                        System.out.println("No es una opción válida.");
                }
        }
        while (opcion != 3);
        System.out.print("Fin del programa.");
    }
}
```

NOTA 1: El resto de clases permanecen igual.

NOTA 2: He cambiado el constructor de la clase Documento para que pueda escribir algo en el método mostrar().

Ejercicio 8.24. Prueba lo estudiado en el ejercicio de la clase Persona. Crea un único array que contenga personas de varios tipos y pídeles que saluden. Los métodos saludar() de la clase PersonaInglesa y PersonaItaliana estarán sobrescritos.

```
public class PruebaPersona {

    public static void main(String[] args) {
        Persona[] personas = new Persona[2];
        personas[0] = new PersonaInglesa();
        personas[1] = new PersonaItaliana();

        personas[0].saludar();
        personas[1].saludar();
    }
}
```

```
public class Persona {
    String nombre;

    public Persona() { // constructor vacío

    }

    public void setNombre(String nuevoNombre) {
        nombre=nuevoNombre;
    }

    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }

    public void saludar (String saludo) {
        System.out.println(saludo);
```

```
	}
}
```

```
public class PersonaItaliana extends Persona{
	public PersonaItaliana() {

	}
	@Override
	public void saludar() {
		System.out.println("Ciao, io sono " + nombre);
	}
}
```

```
public class PersonaInglesa extends Persona {

	public PersonaInglesa() {
		nombre="John";
	}
	public PersonaInglesa(String nuevoNombre) {
		nombre = nuevoNombre;
	}
	public void tomarTe() {
		System.out.println(nombre + " está tomando té.");
	}
	@Override
	public void saludar() {
		System.out.println("Hi, I am " + nombre);
	}
}
```

Ejercicio 8.25. Crea una variante del ejercicio clase Trabajador y derivadas en la que se cree un único array "de trabajadores", que contenga un objeto de cada clase, y exista un método "saludar()" que se sobrescriba en todas las clases hijas.

```
public class PruebaConstructores {

	public static void main(String[] args) {
		Trabajador[] trabajador = new Trabajador[5];
		trabajador[0]= new Analista();
		trabajador[1]= new Ingeniero();
		trabajador[2]= new IngenieroInformatico();
		trabajador[3]=new Programador();
		trabajador [4] = new Trabajador();

		for (int i=0; i<trabajador.length; i++) {
			trabajador[i].saludar();
		}
	}
}
```

```
public class Trabajador {
	public Trabajador() {

	}
```

```
	public void saludar() {
		System.out.println("Hola, soy un trabajador");
	}
}
```

```
public class Analista extends Trabajador{
	public Analista() {

	}
	@Override
	public void saludar() {
		System.out.println("Hola, soy analista.");
	}
}
```

```
public class Ingeniero extends Trabajador{
	public Ingeniero() {

	}
	@Override
	public void saludar() {
		System.out.println("Hola, soy ingeniero.");
	}
}
```

```
public class IngenieroInformatico extends Ingeniero{
	public IngenieroInformatico() {

	}
	@Override
	public void saludar() {
		System.out.println("Hola, soy ingeniero informático.");
	}
}
```

```
public class Programador extends Trabajador{
	public Programador() {

	}
	@Override
	public void saludar() {
		System.out.println("Hola, soy programador.");
	}
}
```

Ejercicio 8.26. Investiga qué hace y dónde se encuentra el método toString(). Ahora amplía el proyecto Libro. Tanto la clase Documento como la clase Libro, tendrán un método toString(), que devuelva una cadena de texto formada por título, autor y ubicación. Crea una clase Articulo, que añada el campo "procedencia". El cuerpo del programa permitirá añadir Artículos o Libros, no documentos genéricos. El método toString() deberá mostrar también el número de páginas

de un libro y la procedencia de un artículo. La opción de mostrar datos llamará a los correspondientes métodos toString().

```
import java.util.Scanner;

public class PruebaLibro {

	public static void main(String[] args) {
		Scanner in = new Scanner (System.in);
		Documento[] documentos = new Documento[1000];
		int contadorObjetos = 0, opcion, numPag;
		String titulo, autor, ubicacion, procedencia;

		do {
			System.out.print("MENÚ:\n1) Añadir un nuevo artículo o
libro\n2) Mostrar base de datos\n3) Salir\nElige una opción: ");
			opcion = in.nextInt();
			switch (opcion) {
			case 1:
				System.out.print("¿Quieres guardar un libro (1) o
un artículo (2)?: ");
				int opcion2 = in.nextInt();
				if (opcion2==1) {
					System.out.print("Introduce el autor: ");
					autor = in.next();
					System.out.print("Introduce el título: ");
					titulo = in.next();
					System.out.print("Introduce la ubicación:
");
					ubicacion = in.next();
					System.out.print("Introduce el número de
páginas: ");
					numPag = in.nextInt();
					documentos[contadorObjetos] = new
Libro(autor, titulo, ubicacion);
					((Libro)
documentos[contadorObjetos]).setNumPaginas(numPag);

					contadorObjetos++;
				} else {
					if (opcion2==2) {
						System.out.print("Introduce el autor:
");
						autor = in.next();
						System.out.print("Introduce el
título: ");
						titulo = in.next();
						System.out.print("Introduce la
ubicación: ");
						ubicacion = in.next();
						System.out.print("Introduce
procedencia: ");
						procedencia = in.next();
						documentos[contadorObjetos]=new
Articulo(autor, titulo, ubicacion);
						((Articulo)
documentos[contadorObjetos]).setProcedencia(procedencia);

						contadorObjetos++;
```

```
                                } else {
                                        System.out.print("No es una opción
válida.");
                                }
                        }
                        break;
                case 2:
                        for (int i=0; i<contadorObjetos; i++) {

        System.out.println(documentos[i].toString());
                        }
                        break;
                case 3:
                        break;
                default:
                        System.out.println("No es una opción válida.");
                }
        }
        while (opcion != 3);
        System.out.print("Fin del programa.");
    }
}
```

```
public class Documento {
    protected String autor;
    protected String titulo;
    protected String ubicacion;

    public Documento() {

    }

    String getAutor() {
        return autor;
    }
    void setAutor(String autor) {
        this.autor = autor;
    }
    String getTitulo() {
        return titulo;
    }
    void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    String getUbicacion() {
        return ubicacion;
    }
    void setUbicacion(String ubicacion) {
        this.ubicacion = ubicacion;
    }

    @Override
    public String toString(){
        return "\nEl libro \"" + titulo + "\" del autor " + autor + "
se encuentra en " + ubicacion;
     }
}
```

```java
public class Libro extends Documento{
	protected int numPaginas;

	public Libro(String nuevoAutor, String nuevoTitulo, String nuevaUbicacion) {
		autor = nuevoAutor;
		titulo = nuevoTitulo;
		ubicacion = nuevaUbicacion;
	}

	public int getNumPaginas() {
		return numPaginas;
	}

	public void setNumPaginas(int numPaginas) {
		this.numPaginas = numPaginas;
	}

	@Override
	public String toString(){
		return super.toString() + " y tiene " + numPaginas + " páginas.";
	}
}
```

```java
public class Articulo extends Documento {

	public Articulo(String nuevoAutor, String nuevoTitulo, String nuevaUbicacion) {
		autor = nuevoAutor;
		titulo = nuevoTitulo;
		ubicacion = nuevaUbicacion;
	}

	String procedencia;

	public String getProcedencia() {
		return procedencia;
	}

	public void setProcedencia(String procedencia) {
		this.procedencia = procedencia;
	}

	@Override
	 public String toString(){
		return super.toString() + " y viene de " + procedencia;
	 }
}
```

NOTA: El método toString() se encuentra en la clase java.lang.Object. **Todas las clases en Java heredan implícitamente de Object**, por lo tanto, cualquier objeto en Java tiene un método toString() disponible, aunque **por defecto no hace nada útil**. Me explico: si tú no sobrescribes el método toString() en tu clase, su implementación por defecto devuelve algo como "NombreDeClase@códigoHash". Así, sobrescribir toString() sirve para que puedas **devolver información personalizada** sobre el objeto: sus atributos, su estado actual, etc.

Ejercicio 8.27. Amplía el ejercicio de la clase Trabajador. Haz que el método "saludar" de todas las clases hijas se apoye en el de la clase "Trabajador".

```
public class PruebaConstructores {

	public static void main(String[] args) {
		Trabajador[] trabajador = new Trabajador[5];
		trabajador[0]= new Analista();
		trabajador[1]= new Ingeniero();
		trabajador[2]= new IngenieroInformatico();
		trabajador[3]=new Programador();
		trabajador [4] = new Trabajador();

		for (int i=0; i<trabajador.length; i++) {
			trabajador[i].saludar();
		}
	}
}
```

```
public class Trabajador {
	public Trabajador() {

	}
	public void saludar() {
		System.out.print("Hola, soy un trabajador");
	}
}
```

```
public class Analista extends Trabajador{
	public Analista() {

	}
	@Override
	public void saludar() {
		super.saludar();
		System.out.println(", trabajo como analista.");
	}
}
```

```
public class Ingeniero extends Trabajador{
	public Ingeniero() {

	}
	@Override
	public void saludar() {
		super.saludar();
		System.out.println(", trabajo como ingeniero.");
	}
}
```

```
public class IngenieroInformatico extends Ingeniero{
	public IngenieroInformatico() {

	}
	@Override
```

```
	public void saludar() {
		super.saludar();
		System.out.println(", trabajo como ingeniero informático.");
	}
}
```

```
public class Programador extends Trabajador{
	public Programador() {

	}
	@Override
	public void saludar() {
		super.saludar();
		System.out.println(", trabajo como programador.");
	}
}
```

Ejercicio 8.28. Ahora amplía el ejercicio de la clase Trabajador y haz que el constructor de todas las clases hijas se apoye en el de la clase "Trabajador".

```
public class PruebaConstructores {

	public static void main(String[] args) {
		Analista analista = new Analista();
	}
}
```

```
public class Trabajador {
	public Trabajador() {
	  System.out.println("Constructor de la clase Trabajador");
	}
}
```

```
public class Analista extends Trabajador{
	public Analista() {
	    super();
	}
}
```

NOTA: La salida sería: Constructor de la clase Trabajador.

Ejercicio 8.29. Utiliza el ejercicio de la clase Trabajador para mostrar la utilidad de la palabra "*this*".

```
public class Trabajador {
	int a;
	int b;
	public Trabajador() {
	     this(10, 20);
	     System.out.println("Dentro del constructor predeterminado \n");
	}
	public Trabajador(int a, int b) {
```

```
            this.a = a;
            this.b = b;
            System.out.println("Dentro del constructor parametrizado" + a +
b);
        }
}
```

```
public class PruebaConstructores {
        public static void main(String[] args) {
                Trabajador trabajador = new Trabajador();
        }
}
```

Ejercicio 8.30. Ejercicio 8.30. Copia y pega el siguiente código en Eclipse:

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce una palabra: ");
        String palabra = sc.next();
        if (palabra == "") {
                System.out.print("Es un intro");
        }
        else {
                System.out.print("NO es un intro");
        }
    }
}
```

Como ves, no hay errores de compilación, pero hay algo que no está bien. Depura el código hasta que detectes el fallo.

Tras depurar el código, tienes que llegar al error en la línea **if (palabra == "")**. El operador == no compara el contenido de dos objetos tipo String, sino si ambos objetos apuntan a la misma dirección de memoria. Aunque dos cadenas tengan el mismo texto, si son objetos distintos, == devuelve false.
Por eso, aunque el usuario pulse intro sin escribir nada, la condición palabra == "" no detectará correctamente la cadena vacía.
En lugar de == deberías usar el método equals() o isEmpty() para comparar las cadenas.

Ejercicio 8.31. Copia y pega el siguiente código en Eclipse:

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce una palabra: ");
        String palabra = sc.next();
        System.out.print("La última letra es: " +
extraerUltimaLetra(palabra));
    }
```

```
    public static String extraerUltimaLetra(String palabra) {
      return palabra.substring(palabra.length());
    }
}
```

Depura el código. ¿Dónde está el error?

El error está en la línea **return palabra.substring(palabra.length()**); porque el índice apunta justo **después del último carácter**, por lo que el resultado es una **cadena vacía**, pero intentar acceder al último carácter **de esta forma está mal** si quieres obtener **una letra concreta**. Tendrías que corregir esa línea y escribir return palabra.substring(palabra.length() - 1);

Ejercicio 8.32. Crea una clase CuentaBancaria con los siguientes atributos **privados**:

- saldo (double)
- titular (String)

Implementa los siguientes métodos **públicos**:

1. depositar(double cantidad): Aumenta el saldo en la cantidad especificada.
2. retirar(double cantidad): Disminuye el saldo solo si hay fondos suficientes.
3. getSaldo(): Retorna el saldo actual.
4. getTitular() y setTitular(String titular): Para obtener y modificar el titular de la cuenta.

Prueba desde la clase main todos los métodos de la clase CuentaBancaria.

```
class CuentaBancaria {
   private double saldo;
   private String titular;

   public void setTitular(String nuevoTitular) {
      titular = nuevoTitular;
   }

   public String getTitular() {
      return titular;
   }

   public void setSaldo(double nuevoSaldo) {
      if (saldo >= 0) {
         saldo = nuevoSaldo;
      } else {
         System.out.println("El saldo no puede ser negativo. Se establece en 0.");
         saldo = 0;
      }
   }

   public double getSaldo() {
      return saldo;
   }
```

```java
    // Método para depositar dinero
    public void depositar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
            System.out.println("Depósito exitoso. Nuevo saldo: " + saldo);
        } else {
            System.out.println("La cantidad a depositar debe ser positiva.");
        }
    }

    // Método para retirar dinero
    public void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            saldo -= cantidad;
            System.out.println("Retiro exitoso. Nuevo saldo: " + saldo);
        } else {
            System.out.println("Fondos insuficientes o cantidad inválida.");
        }
    }

    // Método para mostrar información de la cuenta
    public void mostrarInfo() {
        System.out.println("Titular: " + titular);
        System.out.println("Saldo actual: " + saldo);
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        // Instancia una cuenta bancaria
        CuentaBancaria cuenta = new CuentaBancaria();

        // Establecer valores con setters
        cuenta.setTitular("Juan Pérez");
        cuenta.setSaldo(1000);

        // Mostrar información de la cuenta
        cuenta.mostrarInfo();

        // Realizar transacciones
        cuenta.depositar(500);
        cuenta.retirar(300);
        cuenta.retirar(2000); // Intento de retiro fallido

        // Mostrar saldo final
        cuenta.mostrarInfo();
    }
}
```

Ejercicio 8.33. Crea una clase Estudiante que tenga los siguientes atributos **privados**:

- nombre (String)
- edad (int)
- calificacion (double)

Implementa los siguientes métodos **públicos**:

1. getNombre() y setNombre(String nombre): Para obtener y modificar el nombre.
2. getEdad() y setEdad(int edad): La edad solo puede ser mayor o igual a 0.
3. getCalificacion() y setCalificacion(double calificacion): Debe estar entre **0 y 10**.
4. mostrarInfo(): Imprime los datos del estudiante.

Prueba desde la clase main todos los métodos de la clase Estudiante.

```
class Estudiante {
   private String nombre;
   private int edad;
   private double calificacion;

   public void setNombre(String nuevoNombre) {
      nombre = nuevoNombre;
   }

   public String getNombre() {
      return nombre;
   }

   public void setEdad(int nuevoValor) {
      edad = nuevoValor;
   }

   public int getEdad() {
      return edad;
   }

   public void setCalificacion(double nuevoValor) {
      calificacion = nuevoValor;
   }

   public double getCalificacion() {
      return calificacion;
   }

   public void mostrarInfo() {
      System.out.println("Nombre: " + nombre);
      System.out.println("Edad: " + edad);
      System.out.println("Calificación: " + calificacion);
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      // Instanciar un estudiante
      Estudiante estudiante1 = new Estudiante();

      // Establecer valores usando setters
      estudiante1.setNombre("Carlos Pérez");
      estudiante1.setEdad(18);
      estudiante1.setCalificacion(9.2);

      // Mostrar información
      estudiante1.mostrarInfo();
   }
}
```

Ejercicio 8.34. Crea una clase base Figura con un método dibujar() que imprima "Dibujando una figura genérica".

Crea dos subclases:

- Circulo, que sobrescriba dibujar() e imprima "Dibujando un círculo".
- Cuadrado, que sobrescriba dibujar() e imprima "Dibujando un cuadrado".

En main(), crea un objeto de Circulo y otro de Cuadrado, y llama a sus métodos dibujar() individualmente.

```
class Figura {
   void dibujar() {
      System.out.println("Dibujando una figura genérica");
   }
}
```

```
class Circulo extends Figura {
   void dibujar() {
      System.out.println("Dibujando un círculo");
   }
}
```

```
class Cuadrado extends Figura {
   void dibujar() {
      System.out.println("Dibujando un cuadrado");
   }
}
```

```
public class Main {
   public static void main(String[] args) {
```

```
        Figura f1 = new Circulo();
        f1.dibujar();

        Figura f2 = new Cuadrado();
        f2.dibujar();
    }
}
```

Ejercicio 8.35. Crea una clase base Empleado con un método calcularSalario() que imprima "Salario base de un empleado".

Crea dos subclases:

- EmpleadoTiempoCompleto, que sobrescriba calcularSalario() e imprima "Salario de tiempo completo: 1000 €".
- EmpleadoMedioTiempo, que sobrescriba calcularSalario() e imprima "Salario de medio tiempo: 500 €".

En main(), crea un objeto de EmpleadoTiempoCompleto y otro de EmpleadoMedioTiempo, y llama a sus métodos calcularSalario() individualmente.

```
class Empleado {
    void calcularSalario() {
        System.out.println("Salario base de un empleado");
    }
}
```

```
class EmpleadoTiempoCompleto extends Empleado {
    void calcularSalario() {
        System.out.println("Salario de tiempo completo: 1000 €");
    }
}
```

```
class EmpleadoMedioTiempo extends Empleado {
    void calcularSalario() {
        System.out.println("Salario de medio tiempo: 500 €");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Empleado e1 = new EmpleadoTiempoCompleto();
        e1.calcularSalario();

        Empleado e2 = new EmpleadoMedioTiempo();
        e2.calcularSalario();
    }
}
```

Ejercicio 8.36. Crea un sistema para gestionar productos en una tienda, utilizando modificadores de acceso. Para ello:

1. Crea una clase base Producto que tenga los siguientes atributos:
   - nombre (String) **private**
   - precio (double) **private**
   - cantidad (int) **protected**
   - codigo (String) **default (sin modificador)**

   Métodos en Producto:
   - Un método **public** mostrarInfo() que imprima el nombre y el precio del producto.
   - Un método **private** calcularDescuento() que calcule un 5% de descuento sobre el precio.
2. Crea una subclase ProductoElectronico que herede de Producto y tenga:
   - Un atributo adicional marca (String) **private**.
   - Un método **public** mostrarInfoElectronico() que imprima la marca y el código del producto.
3. Crea una clase Tienda con:
   - Un atributo **default** totalProductos (int) que llevará el conteo de productos en la tienda.
   - Un método **public** agregarProducto(Producto producto) que aumente el número de productos en la tienda.
   - Un método **protected** mostrarTotalProductos() que imprima la cantidad de productos en la tienda.
4. En main(), crea instancias de Producto y ProductoElectronico, usa los métodos set para asignar valores, y muestra la información.

```
class Producto {
   private String nombre;
   private double precio;
   protected int cantidad;
   String codigo;

   public void setNombre(String nuevoNombre) {
      nombre = nuevoNombre;
   }

   public String getNombre() {
      return nombre;
   }
```

```
    public void setPrecio(double nuevoPrecio) {
        precio = nuevoPrecio;
    }

    public double getPrecio() {
        return precio;
    }

    public void setCantidad(int nuevaCantidad) {
        cantidad = nuevaCantidad;
    }

    public int getCantidad() {
        return cantidad;
    }

    public void setCodigo(String nuevoCodigo) {
        codigo = nuevoCodigo;
    }

    public String getCodigo() {
        return codigo;
    }

    public void mostrarInfo() {
        System.out.println("Producto: " + nombre + ", Precio: " + precio);
    }

    private void calcularDescuento() {
        double descuento = precio * 0.05; // 5% descuento
        System.out.println("Descuento: " + descuento);
    }
}
```

```
class ProductoElectronico extends Producto {
    private String marca;

    public void setMarca(String nuevaMarca) {
        marca = nuevaMarca;
    }

    public String getMarca() {
        return marca;
    }

    public void mostrarInfoElectronico() {
        mostrarInfo(); // Llamando a mostrarInfo de la clase base
        System.out.println("Marca: " + marca + ", Código: " + codigo);
    }
}
```

```
class Tienda {
   int totalProductos;

   public void agregarProducto(Producto producto) {
      totalProductos++;
      System.out.println("Producto agregado: " + producto.getNombre());
   }

   protected void mostrarTotalProductos() {
      System.out.println("Total productos en la tienda: " + totalProductos);
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Producto producto1 = new Producto();
      producto1.setNombre("Camiseta");
      producto1.setPrecio(19.99);
      producto1.setCantidad(10);
      producto1.setCodigo("C123");

      ProductoElectronico productoElectronico1 = new ProductoElectronico();
      productoElectronico1.setNombre("Laptop");
      productoElectronico1.setPrecio(899.99);
      productoElectronico1.setCantidad(5);
      productoElectronico1.setCodigo("L456");
      productoElectronico1.setMarca("Dell");

      producto1.mostrarInfo();
      productoElectronico1.mostrarInfoElectronico();

      Tienda tienda = new Tienda();
      tienda.agregarProducto(producto1);
      tienda.agregarProducto(productoElectronico1);
      tienda.mostrarTotalProductos();
   }
}
```

Ejercicio 8.37. Crea un sistema para gestionar cuentas bancarias, utilizando modificadores de acceso. Para ello:

1. Crea una clase base CuentaBancaria que tenga como atributos:
   - numeroCuenta (String) **private**
   - saldo (double) **private**
   - titular (String) **protected**
   - tipoCuenta (String) **default (sin modificador)**

   Métodos en CuentaBancaria:

- Un método **public** mostrarInformacion() que imprima el titular, el número de cuenta y el saldo.
- Un método **private** calcularIntereses() que calcule un 2% de intereses sobre el saldo y lo imprima.

2. Crea una subclase CuentaAhorro que herede de CuentaBancaria y tenga:
   - Un atributo adicional intereses (double) **private**.
   - Un método **public** mostrarInformacionCuentaAhorro() que imprima el tipo de cuenta y los intereses.
3. Crea una clase Banco con:
   - Un atributo **default** totalCuentas (int) que llevará el conteo de cuentas en el banco.
   - Un método **public** agregarCuenta(CuentaBancaria cuenta) que aumente el número de cuentas en el banco.
   - Un método **protected** mostrarTotalCuentas() que imprima la cantidad de cuentas en el banco.
4. En main(), crea instancias de CuentaBancaria y CuentaAhorro, usa los métodos set para asignar valores, y muestra la información.

```
class CuentaBancaria {
   private String numeroCuenta;
   private double saldo;
   protected String titular;
   String tipoCuenta;

   public void setNumeroCuenta(String nuevoNumeroCuenta) {
      numeroCuenta = nuevoNumeroCuenta;
   }

   public String getNumeroCuenta() {
      return numeroCuenta;
   }

   public void setSaldo(double nuevoSaldo) {
      saldo = nuevoSaldo;
   }

   public double getSaldo() {
      return saldo;
   }

   public void setTitular(String nuevoTitular) {
      titular = nuevoTitular;
   }

   public String getTitular() {
```

```
        return titular;
    }

    public void setTipoCuenta(String nuevoTipoCuenta) {
        tipoCuenta = nuevoTipoCuenta;
    }

    public String getTipoCuenta() {
        return tipoCuenta;
    }

    public void mostrarInformacion() {
        System.out.println("Titular: " + titular + ", Número de Cuenta: " + numeroCuenta + ",
Saldo: " + saldo);
    }

    private void calcularIntereses() {
        double intereses = saldo * 0.02; // 2% de intereses
        System.out.println("Intereses calculados: " + intereses);
    }
}
```

```
class CuentaAhorro extends CuentaBancaria {
    private double intereses;

    public void setIntereses(double nuevosIntereses) {
        intereses = nuevosIntereses;
    }

    public double getIntereses() {
        return intereses;
    }

    public void mostrarInformacionCuentaAhorro() {
        mostrarInformacion();  // Llamando a mostrarInformacion de la clase base
        System.out.println("Tipo de cuenta: " + tipoCuenta + ", Intereses: " + intereses + "%");
    }
}
```

```
class Banco {
    int totalCuentas;

    public void agregarCuenta(CuentaBancaria cuenta) {
        totalCuentas++;
        System.out.println("Cuenta agregada: " + cuenta.getNumeroCuenta());
    }

    protected void mostrarTotalCuentas() {
        System.out.println("Total cuentas en el banco: " + totalCuentas);
    }
```

```
}
```

```
public class Main {
   public static void main(String[] args) {
      CuentaBancaria cuenta1 = new CuentaBancaria();
      cuenta1.setNumeroCuenta("123456789");
      cuenta1.setSaldo(1000.0);
      cuenta1.setTitular("Juan Pérez");
      cuenta1.setTipoCuenta("Corriente");

      CuentaAhorro cuentaAhorro1 = new CuentaAhorro();
      cuentaAhorro1.setNumeroCuenta("987654321");
      cuentaAhorro1.setSaldo(5000.0);
      cuentaAhorro1.setTitular("María Gómez");
      cuentaAhorro1.setTipoCuenta("Ahorro");
      cuentaAhorro1.setIntereses(3.0);

      cuenta1.mostrarInformacion();
      cuentaAhorro1.mostrarInformacionCuentaAhorro();

      Banco banco = new Banco();
      banco.agregarCuenta(cuenta1);
      banco.agregarCuenta(cuentaAhorro1);
      banco.mostrarTotalCuentas();
   }
}
```

Ejercicio 8.38. Vamos a gestionar los productos de una tienda. Para ello, vamos a crear una clase **Producto** que tenga los atributos: nombre, precio y cantidad. Usaremos **constructores** para inicializar estos atributos al crear instancias de la clase. Además, crearemos una subclase **ProductoElectronico** que herede de **Producto** y agregue un atributo adicional para la **marca**. También, incluiremos un método para mostrar la información del producto. Así:

1. **Producto**:
   - Atributos:
     - nombre (String)
     - precio (double)
     - cantidad (int)
   - Constructor que reciba nombre, precio y cantidad como parámetros y los inicialice.
   - Método **mostrarInformacion()** que imprima los detalles del producto.
2. **ProductoElectronico** (subclase de Producto):
   - Atributo adicional: marca (String)
   - Constructor que reciba nombre, precio, cantidad, y marca como parámetros.

- Método **mostrarInformacionElectronico()** que imprima los detalles del producto electrónico.

3. **Tienda**:
   - Atributo: productos (un arreglo de tipo Producto de tamaño fijo).
   - Método **agregarProducto()** que reciba un objeto de tipo Producto y lo agregue al arreglo.
   - Método **mostrarProductos()** que imprima todos los productos de la tienda.
4. En el main(), crea instancias de Producto y ProductoElectronico usando los constructores, agrega los productos a la tienda, y muestra la información.

```
class Producto {
   private String nombreProducto;
   private double precioProducto;
   private int cantidadProducto;

   public Producto(String nombre, double precio, int cantidad) {
      nombreProducto = nombre;
      precioProducto = precio;
      cantidadProducto = cantidad;
   }

   public String obtenerNombre() {
      return nombreProducto;
   }

   public double obtenerPrecio() {
      return precioProducto;
   }

   public int obtenerCantidad() {
      return cantidadProducto;
   }

   public void mostrarInformacion() {
      System.out.println("Producto: " + nombreProducto + ", Precio: " + precioProducto + ",
Cantidad: " + cantidadProducto);
   }
}
```

```
class ProductoElectronico extends Producto {
   private String marcaProductoElectronico;

   public ProductoElectronico(String nombre, double precio, int cantidad, String marca) {
      nombreProducto = nombre;
      precioProducto = precio;
      cantidadProducto = cantidad;
      marcaProductoElectronico = marca;
   }
```

```
    public void mostrarInformacionElectronico() {
        mostrarInformacion();  // Llamando al método de la clase base
        System.out.println("Marca: " + marcaProductoElectronico);
    }
}
```

```
class Tienda {
    private Producto[] productos;
    private int indice;

    public Tienda(int capacidad) {
        productos = new Producto[capacidad];
        indice = 0;
    }

    public void agregarProducto(Producto producto) {
        if (indice < productos.length) {
            productos[indice] = producto;
            indice++;
        } else {
            System.out.println("La tienda está llena.");
        }
    }

    public void mostrarProductos() {
        System.out.println("Productos disponibles en la tienda:");
        for (int i = 0; i < indice; i++) {
            productos[i].mostrarInformacion();
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Producto producto1 = new Producto("Camiseta", 19.99, 10);
        ProductoElectronico productoElectronico1 = new ProductoElectronico("Laptop", 899.99,
5, "Dell");

        Tienda tienda = new Tienda(5);
        tienda.agregarProducto(producto1);
        tienda.agregarProducto(productoElectronico1);

        tienda.mostrarProductos();
    }
}
```

Ejercicio 8.39. Vamos a gestionar una biblioteca de películas. Para ello vamos a crear una clase **Pelicula** que tenga los atributos: titulo, duracion (en minutos), y anioEstreno. Luego, se creará una subclase **PeliculaDeAccion** que herede de **Pelicula** y tenga un atributo adicional para la **clasificacion**. Usaremos constructores para inicializar los atributos.

1. **Pelicula**:
    - Atributos:
        - tituloPelicula (String)
        - duracionPelicula (int)
        - anioEstrenoPelicula (int)
    - Constructor que reciba titulo, duracion y anioEstreno como parámetros y los inicialice.
    - Método **mostrarInformacion()** que imprima los detalles de la película.
2. **PeliculaDeAccion** (subclase de Pelicula):
    - Atributo adicional: clasificacionPelicula (String)
    - Constructor que reciba titulo, duracion, anioEstreno y clasificacion como parámetros.
    - Método **mostrarInformacionAccion()** que imprima los detalles de la película de acción.
3. **Biblioteca**:
    - Atributo: peliculas (un arreglo de tipo Pelicula de tamaño fijo).
    - Método **agregarPelicula()** que reciba un objeto de tipo Pelicula y lo agregue al arreglo.
    - Método **mostrarPeliculas()** que imprima todos los detalles de las películas.
4. En el main(), crea instancias de Pelicula y PeliculaDeAccion usando los constructores, agrega las películas a la biblioteca y muestra la información de todas las películas.

```
class Pelicula {
   private String tituloPelicula;
   private int duracionPelicula;
   private int anioEstrenoPelicula;

   public Pelicula(String titulo, int duracion, int anioEstreno) {
      tituloPelicula = titulo;
      duracionPelicula = duracion;
      anioEstrenoPelicula = anioEstreno;
   }

   public String obtenerTitulo() {
      return tituloPelicula;
   }
```

```
    public int obtenerDuracion() {
        return duracionPelicula;
    }

    public int obtenerAnioEstreno() {
        return anioEstrenoPelicula;
    }

    public void mostrarInformacion() {
        System.out.println("Título: " + tituloPelicula + ", Duración: " + duracionPelicula + "
minutos, Año de estreno: " + anioEstrenoPelicula);
    }
}
```

```
class PeliculaDeAccion extends Pelicula {
    private String clasificacionPelicula;

    public PeliculaDeAccion(String titulo, int duracion, int anioEstreno, String clasificacion) {
        tituloPelicula = titulo;
        duracionPelicula = duracion;
        anioEstrenoPelicula = anioEstreno;
        clasificacionPelicula = clasificacion;
    }

    public void mostrarInformacionAccion() {
        mostrarInformacion();  // Llamando al método de la clase base
        System.out.println("Clasificación: " + clasificacionPelicula);
    }
}
```

```
class Biblioteca {
    private Pelicula[] peliculas;
    private int indice;

    public Biblioteca(int capacidad) {
        peliculas = new Pelicula[capacidad];
        indice = 0;
    }

    public void agregarPelicula(Pelicula pelicula) {
        if (indice < peliculas.length) {
            peliculas[indice] = pelicula;
            indice++;
        } else {
            System.out.println("La biblioteca está llena.");
        }
    }

    public void mostrarPeliculas() {
```

```
        System.out.println("Películas disponibles en la biblioteca:");
        for (int i = 0; i < indice; i++) {
            peliculas[i].mostrarInformacion();
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Pelicula pelicula1 = new Pelicula("La Casa de Papel", 60, 2017);
        PeliculaDeAccion peliculaDeAccion1 = new PeliculaDeAccion("Rocky", 120, 1983,
"acción");

        Biblioteca biblioteca = new Biblioteca(5);
        biblioteca.agregarPelicula(pelicula1);
        biblioteca.agregarPelicula(peliculaDeAccion1);

        biblioteca.mostrarPeliculas();
    }
}
```

Ejercicio 8.40. Implementa un sistema para gestionar una tienda de vehículos. Cada vehículo tiene una marca y una descripción específica según su tipo. Así:

Clases a implementar:

- Vehiculo (Clase base)
- Coche (Subclase de Vehiculo)
- Moto (Subclase de Vehiculo)
- Bicicleta (Subclase de Vehiculo)

Polimorfismo:

- Todas las subclases deben sobrescribir el método obtenerDescripcion() para devolver información específica de cada tipo de vehículo.

Sobrecarga:

- La clase Coche debe tener una versión adicional de obtenerDescripcion() que reciba un parámetro indicando el tipo de combustible.

Pruebas:

- Crear objetos de cada clase (Coche, Moto, Bicicleta).
- Llamar al método obtenerDescripcion() en cada objeto.
- En el caso de Coche, llamar también a la versión sobrecargada con el tipo de combustible.

Ejemplo de salida esperada:

```
Coche de la marca Toyota con 4 puertas
Moto de la marca Honda con casco incluido
Bicicleta de la marca Giant con 21 cambios
Coche de la marca Toyota con 4 puertas, usa Gasolina
```

```
class Vehiculo {
   String marcaVehiculo;

   public Vehiculo(String marcaParametro) {
      marcaVehiculo = marcaParametro;
   }

   public String obtenerDescripcion() {
      return "Vehículo de la marca " + marcaVehiculo;
   }
}
```

```
class Coche extends Vehiculo {
   int puertasCoche;

   public Coche(String marcaParametro, int puertasParametro) {
      marcaVehiculo = marcaParametro;
      puertasCoche = puertasParametro;
   }

   // Sobrecarga con parámetro adicional (tipo de combustible)
   public String obtenerDescripcion(String tipoCombustible) {
      return "Coche de la marca " + marcaVehiculo + " con " + puertasCoche + " puertas, usa "
+ tipoCombustible;
   }

   public String obtenerDescripcion() {
      return "Coche de la marca " + marcaVehiculo + " con " + puertasCoche + " puertas";
   }
}
```

```
class Moto extends Vehiculo {
   boolean incluyeCasco;

   public Moto(String marcaParametro, boolean cascoParametro) {
      marcaVehiculo = marcaParametro;
      incluyeCasco = cascoParametro;
   }

   public String obtenerDescripcion() {
      if (incluyeCasco) {
         return "Moto de la marca " + marcaVehiculo + " con casco incluido";
      }
```

```
        return "Moto de la marca " + marcaVehiculo + " sin casco incluido";
    }
}
```

```
class Bicicleta extends Vehiculo {
    int cambiosBicicleta;

    public Bicicleta(String marcaParametro, int cambiosParametro) {
        marcaVehiculo = marcaParametro;
        cambiosBicicleta = cambiosParametro;
    }

    public String obtenerDescripcion() {
        return "Bicicleta de la marca " + marcaVehiculo + " con " + cambiosBicicleta + " cambios";
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Coche cocheEjemplo = new Coche("Toyota", 4);
        Moto motoEjemplo = new Moto("Honda", true);
        Bicicleta bicicletaEjemplo = new Bicicleta("Giant", 21);

        // Polimorfismo: cada objeto usa su versión de obtenerDescripcion()
        System.out.println(cocheEjemplo.obtenerDescripcion());
        System.out.println(motoEjemplo.obtenerDescripcion());
        System.out.println(bicicletaEjemplo.obtenerDescripcion());

        // Sobrecarga en Coche: llamada con un parámetro adicional
        System.out.println(cocheEjemplo.obtenerDescripcion("Gasolina"));
    }
}
```

Ejercicio 8.41. Implementa un sistema para gestionar una empresa. Cada empleado tiene un nombre y un salario base, pero dependiendo del tipo de empleado, el cálculo del salario varía.

Clases a implementar:

- Empleado (Clase base)
- Gerente (Subclase de Empleado)
- Ingeniero (Subclase de Empleado)
- Asistente (Subclase de Empleado)

Polimorfismo:

Cada tipo de empleado debe sobrescribir el método calcularSalario(), aplicando su propia lógica de cálculo:

- **Gerente:** Recibe un bono adicional.
- **Ingeniero:** Tiene un extra por certificaciones.

- **Asistente:** No tiene extras, solo el salario base.

Sobrecarga:

La clase Ingeniero debe tener una versión sobrecargada de calcularSalario() que reciba el número de certificaciones y aumente el salario basándose en ellas.

Pruebas:

- Crear objetos de cada clase (Gerente, Ingeniero, Asistente).
- Llamar al método calcularSalario() en cada objeto.
- Para Ingeniero, llamar también a la versión sobrecargada con el número de certificaciones.

Ejemplo de salida esperada:

```
Salario del Gerente: 7500
Salario del Ingeniero: 5000
Salario del Asistente: 3000
Salario del Ingeniero con 3 certificaciones: 5300
```

```
class Empleado {
   String nombreEmpleado;
   double salarioBase;

   public Empleado(String nombreParametro, double salarioParametro) {
      nombreEmpleado = nombreParametro;
      salarioBase = salarioParametro;
   }

   public double calcularSalario() {
      return salarioBase;
   }
}
```

```
class Gerente extends Empleado {
   double bonoGerente;

   public Gerente(String nombreParametro, double salarioParametro, double
bonoParametro) {
      nombreEmpleado = nombreParametro;
      salarioBase = salarioParametro;
      bonoGerente = bonoParametro;
   }

   public double calcularSalario() {
      return salarioBase + bonoGerente;
   }
}
```

```
class Ingeniero extends Empleado {
   double extraCertificacion;

   public Ingeniero(String nombreParametro, double salarioParametro, double extraParametro) {
      nombreEmpleado = nombreParametro;
      salarioBase = salarioParametro;
      extraCertificacion = extraParametro;
   }

   // Método sobrecargado para calcular salario con certificaciones
   public double calcularSalario(int cantidadCertificaciones) {
      return salarioBase + (extraCertificacion * cantidadCertificaciones);
   }

   public double calcularSalario() {
      return salarioBase;
   }
}
```

```
class Asistente extends Empleado {
   public Asistente(String nombreParametro, double salarioParametro) {
      nombreEmpleado = nombreParametro;
      salarioBase = salarioParametro;
   }

   public double calcularSalario() {
      return salarioBase;
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Gerente gerenteEjemplo = new Gerente("Carlos", 7000, 500);
      Ingeniero ingenieroEjemplo = new Ingeniero("Ana", 5000, 100);
      Asistente asistenteEjemplo = new Asistente("Luis", 3000);

      // Polimorfismo: llamada a calcularSalario() según el tipo de empleado
      System.out.println("Salario del Gerente: " + gerenteEjemplo.calcularSalario());
      System.out.println("Salario del Ingeniero: " + ingenieroEjemplo.calcularSalario());
      System.out.println("Salario del Asistente: " + asistenteEjemplo.calcularSalario());

      // Sobrecarga en Ingeniero: cálculo con certificaciones
      System.out.println("Salario del Ingeniero con 3 certificaciones: " + ingenieroEjemplo.calcularSalario(3));
   }
}
```

Ejercicio 8.42. Desarrolla un sistema que modele distintos tipos de dispositivos inteligentes con diferentes niveles de herencia y constructores encadenados para analizar el **orden de ejecución de los constructores** cuando se crean objetos de la subclase más específica.

Clases a implementar:

- Dispositivo (Clase base)
- DispositivoInteligente (Subclase de Dispositivo)
- Telefono (Subclase de DispositivoInteligente)
- Smartphone (Subclase de Telefono)

Inicialización de Atributos:

- Cada clase debe tener atributos propios, como **nombre**, **conectividad**, **sistema operativo**, **cámara**, etc.
- Cada constructor debe inicializar estos atributos e imprimir un mensaje que indique su ejecución.

Crea un objeto de cada clase y observa el orden en que se ejecutan los constructores y cómo se inicializan los atributos.

```
class Dispositivo {
   String nombre;

   public Dispositivo() {
      nombre = "Electrónico";
      System.out.println("Constructor de Dispositivo ejecutado");
      System.out.println("Nombre: " + nombre);
      System.out.println();
   }
}
```

```
class DispositivoInteligente extends Dispositivo {
   String conectividad;

   public DispositivoInteligente() {
      conectividad = "WiFi";
      System.out.println("Constructor de DispositivoInteligente ejecutado");
      System.out.println("Conectividad: " + conectividad);
      System.out.println();
   }
}
```

```
class Telefono extends DispositivoInteligente {
   String sistemaOperativo;

   public Telefono() {
      sistemaOperativo = "Android";
      System.out.println("Constructor de Telefono ejecutado");
      System.out.println("Sistema Operativo: " + sistemaOperativo);
      System.out.println();
```

```
    }
}
```

```
class Smartphone extends Telefono {
    String camara;

    public Smartphone() {
        camara = "108 MP";
        System.out.println("Constructor de Smartphone ejecutado");
        System.out.println("Cámara: " + camara);
        System.out.println();
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Smartphone miSmartphone = new Smartphone();
    }
}
```

Ejercicio 8.43. Desarrolla un programa que administre una flota de vehículos utilizando **arrays de objetos**. El sistema permitirá almacenar información de varios vehículos y mostrará un resumen de todos los vehículos registrados. Para ello:

- Implementa las clases Vehiculo (Clase base con atributos generales), Automovil y Camion (Subclases de Vehiculo con atributos específicos).
- Crea un array de objetos para almacenar múltiples vehículos. El array debe permitir agregar tanto automóviles como camiones.
- Implementa un método para agregar vehículos al array. Y otro método para mostrar todos los vehículos registrados.
- Para probar el sistema, crea un array de 5 vehículos y muestra la información en la consola.

```
class Vehiculo {
    String marca;
    String modelo;
    int anio;

    public Vehiculo(String marcaVehiculo, String modeloVehiculo, int anioVehiculo) {
        marca = marcaVehiculo;
        modelo = modeloVehiculo;
        anio = anioVehiculo;
    }

    public void mostrarInfo() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Año: " + anio);
    }
}
```

```
class Automovil extends Vehiculo {
   int puertas;

   public Automovil(String marcaVehiculo, String modeloVehiculo, int anioVehiculo, int numPuertas) {
      marca = marcaVehiculo;
      modelo = modeloVehiculo;
      anio = anioVehiculo;
      puertas = numPuertas;
   }

   public void mostrarInfo() {
      System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Año: " + anio);
      System.out.println("Puertas: " + puertas);
   }
}
```

```
class Camion extends Vehiculo {
   int cargaMaxima;

   public Camion(String marcaVehiculo, String modeloVehiculo, int anioVehiculo, int carga) {
      marca = marcaVehiculo;
      modelo = modeloVehiculo;
      anio = anioVehiculo;
      cargaMaxima = carga;
   }

   public void mostrarInfo() {
      System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Año: " + anio);
      System.out.println("Carga Máxima: " + cargaMaxima + " toneladas");
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Vehiculo[] flota = new Vehiculo[5];

      flota[0] = new Automovil("Toyota", "Corolla", 2020, 4);
      flota[1] = new Automovil("Honda", "Civic", 2022, 4);
      flota[2] = new Camion("Volvo", "FH16", 2019, 20);
      flota[3] = new Automovil("Ford", "Focus", 2018, 4);
      flota[4] = new Camion("Mercedes", "Actros", 2021, 25);

      System.out.println("Vehículos registrados: ");
      for (int i = 0; i < flota.length; i++) {
         flota[i].mostrarInfo();
      }
   }
}
```

Ejercicio 8.44. Desarrolla un programa que administre un **inventario de dispositivos electrónicos** utilizando **arrays de objetos**. El sistema permitirá almacenar información de varios dispositivos y mostrará un resumen de todos los dispositivos registrados. Para ello:

- Implementa las clases Dispositivo (Clase base con atributos generales), Smartphone y Laptop (Subclases de Dispositivo con atributos específicos).
- Crea un array de objetos para almacenar múltiples dispositivos. El array debe permitir agregar tanto smartphones como laptops.
- Implementa un método para agregar dispositivos al array y otro método para mostrar todos los dispositivos registrados.
- Para probar el sistema crea un array de 5 dispositivos (combinando smartphones y laptops) y muestra la información de los dispositivos en la consola.

```
class Dispositivo {
   String marca;
   String modelo;
   double precio;

   public Dispositivo(String marcaDispositivo, String modeloDispositivo, double
precioDispositivo) {
      marca = marcaDispositivo;
      modelo = modeloDispositivo;
      precio = precioDispositivo;
   }

   public void mostrarInfo() {
      System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Precio: " + precio);
   }
}
```

```
class Smartphone extends Dispositivo {
   int almacenamiento;

   public Smartphone(String marcaDispositivo, String modeloDispositivo, double
precioDispositivo, int almacenamientoGB) {
      marca = marcaDispositivo;
      modelo = modeloDispositivo;
      precio = precioDispositivo;
      almacenamiento = almacenamientoGB;
   }

   public void mostrarInfo() {
      System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Precio: " + precio);
      System.out.println("Almacenamiento: " + almacenamiento + "GB");
   }
}
```

```
class Laptop extends Dispositivo {
   double tamanoPantalla;
```

```
   public Laptop(String marcaDispositivo, String modeloDispositivo, double precioDispositivo,
double pantallaPulgadas) {
      marca = marcaDispositivo;
      modelo = modeloDispositivo;
      precio = precioDispositivo;
      tamanoPantalla = pantallaPulgadas;
   }

   public void mostrarInfo() {
      System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Precio: " + precio);
      System.out.println("Tamaño de pantalla: " + tamanoPantalla + " pulgadas");
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      Dispositivo[] inventario = new Dispositivo[5];

      inventario[0] = new Smartphone("Samsung", "Galaxy S21", 799.99, 128);
      inventario[1] = new Laptop("Dell", "XPS 13", 1199.99, 13.3);
      inventario[2] = new Smartphone("Apple", "iPhone 14", 999.99, 256);
      inventario[3] = new Laptop("HP", "Pavilion 15", 799.99, 15.6);
      inventario[4] = new Smartphone("Google", "Pixel 7", 599.99, 128);

      System.out.println("Inventario de dispositivos electrónicos");
      for (int i = 0; i < inventario.length; i++) {
         inventario[i].mostrarInfo();
      }
   }
}
```

Ejercicio 8.45. Crea un sistema que maneje diferentes tipos de vehículos usando herencia y sobreescritura de métodos:

1. Crea una clase base Vehiculo con:
   - Atributos marca y modelo.
   - Un constructor que reciba la marca y el modelo.
   - Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
   - Un método mostrarInfo() que imprima la información del vehículo.
2. Crea una clase Coche que extienda Vehiculo con:
   - Añade el atributo puertas (número de puertas).
   - Un constructor que reciba la marca, el modelo y las puertas, usando super para inicializar los atributos de la clase base.

- Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
- Sobrescribe (@Override) el método mostrarInfo() para incluir el número de puertas.

3. Crea una clase Moto que extienda Vehiculo con:
   - Añade el atributo cilindrada.
   - Un constructor que reciba la marca, el modelo y la cilindrada, usando super.
   - Constructor sin parámetros con valores predeterminados. Usa this en el constructor.
   - Sobrescribe mostrarInfo() para incluir la cilindrada.
4. En la función main, crea un array de 4 Vehiculos con instancias de Coche y Moto. Luego llama a mostrarInfo() con cada objeto.

```
class Vehiculo {
   protected String marca;
   protected String modelo;

   public Vehiculo(String marca, String modelo) {
      this.marca = marca;
      this.modelo = modelo;
   }

   // Constructor sin parámetros con valores predeterminados
   public Vehiculo() {
      this("Desconocida", "Genérico");
   }

   public void mostrarInfo() {
      System.out.println("Vehículo: " + marca + " " + modelo);
   }
}
```

```
class Coche extends Vehiculo {
   private int puertas;

   public Coche(String marca, String modelo, int puertas) {
      super(marca, modelo);
      this.puertas = puertas;
   }

   // Constructor sin parámetros con valores predeterminados
   public Coche() {
      this("Toyota", "Corolla", 4);
   }

   // Sobrescribe el método mostrarInfo
```

```
    @Override
    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("Número de puertas: " + this.puertas);
    }
}
```

```
class Moto extends Vehiculo {
    private int cilindrada;

    public Moto(String marca, String modelo, int cilindrada) {
        super(marca, modelo);
        this.cilindrada = cilindrada;
    }

    // Constructor sin parámetros con valores predeterminados
    public Moto() {
        this("Honda", "CBR600", 600);
    }

    // Sobrescribe el método mostrarInfo
    @Override
    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("Cilindrada: " + this.cilindrada + "cc");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Vehiculo[] vehiculos = {
            new Coche("Ford", "Fiesta", 5),
            new Coche(),
            new Moto("Yamaha", "R1", 1000),
            new Moto()
        };

        for (int i = 0; i < vehiculos.length; i++) {
            vehiculos[i].mostrarInfo();
        }
    }
}
```

Ejercicio 8.46. Crea un sistema de empleados con diferentes tipos (Gerente y Desarrollador). Cada uno tiene atributos específicos y un método calcularSalario(), que será sobrescrito en cada clase derivada.

1. **Crea la clase base Empleado** con:
    - Atributos nombre, salarioBase y id.

- Un constructor que use this() para asignar valores por defecto.
- Un método calcularSalario() que devuelva salarioBase.
- Un método mostrarInfo() que muestre el nombre y salario.

2. **Crea la clase Gerente que extienda Empleado** con:
   - Atributo bonoGerencial.
   - Un constructor que use super() para llamar al constructor de Empleado.
   - Sobrescribe calcularSalario() para sumar bonoGerencial.
3. **Crea la clase Desarrollador que extienda Empleado** con:
   - Atributo horasExtras y pagoPorHoraExtra.
   - Un constructor que use super().
   - Sobrescribe calcularSalario() para sumar el pago por horas extras.
4. En la función main, crea un array de 4 Empleados con instancias de Gerente y Desarrollador. Luego llama a mostrarInfo() con cada objeto.

```
class Empleado {
   protected String nombre;
   protected double salarioBase;
   protected int id;

   public Empleado(String nombre, double salarioBase, int id) {
      this.nombre = nombre;
      this.salarioBase = salarioBase;
      this.id = id;
   }

   public Empleado() {
      this("Empleado Genérico", 1000.0, 0);
   }

   public double calcularSalario() {
      return salarioBase;
   }

   public void mostrarInfo() {
      System.out.println("ID: " + id + "  Nombre: " + nombre + " Salario: " + calcularSalario());
   }
}
```

```
class Gerente extends Empleado {
   private double bonoGerencial;

   public Gerente(String nombre, double salarioBase, int id, double bonoGerencial) {
      super(nombre, salarioBase, id);
      this.bonoGerencial = bonoGerencial;
```

```
    }

    public Gerente() {
        this("Gerente Default", 3000.0, 1, 500.0);
    }

    @Override
    public double calcularSalario() {
        return super.calcularSalario() + this.bonoGerencial; //En este caso se está accediendo al
método calcularSalario con el objeto Gerente
    }
}
```

```
class Desarrollador extends Empleado {
    private int horasExtras;
    private double pagoPorHoraExtra;

    public Desarrollador(String nombre, double salarioBase, int id, int horasExtras, double
pagoPorHoraExtra) {
        super(nombre, salarioBase, id);
        this.horasExtras = horasExtras;
        this.pagoPorHoraExtra = pagoPorHoraExtra;
    }

    public Desarrollador() {
        this("Desarrollador Default", 2500.0, 2, 10, 20.0);
    }

    @Override
    public double calcularSalario() {
        return super.calcularSalario() + (horasExtras * pagoPorHoraExtra);//En este caso se está
accediendo al getter calcularSalario() con el objeto Desarrollador
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Empleado[] empleados = new Empleado[4];

        for (int i = 0; i < empleados.length; i++) {
            if (i % 2 == 0) {
                empleados[i] = new Gerente("Gerente" + i, 3500.0, i + 1, 700.0);
            } else {
                empleados[i] = new Desarrollador("Dev" + i, 2700.0, i + 1, 5 + i, 25.0);
            }
        }

        for (int i = 0; i < empleados.length; i++) {
            empleados[i].mostrarInfo();
        }
```

```
    }
}
```

NOTA: En el ejercicio, calcularSalario() **no es un método estático**, así que solo se puede llamar a través de un **objeto**. Pero... ¿dónde está ese objeto? **Está implícito**.

En Java, cuando llamas a un método dentro de otro método de la misma instancia, **Java asume que es this.calcularSalario()**, aunque no lo escribas explícitamente.

Es decir, la línea

```
System.out.println("ID: " + id + "  Nombre: " + nombre + " Salario: " + calcularSalario());
```

es equivalente a:

```
System.out.println("ID: " + id + " Nombre: " + nombre + "  Salario: " + this.calcularSalario());
```

Donde this se refiere al objeto actual en el que se está ejecutando mostrarInfo().

Ejercicio 8.47. Copia y pega el siguiente código en Eclipse y ejecútalo. El programa **compila sin errores**, pero **lanza una excepción** si introduces una palabra. Usa el depurador para detectar el fallo.

```
import java.util.Scanner;

public class depurarCodigo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce una palabra: ");
        String palabra = sc.next();
        System.out.print("La primera letra es: " + obtenerPrimeraLetra(palabra));
    }

    public static String obtenerPrimeraLetra(String texto) {
        return texto.substring(0, 0);
    }
}
```

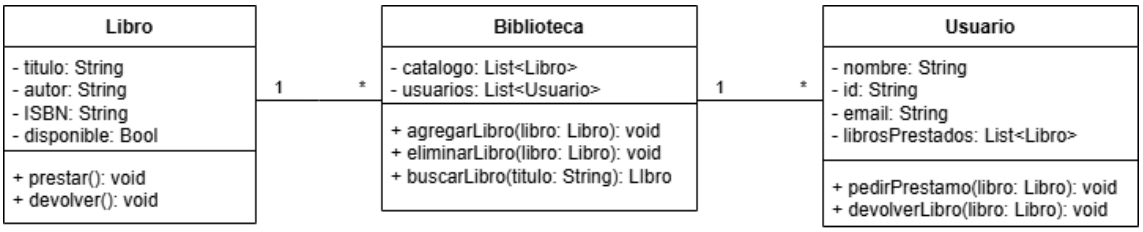
El fallo está en la línea **return texto.substring(0, 0**); que devuelve una cadena vacía porque substring(0, 0) no devuelve ningún carácter.
Para corregirlo haz return texto.substring(0, 1);

EJERCICIOS RESUELTOS DEL TEMA 9

Ejercicio 9.1. Una biblioteca necesita un sistema para gestionar los libros y los préstamos a los usuarios. El sistema debe permitir registrar libros, usuarios y manejar los préstamos de libros. Los requisitos son:

- Cada **Libro** tiene un título, un autor, un ISBN único y un estado de disponibilidad.
- Un **Usuario** tiene un nombre, un ID único, un correo electrónico y una lista de libros prestados.
- La **Biblioteca** debe gestionar un catálogo de libros y una lista de usuarios.
- Un usuario puede **pedir prestado** un libro si está disponible y **devolverlo** posteriormente.
- La biblioteca debe permitir **agregar** y **eliminar** libros de su catálogo.
- Debe existir una función para **buscar libros** por título.

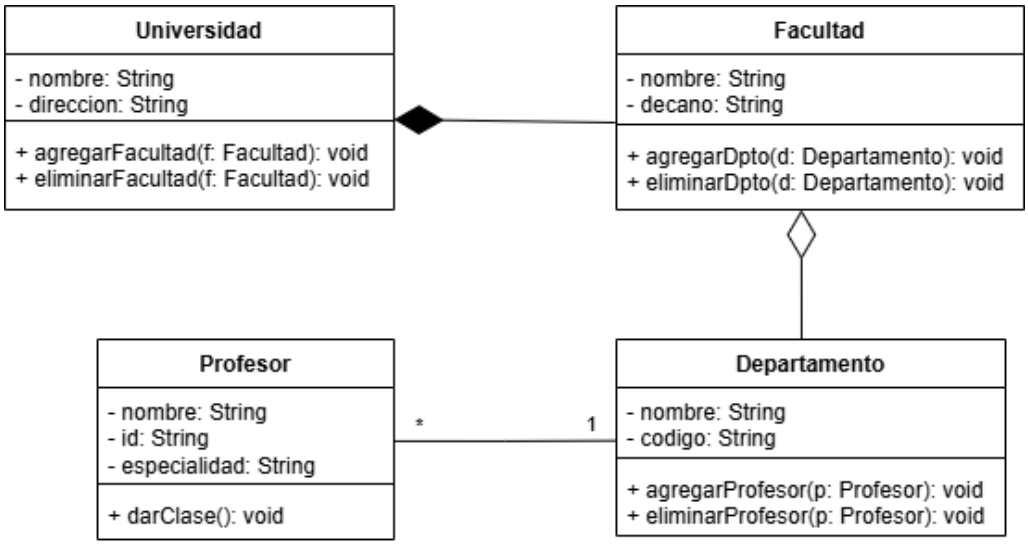
Diseña el **diagrama de clases UML**.



Ejercicio 9.2. Se necesita modelar un sistema para gestionar las facultades, departamentos y profesores de una universidad. Los requisitos son:

- Una universidad está compuesta por múltiples facultades.
- Si la universidad deja de existir, todas sus facultades también desaparecen. (**Composición**).
- Cada facultad pertenece a una universidad y puede tener varios departamentos.
- Si una facultad se elimina, sus departamentos pueden seguir existiendo de manera independiente. (**Agregación**).
- Cada departamento pertenece a una facultad y tiene profesores.
- Un profesor solo puede pertenecer a un departamento.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.



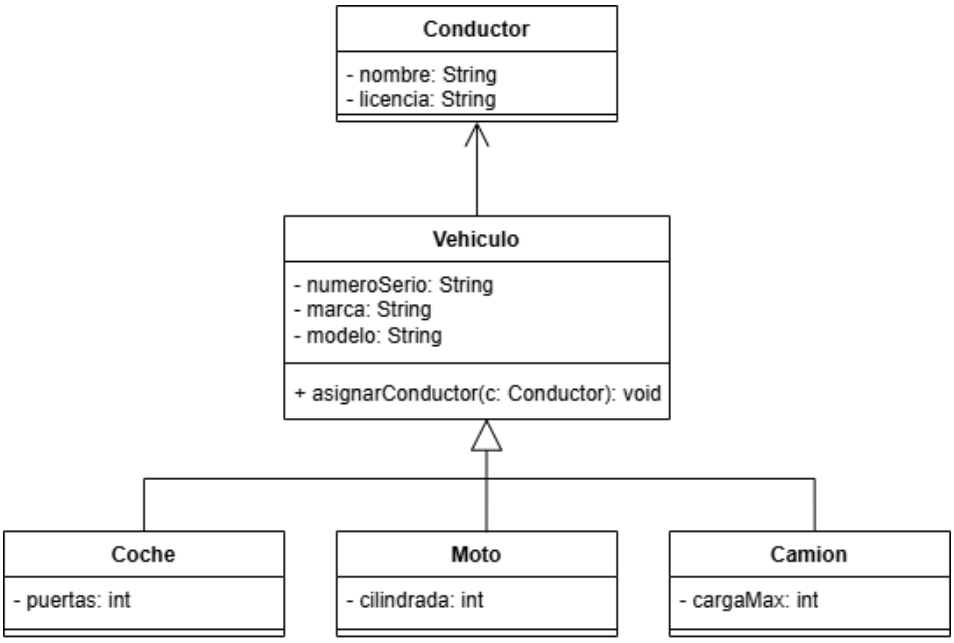
Composición: Si la Universidad desaparece, sus Facultades también.

Agregación: Si la Facultad desaparece, los Departamentos pueden seguir existiendo.

Ejercicio 9.3. Se necesita modelar un sistema para gestionar los vehículos de una empresa de transporte. Los requisitos son:

- Todos los vehículos tienen un **número de serie**, **marca** y **modelo**.
- Existen diferentes tipos de vehículos: **Coche**, **Moto** y **Camión**, que heredan de **Vehículo**.
- **Coche**: Tiene un atributo adicional **cantidad de puertas**.
- **Moto**: Tiene un atributo **cilindrada**.
- **Camión**: Tiene un atributo **capacidad de carga**.
- Cada vehículo puede estar asignado a **un Conductor**.
- Un **Conductor** tiene un **nombre** y un **número de licencia**.
- La relación entre conductor y vehículo es **unidireccional**.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.

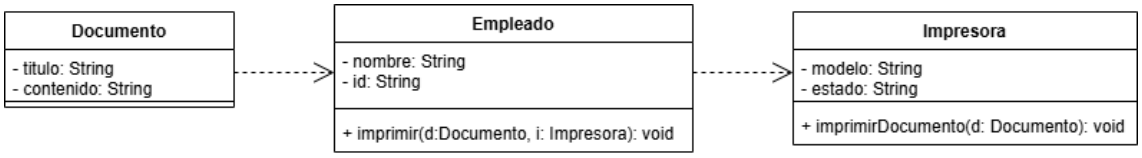


NOTA: Aunque no tenga lógica la relación entre vehículo y conductor para ti, recuerda que es solo una cuestión de diseño…

Ejercicio 9.4. Se necesita modelar un sistema donde los empleados pueden enviar documentos a una impresora para ser impresos. Los requisitos son:

- **Empleado**: Representa a una persona que trabaja en la oficina y necesita imprimir documentos.
- **Documento**: Representa un archivo que el empleado quiere imprimir.
- **Impresora**: Se encarga de imprimir los documentos enviados.
- Un **Empleado depende** de la clase **Impresora** porque necesita una impresora para imprimir.
- Un **Documento depende** de la clase **Empleado**, ya que sin un empleado no hay documentos que imprimir.

Diseña el **diagrama de clases UML** con las asociaciones adecuadas.



RECUERDA: La relación de dependencia significa que una clase **usa temporalmente** otra clase sin poseerla permanentemente, es decir, un empleado **usa temporalmente** la impresora para imprimir un documento, pero no la **posee permanentemente**.

Del mismo modo, un documento **existe porque un empleado lo crea**, por lo que el documento depende del empleado.

EJERCICIOS RESUELTOS DEL TEMA 10

Ejercicio 10.1. Crea un programa que pida al usuario 3 números reales y 3 cadenas de texto y lo guarde en una pila **no** genérica. Después muestra el contenido en orden contrario. ¿Has tenido algún problema? ¿Por qué?

```
package DecimoTema;

import java.util.Scanner;
import java.util.Stack;

public class Ejercicio10_1 {

	public static void main(String args[]) {
		Stack pila = new Stack(); // NO genérica. Pila sin tipo

		Scanner in = new Scanner (System.in);

		for (int i=0; i<3; i++) {
			System.out.print("Introduce un número: ");
			float num = in.nextFloat();
			pila.push(num);
		}
		for (int i=0; i<3; i++) {
			System.out.print("Introduce una palabra: ");
			String palabra = in.next();
			pila.push(palabra);
		}
		int total = pila.size();
		for (int i=0; i<total; i++) {
			System.out.println("El contenido de la pila es: " +
pila.pop());
		}
	}
}
```

No ha habido ningún problema en la implementación porque solo se ha mostrado el contenido de la pila por consola (object.toString()). Si hubiese tenido que hacer alguna operación con los datos de la pila, habría tenido que hacer casting obligatoriamente.

Ejercicio 10.2. La "notación polaca inversa" es una forma de expresar operaciones que consiste en indicar los operandos antes del correspondiente operador. Por ejemplo, en vez de "3+4" se escribiría "3 4 +". Es una notación que no necesita paréntesis y que se puede resolver usando una pila: si se recibe un dato numérico, éste se guarda en la pila; si se recibe un operador, se obtienen los dos operandos que hay en la cima de la pila, se realiza la operación y se apila su resultado. El proceso termina cuando sólo hay un dato en la pila. Por ejemplo, "3 4 +" se convierte en: apilar 3, apilar 4, sacar dos datos y sumarlos, guardar 7, terminado. Impleméntalo y comprueba si el resultado de "3 4 6 5 - + * 6 +" es 21. Realiza el ejercicio con funciones.

```
package DecimoTema;

import java.util.Scanner;
import java.util.Stack;

public class Ejercicio10_2 {
	static Stack<String> pila = new Stack<>();
```

```
	public static void main(String args[]) {
		String dato;
		do {
			Scanner in = new Scanner (System.in);
			System.out.print("Introduce un carácter (Intro para
salir): ");
			dato = in.nextLine();

			if (dato.equals("0") || dato.equals("1") ||
dato.equals("2") || dato.equals("3") || dato.equals("4") ||
dato.equals("5") || dato.equals("6") || dato.equals("7") ||
dato.equals("8") || dato.equals("9"))

			{
			apilar(dato);
		}
		else if (dato.equals("+") || dato.equals("-") ||
dato.equals("*") || dato.equals("/"))
		{
			desapilar(dato);
		}
		else if (dato != "")
			System.out.println("Error");
	      else break;

		} while (dato != "");

		System.out.println("Fin del programa.");
	}
	public static void apilar(String caracter)
	{
	    pila.push(caracter);
	}

	public static void desapilar(String simbolo)
	{
	    int num1 = Integer.parseInt(pila.pop());
	    int num2 = Integer.parseInt(pila.pop());
	    //Si no hubiese suficientes elementos en la pila para realizar
la operación, saltaría una excepción
	    String operacion;
	    if (simbolo.equals("+")) operacion = Integer.toString(num2 +
num1);
	    else if (simbolo.equals("-")) operacion = Integer.toString(num2
- num1);
	    else if (simbolo.equals("*")) operacion = Integer.toString(num2
* num1);
	    else operacion = Integer.toString(num2 / num1);
	    System.out.println("El resultado de la operación anterior es: "
+ operacion);
	    apilar(operacion);
	}
}
```

Ejercicio 10.3. Crea un programa que pida al usuario 5 números reales de doble precisión, los guarde en una cola y luego los muestre en pantalla.

```java
package DecimoTema;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class Ejercicio10_3 {
    public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
        Queue<Double> cola = new LinkedList<>();
        for (int i=0; i<5; i++) {
             System.out.print("Introduce un número real de doble precisión:
");
             cola.add(sc.nextDouble());
        }
        int total = cola.size();
        for (int i=0; i<total; i++) {
            System.out.println("La posición " + i + " de la cola es: " +
cola.poll());
        }
    }
}
```

Ejercicio 10.4. Crea un programa que pida frases al usuario, hasta que introduzca una frase vacía. En ese momento, mostrará todas las frases que se habían introducido.

```java
package DecimoTema;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class Ejercicio10_4 {
    public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
        Queue<String> cola = new LinkedList<>();
        String frase;
        do{
             System.out.print("Introduce una frase: ");
             frase = sc.nextLine();
             if (frase.equals("")) break;
             cola.add(frase);
        } while (frase!="");
        int total = cola.size();
        for (int i=0; i<total; i++) {
            System.out.println("La posición " + i + " de la cola es: " +
cola.poll());
        }
    }
}
```

Ejercicio 10.5. Escribe un programa que:

1. Cree una lista para almacenar nombres (Strings).
2. Pida al usuario que introduzca 5 nombres y los añada a la lista.

3. Muestre todos los nombres almacenados.
4. Pida al usuario que introduzca un índice y muestre el nombre que está en esa posición (validando que el índice sea válido).
5. Pida al usuario un índice para eliminar el nombre en esa posición y lo elimine de la lista (validando el índice).
6. Finalmente, muestre la lista actualizada con los nombres que quedan.

```
package DecimoTema;

import java.util.List;
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio10_5 {
   public static void main(String[] args) {
      List<String> nombres = new ArrayList<>();
      Scanner sc = new Scanner(System.in);

      // Paso 2: Pedir 5 nombres
      for (int i = 0; i < 5; i++) {
         System.out.print("Introduce un nombre: ");
         nombres.add(sc.nextLine());
      }

      // Paso 3: Mostrar todos los nombres
      System.out.println("\nNombres introducidos:");
      for (String nombre : nombres) {
         System.out.println(nombre);
      }

      // Paso 4: Pedir índice para mostrar nombre
      System.out.print("\nIntroduce un índice para mostrar el nombre (0 a " +
(nombres.size() - 1) + "): ");
      int index = sc.nextInt();
      sc.nextLine(); // limpiar buffer
      if (index >= 0 && index < nombres.size()) {
         System.out.println("Nombre en la posición " + index + ": " + nombres.get(index));
      } else {
         System.out.println("Índice fuera de rango.");
      }

      // Paso 5: Pedir índice para eliminar nombre
      System.out.print("\nIntroduce un índice para eliminar el nombre (0 a " +
(nombres.size() - 1) + "): ");
      int eliminarIndex = sc.nextInt();
      sc.nextLine(); // limpiar buffer
      if (eliminarIndex >= 0 && eliminarIndex < nombres.size()) {
         String eliminado = nombres.remove(eliminarIndex);
         System.out.println("Nombre eliminado: " + eliminado);
      } else {
```

```
        System.out.println("Índice fuera de rango.");
    }

    // Paso 6: Mostrar lista actualizada
    System.out.println("\nLista actualizada:");
    for (String nombre : nombres) {
        System.out.println(nombre);
    }

    sc.close();
  }
}
```

Ejercicio 10.6. Crea un programa que, cuando el usuario introduzca el nombre de un número del 1 al 5 en inglés (por ejemplo, "two"), diga su traducción en español ("dos"). Saldrá del programa cuando pulse intro.

```
package DecimoTema;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ejercicio10_6 {
    public static void main(String[] args) {
       Map<String, String> map = new HashMap<>();

        map.put("one", "uno");
        map.put("two", "dos");
        map.put("three", "tres");
        map.put("four", "cuatro");
        map.put("five", "cinco");

       Scanner sc = new Scanner (System.in);
       String buscar;
       do {
              System.out.print("Introduce un elemento para la búsqueda: ");
              buscar = sc.nextLine();
              if (buscar == "") break;
              if (map.containsKey(buscar)) {
                     System.out.println("La traducción es: " +
map.get(buscar));
              } else {
                     System.out.println("El valor NO ha sido encontrado en
el mapa.");
              }
       } while (buscar!="");
              System.out.println("Fin del programa.");
    }
}
```

Ejercicio 10.7. Crea un programa que introduzca 5 elementos clave-valor en un mapa. Después mostrará por consola el mapa completo ¿Qué orden establece? ¿Por qué? Más tarde eliminará uno de los elementos, reemplazará uno de los valores del mapa con “replace” y volverá a

mostrar el mapa por consola. Luego pedirá al usuario que introduzca valores y se buscarán en el mapa. Si no se encuentran mostrará el mensaje “no encontrado”. Se saldrá del bucle cuando el usuario pulse 0.

```
package DecimoTema;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ejercicio10_7 {
    public static void main(String[] args) {
      Map<String, Integer> map = new HashMap<>();

        map.put("valor1", 15);
        map.put("valor2", 20);
        map.put("valor3", 1000);
        map.put("valor4", 1500);
        map.put("valor5", 2);

      System.out.println(map);

      map.remove("valor3");
      map.replace("valor4", 1500, 7);
      System.out.println(map);

      Scanner sc = new Scanner (System.in);
      int buscar;
      do {
            System.out.print("Introduce un elemento para la búsqueda: ");
            buscar = sc.nextInt();
            if (buscar == 0) break;
            if (map.containsValue(buscar)) {
                  System.out.println("El valor ha sido encontrado en el
mapa.");
            } else {
                  System.out.println("El valor NO ha sido encontrado en
el mapa.");
            }
      } while (buscar!=0);
            System.out.println("Fin del programa.");
    }
}
```

Ejercicio 10.8. Crea un programa que haga lo siguiente:

1. Cree un Map<String, String> para almacenar nombres y números de teléfono.
2. Pida al usuario que introduzca 3 contactos (nombre y número).
3. Después, pregunte al usuario por un nombre y muestre el número de teléfono asociado.
4. Si el nombre no está en la agenda, debe indicarlo con un mensaje.
5. Finalmente, muestra todos los contactos almacenados.

```
package DecimoTema;
```

```
import java.util.Map;
import java.util.HashMap;
import java.util.Scanner;

public class Ejercicio10_8 {
   public static void main(String[] args) {
      Map<String, String> agenda = new HashMap<>();
      Scanner sc = new Scanner(System.in);

      // Insertar 3 contactos
      for (int i = 1; i <= 3; i++) {
         System.out.print("Introduce el nombre del contacto " + i + ": ");
         String nombre = sc.nextLine();
         System.out.print("Introduce el número de " + nombre + ": ");
         String numero = sc.nextLine();
         agenda.put(nombre, numero);
      }

      // Buscar un contacto
      System.out.print("\n¿De qué persona quieres consultar el número? ");
      String consulta = sc.nextLine();
      if (agenda.containsKey(consulta)) {
         System.out.println("El número de " + consulta + " es: " + agenda.get(consulta));
      } else {
         System.out.println("Ese contacto no está en la agenda.");
      }

      // Mostrar todos los contactos
      System.out.println("\nLista completa de contactos:");
      for (String nombre : agenda.keySet()) {
         System.out.println(nombre + ": " + agenda.get(nombre));
      }

      sc.close();
   }
}
```

Ejercicio 10.9. Desarrolla un programa que permita almacenar palabras en una lista hasta que el usuario pulse Intro. Después muestra el contenido de la lista.

```
package DecimoTema;

import java.util.List;
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio10_9 {
    public static void main(String[] args) {
       List<String> miLista = new ArrayList<>();
       Scanner sc = new Scanner(System.in);
```

```
        String palabra;
        do {
                System.out.print("Introduce una palabra: ");
                palabra = sc.nextLine();
                if (palabra=="") break;
                miLista.add(palabra);
        } while (palabra!="");

        for (String cadaPalabra: miLista) {
                System.out.println(cadaPalabra);
        }
    }
}
```

Ejercicio 10.10. A partir del ejercicio anterior, ordena la lista por orden alfabético y vuelve a mostrarla.

```
package DecimoTema;

import java.util.List;
import java.util.ArrayList;
import java.util.Scanner;

public class Ejercicio10_10 {
    public static void main(String[] args) {
        List<String> miLista = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        String palabra;
        do {
                System.out.print("Introduce una palabra: ");
                palabra = sc.nextLine();
                if (palabra=="") break;
                miLista.add(palabra);
        } while (palabra!="");

        for (String cadaPalabra: miLista) {
                System.out.println(cadaPalabra);
        }
        miLista.sort(null);
        System.out.println("\n\nAhora por orden alfabético: ");
        for (String cadaPalabra: miLista) {
                System.out.println(cadaPalabra);
        }
    }
}
```

Ejercicio 10.11. Desarrolla un programa que use una pila para invertir una cadena dada por el usuario. Usa funciones y control de excepciones try-catch.

```
package DecimoTema;

import java.util.Stack;
import java.util.Scanner;

public class Ejercicio10_11 {
   public static String invertirCadena(String str) {
```

```
        Stack<Character> pila = new Stack<>();
        for (char c : str.toCharArray()) {
            pila.push(c);
        }

        StringBuilder resultado = new StringBuilder();
        while (!pila.isEmpty()) {
            resultado.append(pila.pop());
        }

        return resultado.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingresa una cadena: ");
        String input = sc.nextLine();
        try {
            System.out.println("Cadena invertida: " + invertirCadena(input));
        } catch (Exception e) {
            System.out.println("Error al invertir la cadena: " + e.getMessage());
        }
        sc.close();
    }
}
```

Ejercicio 10.12. Verifica si una palabra se lee igual de izquierda a derecha y de derecha a izquierda (palíndromo) usando una pila. Usa funciones y control de excepciones.

```
package DecimoTema;

import java.util.Stack;
import java.util.Scanner;

public class Ejercicio10_12 {
    public static boolean esPalindromo(String palabra) {
        Stack<Character> pila = new Stack<>();

        for (char c : palabra.toCharArray()) {
            pila.push(c);
        }

        StringBuilder inversa = new StringBuilder();
        while (!pila.isEmpty()) {
            inversa.append(pila.pop());
        }

        return palabra.equals(inversa.toString()); // Comparación exacta
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingresa una palabra: ");
        String palabra = sc.nextLine();
        try {
            System.out.println(esPalindromo(palabra) ? "Es un palíndromo" : "No es un palíndromo");
        } catch (Exception e) {
            System.out.println("Error al verificar la palabra.");
        }
        sc.close();
    }
}
```

Ejercicio 10.13. Haz el ejercicio anterior usando una cola y una pila.

```
package DecimoTema;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
import java.util.Scanner;

public class Ejercicio10_13 {
    public static boolean esPalindromo(String palabra) {
        Queue<Character> cola = new LinkedList<>();
        Stack<Character> pila = new Stack<>();

        try {
            for (char c : palabra.toCharArray()) {
                cola.add(c);
                pila.push(c);
            }

            while (!cola.isEmpty()) {
                if (!cola.poll().equals(pila.pop())) {
                    return false;
                }
            }
        } catch (Exception e) {
            System.out.println("Error al verificar el palíndromo: " + e.getMessage());
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingresa una palabra: ");
        String palabra = sc.nextLine();
        try {
            System.out.println(esPalindromo(palabra) ? "Es un palíndromo" : "No es un palíndromo");
```

```
        } catch (Exception e) {
            System.out.println("Error en la verificación.");
        }
        sc.close();
    }
}
```

Ejercicio 10.14. Mueve los primeros n elementos de una cola al final. Por ejemplo, si la entrada es: [1, 2, 3, 4, 5] y n = 2, la salida sería [3, 4, 5, 1, 2].

```
package DecimoTema;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class Ejercicio10_14 {
    public static Queue<Integer> rotarCola(Queue<Integer> cola, int n) {
        try {
            for (int i = 0; i < n; i++) {
                cola.add(cola.poll()); //El primer elemento de la cola es extraído y luego agregado al
final para rotar la cola.
            }
        } catch (Exception e) {
            System.out.println("Error al rotar la cola: " + e.getMessage());
        }
        return cola;
    }

    public static void main(String[] args) {
        Queue<Integer> cola = new LinkedList<>();
        cola.add(1);
        cola.add(2);
        cola.add(3);
        cola.add(4);
        cola.add(5);

        Scanner sc = new Scanner(System.in);
        System.out.print("Número de rotaciones: ");
        int n = sc.nextInt();

        System.out.println("Cola original: " + cola);
        Queue<Integer> colaRotada = rotarCola(cola, n);
        System.out.println("Cola rotada: " + colaRotada);
        sc.close();
    }
}
```

Ejercicio 10.15. Crea un ArrayList de números enteros y ordénalo de menor a mayor. Utiliza la clase Collections para ordenar. Echa un vistazo al resto de métodos que te ofrece esta clase para trabajar con listas.

```
package DecimoTema;

import java.util.ArrayList;
import java.util.Collections;

public class Ejercicio10_15 {
   public static void main(String[] args) {
      try {
         ArrayList<Integer> lista = new ArrayList<>();
         lista.add(3);
         lista.add(1);
         lista.add(4);
         lista.add(2);
         lista.add(5);

         // Ordenar de menor a mayor
         Collections.sort(lista);

         System.out.println("Lista ordenada: " + lista);
      } catch (Exception e) {
         System.out.println("Error al ordenar la lista: " + e.getMessage());
      }
   }
}
```

Ejercicio 10.16. Dado un ArrayList de enteros, recorre la lista con un bucle foreach y calcula la suma de todos los elementos.

```
package DecimoTema;

import java.util.ArrayList;

public class Ejercicio10_16 {
   public static void main(String[] args) {
      try {
         ArrayList<Integer> lista = new ArrayList<>();
         lista.add(1);
         lista.add(2);
         lista.add(3);
         lista.add(4);
         lista.add(5);

         int suma = 0;
         for (int num : lista) {
            suma += num;
         }
```

```
            System.out.println("La suma de los elementos es: " + suma);
        } catch (Exception e) {
            System.out.println("Error al recorrer la lista: " + e.getMessage());
        }
    }
}
```

Ejercicio 10.17. Crea un Map<String, Double> donde las claves sean los nombres de productos y los valores sus precios. El usuario introducirá una clave y el programa verifica si el producto específico está en el mapa y dice su precio. Utiliza distintas funciones y controla las posibles excepciones.

```
package DecimoTema;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ejercicio10_17 {
    public static void main(String[] args) {
        try {
            Map<String, Double> productos = new HashMap<>();
            productos.put("Manzana", 1.50);
            productos.put("Plátano", 0.80);
            productos.put("Pera", 1.20);

            Scanner scanner = new Scanner(System.in);
            System.out.print("Ingrese el nombre del producto para verificar: ");
            String productoBuscado = scanner.nextLine();

            verificarClave(productos, productoBuscado);

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    public static void verificarClave(Map<String, Double> mapa, String clave) {
        if (mapa.containsKey(clave)) {
            System.out.println("El producto '" + clave + "' está en el mapa.");
            System.out.println("Precio: " + mapa.get(clave));
        } else {
            System.out.println("El producto '" + clave + "' no está en el mapa.");
        }
    }
}
```

Ejercicio 10.18. Dado un Map<String, String>, elimina un elemento a través de su clave y luego imprime el mapa resultante especificando su tamaño. Controla las posibles excepciones.

```
package DecimoTema;

import java.util.HashMap;
import java.util.Map;

public class Ejercicio10_18 {
   public static void main(String[] args) {
      try {
         Map<String, String> paises = new HashMap<>();
         paises.put("España", "Madrid");
         paises.put("Francia", "París");
         paises.put("Italia", "Roma");

         System.out.println("Mapa original: " + paises);

         paises.remove("Francia");

         System.out.println("Mapa después de eliminar Francia: " + paises);

         System.out.println("El mapa contiene " + paises.size() + " elementos.");

      } catch (Exception e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
}
```

Ejercicio 10.19. Crea un programa que gestione un mapa de productos y sus precios (clave-valor). El programa debe mostrar el mapa original con algunos productos y precios predefinidos, luego debe pedir al usuario que ingrese el nombre de un producto que desea **actualizar o agregar**. Si el producto ya existe en el mapa, se debe solicitar al usuario un nuevo precio para ese producto y actualizar el valor correspondiente en el mapa. Si el producto no existe en el mapa, se debe agregar el producto con un precio predeterminado de 5 euros. Muestra el mapa antes y después de la actualización o adición. La función que realiza la actualización o adición debe estar separada en una función distinta. Controla las posibles excepciones.

```
package DecimoTema;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ejercicio10_19 {
   public static void main(String[] args) {
      try {
         Map<String, Integer> productos = new HashMap<>();
         productos.put("Manzana", 2);
         productos.put("Plátano", 1);
         productos.put("Pera", 3);

         System.out.println("Mapa original: " + productos);
```

```
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese el nombre del producto que desea actualizar: ");
        String productoBuscado = scanner.nextLine();

        actualizarValor(productos, productoBuscado);

        System.out.println("Mapa después de la actualización: " + productos);

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
  }

  public static void actualizarValor(Map<String, Integer> mapa, String clave) {
    Scanner scanner = new Scanner(System.in);

    if (mapa.containsKey(clave)) {
        // Si la clave existe, pedir un nuevo valor al usuario
        System.out.print("El producto '" + clave + "' ya existe. Ingrese el nuevo precio: ");
        int nuevoValor = scanner.nextInt();
        mapa.put(clave, nuevoValor);
        System.out.println("Producto actualizado.");
    } else {
        System.out.println("El producto '" + clave + "' no existe. Se agregará con un precio
predeterminado de 5.");
        mapa.put(clave, 5);
        System.out.println("Producto agregado.");
    }
  }
}
```

Ejercicio 10.20. Crea una lista de cadenas de texto, luego elimina un elemento dado por el usuario. Después, ordénala alfabéticamente.

```
package DecimoTema;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.Collections;

public class Ejercicio10_20 {
  public static void main(String[] args) {
    List<String> frutas = new ArrayList<>();
    frutas.add("Manzana");
    frutas.add("Plátano");
    frutas.add("Pera");
    frutas.add("Naranja");

    System.out.println("Lista original: " + frutas);
```

```
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese el nombre de la fruta a eliminar: ");
        String frutaEliminar = scanner.nextLine();

        if (frutas.contains(frutaEliminar)) {
            frutas.remove(frutaEliminar);
            System.out.println("Fruta eliminada: " + frutaEliminar);
        } else {
            System.out.println("La fruta no está en la lista.");
        }

        System.out.println("Lista después de la eliminación: " + frutas);

         Collections.sort(numeros);

        System.out.println("Lista ordenada: " + numeros);
    }
}
```

Ejercicio 10.21. Dada una lista de países, desarrolla un programa que permita al usuario buscar un país, reemplazar uno por otro o eliminar todos los países de la lista. Utiliza distintas funciones para realizar las acciones.

```
package DecimoTema;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ejercicio10_21 {
    public static void main(String[] args) {
        List<String> paises = new ArrayList<>();
        paises.add("España");
        paises.add("Francia");
        paises.add("Italia");
        paises.add("Alemania");

        Scanner scanner = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\nLista actual de países: " + paises);
            System.out.println("Seleccione una opción:");
            System.out.println("1. Buscar un país");
            System.out.println("2. Reemplazar un país");
            System.out.println("3. Eliminar todos los países");
            System.out.println("4. Salir");
            System.out.print("Opción: ");
            opcion = scanner.nextInt();
            scanner.nextLine();  // Consumir el salto de línea pendiente
```

```
        switch (opcion) {
            case 1:
                buscarPais(paises, scanner);
                break;
            case 2:
                reemplazarPais(paises, scanner);
                break;
            case 3:
                eliminarTodos(paises);
                break;
            case 4:
                System.out.println("Saliendo del programa...");
                break;
            default:
                System.out.println("Opción no válida, intente de nuevo.");
        }
    } while (opcion != 4);

    scanner.close();
}

public static void buscarPais(List<String> lista, Scanner scanner) {
    System.out.print("Ingrese el nombre del país a buscar: ");
    String paisBuscado = scanner.nextLine();

    if (lista.contains(paisBuscado)) {
        System.out.println("El país '" + paisBuscado + "' está en la lista.");
    } else {
        System.out.println("El país '" + paisBuscado + "' no está en la lista.");
    }
}

public static void reemplazarPais(List<String> lista, Scanner scanner) {
    System.out.print("Ingrese el nombre del país que desea reemplazar: ");
    String paisAReemplazar = scanner.nextLine();

    if (lista.contains(paisAReemplazar)) {
        System.out.print("Ingrese el nuevo nombre del país: ");
        String nuevoPais = scanner.nextLine();
        int index = lista.indexOf(paisAReemplazar);
        lista.set(index, nuevoPais);
        System.out.println("El país ha sido reemplazado.");
    } else {
        System.out.println("El país '" + paisAReemplazar + "' no está en la lista.");
    }
}

public static void eliminarTodos(List<String> lista) {
    lista.clear();
    System.out.println("Todos los países han sido eliminados de la lista.");
}
```

```
}
```

NOTA: Una vez que se eliminan todos los países, aunque el programa sigue activo, ya no se puede buscar ni reemplazar nada útil. Te reto a que mejores el comportamiento del programa.

Ejercicio 10.22. Implementa un programa que muestre un menú para:

1. **Agregar una nueva tarea**
    - Solicitar al usuario una **categoría** (ej: "Trabajo", "Estudio", "Personal").
    - Solicitar una **descripción** de la tarea (ej: "Hacer informe").
    - Almacenar la tarea en:
        - La **lista general** de tareas.
        - El **mapa** bajo su categoría correspondiente.
        - La **cola** de tareas pendientes.
2. **Completar la siguiente tarea pendiente**
    - Extraer la tarea más antigua de la **cola** (FIFO).
    - Moverla a la **pila** del histórico (para consultarla luego).
3. **Mostrar la última tarea completada**
    - Consultar la **pila** del histórico (sin eliminar la tarea).
4. **Listar tareas por categoría**
    - Mostrar todas las tareas de una categoría específica usando el **mapa**.
5. **Mostrar todas las tareas registradas**
    - Recorrer la **lista general** de tareas.
6. **Salir del programa**

Gestiona las tareas utilizando:

- Una **pila** (stack) para el histórico de tareas completadas (LIFO).
- Una **cola** (queue) para manejar tareas pendientes (FIFO).
- Un **mapa** (map) para organizar tareas por categoría.
- Una **lista** (list) para almacenar todas las tareas registradas.

```
package DecimoTema;

import java.util.*;
```

```
public class Ejercicio10_22 {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    Map<String, List<String>> tareasPorCategoria = new HashMap<>();
    Queue<String> tareasPendientes = new LinkedList<>();
    Stack<String> historicoTareas = new Stack<>();
    List<String> todasLasTareas = new ArrayList<>();

    while (true) {
      System.out.println("MENÚ DE GESTIÓN DE TAREAS");
      System.out.println("1. Agregar nueva tarea");
      System.out.println("2. Completar siguiente tarea pendiente");
      System.out.println("3. Mostrar última tarea completada");
      System.out.println("4. Listar tareas por categoría");
      System.out.println("5. Mostrar todas las tareas existentes");
      System.out.println("6. Salir");
      System.out.print("Seleccione una opción: ");

      int opcion = scanner.nextInt();
      scanner.nextLine(); // Limpia el buffer

      switch (opcion) {
        case 1:
          System.out.print("Ingrese la categoría de la tarea: ");
          String categoria = scanner.nextLine();
          System.out.print("Ingrese la descripción de la tarea: ");
          String tarea = scanner.nextLine();

          todasLasTareas.add(tarea); // Agregar a la lista de todas las tareas

          tareasPorCategoria.putIfAbsent(categoria, new ArrayList<>());
          tareasPorCategoria.get(categoria).add(tarea);

          tareasPendientes.add(tarea); // Agregar a la cola de tareas pendientes

          System.out.println("Tarea agregada con éxito.");
          break;

        case 2:
          if (tareasPendientes.isEmpty()) {
            System.out.println("No hay tareas pendientes!");
          } else {
            String tareaCompletada = tareasPendientes.poll();
            historicoTareas.push(tareaCompletada);
            System.out.println("Tarea completada: " + tareaCompletada);
          }
          break;

        case 3:
          if (historicoTareas.isEmpty()) {
            System.out.println("No hay tareas completadas aún.");
```

```
                } else {
                    System.out.println("Última tarea completada: " + historicoTareas.peek());
                }
                break;

            case 4:
                System.out.print("Introduce la categoría: ");
                String cat = scanner.nextLine();

                if (tareasPorCategoria.containsKey(cat)) {
                    System.out.println("Tareas en la categoría " + cat + ":");
                    for (String t : tareasPorCategoria.get(cat)) {
                        System.out.println(t);
                    }
                } else {
                    System.out.println("No existen tareas en esa categoría.");
                }
                break;

            case 5:
                System.out.println("Todas las tareas registradas:");
                for (int i = 0; i < todasLasTareas.size(); i++) {
                    System.out.println(todasLasTareas.get(i));
                }
                break;

            case 6:
                System.out.println("Saliendo del programa.");
                scanner.close();
                return;

            default:
                System.out.println("Opción no válida.");
        }
    }
  }
}
```

EJERCICIOS RESUELTOS DEL TEMA 11

Ejercicio 11.1. Define las clases cocina, dormitorio y aseo. Todas tendrán atributos superficie y volumen. Crea una clase abstracta que contenga un método para escribir por consola estos atributos.

```
public class Ejercicio11_1 {
    static void main(String[] args) {
        Cocina c = new Cocina(20, 50);
        c.mostrarAtributos();
        Aseo a = new Aseo(10, 25);
        a.mostrarAtributos();
        Dormitorio d = new Dormitorio(30, 70);
        d.mostrarAtributos();
    }
}
```

```
public abstract class Estancia {
    protected float superficie;
    protected float volumen;

    public abstract void mostrarAtributos();
}
```

```
public class Cocina extends Estancia {
    public Cocina (float superficie, float volumen) {
        this.superficie = superficie;
        this.volumen = volumen;
    }

    @Override
    public void mostrarAtributos() {
        System.out.println("La superficie de la cocina es " + superficie + "
m2 y el volumen " + volumen + " m3.");
    }
}
```

```
public class Dormitorio extends Estancia {

    public Dormitorio(float superficie, float volumen) {
        this.superficie = superficie;
        this.volumen = volumen;
    }

    @Override
    public void mostrarAtributos() {
        System.out.println("La superficie del dormitorio es " + superficie +
" m2 y el volumen " + volumen + " m3.");
    }
}
```

```
public class Aseo extends Estancia {

    public Aseo (float superficie, float volumen) {
        this.superficie = superficie;
        this.volumen = volumen;
    }
```

```
    @Override
    public void mostrarAtributos() {
        System.out.println("La superficie del aseo es " + superficie + " m2 y
el volumen " + volumen + " m3.");
    }
}
```

Ejercicios 11.2. Haz una clase abstracta Shape que defina dos métodos abstractos area() y perimeter(). Las subclases Circle y Rectangle implementarán estos métodos para calcular el área y el perímetro de un círculo y un rectángulo. Crea una instancia de Circle y Rectangle y utiliza los métodos area() y perimeter() para calcular y mostrar el área y el perímetro de cada figura.

```
public abstract class Shape {
    protected int x;
    protected int y;

    public Shape(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public abstract double area();

    public abstract double perimeter();
}
```

```
public class Circle extends Shape {
    private double radius;

    public Circle(int x, int y, double radius) {
        super(x, y);
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * radius;
    }
}
```

```
public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(int x, int y, double width, double height) {
        super(x, y);
        this.width = width;
        this.height = height;
    }
```

```
    @Override
    public double area() {
        return width * height;
    }

    @Override
    public double perimeter() {
        return 2 * (width + height);
    }
}
```

```
public class Ejercicio11_2 {
    public static void main(String[] args) {
        Circle circle = new Circle(0, 0, 5);
        Rectangle rectangle = new Rectangle(0, 0, 10, 5);

        System.out.println("Área del círculo: " + circle.area());
        System.out.println("Perímetro del círculo: " + circle.perimeter());

        System.out.println("Área del rectángulo: " + rectangle.area());
        System.out.println("Perímetro del rectángulo: " +
rectangle.perimeter());
    }
}
```

Ejercicio 11.3. Piensa en una funcionalidad como acelerar. ¿Qué objetos pueden acelerar? Declara una interfaz y haz que las clases Coche, Guepardo y Humano la implementen. Después instancia un objeto de cada clase a través de la interfaz e invoca el método Acelerar.

```
public class Ejercicio11_3 {

    public static void main(String[] args) {
        IAcelerable humano = new Humano();
        humano.acelerar();
        IAcelerable coche = new Coche();
        coche.acelerar();
        IAcelerable guepardo = new Guepardo();
        guepardo.acelerar();
    }
}
```

```
public interface IAcelerable {
    void acelerar();
}
```

```
public class Humano implements IAcelerable {

    public void acelerar() {
        System.out.println("Acelera el humano");
    }
}
```

```
public class Coche implements IAcelerable {
```

```
    public void acelerar() {
        System.out.println("Acelera el coche");
    }
}
```

```
public class Guepardo implements IAcelerable {

    public void acelerar()
    {
        System.out.println("Acelera el guepardo");
    }
}
```

Ejercicio 11.4. Supongamos que tenemos una interfaz llamada Ave que tiene varios métodos para modelar el comportamiento de las aves, tales como volar(), ponerHuevos(), cantar(), etc. Sin embargo, no todas las aves son capaces de realizar todas estas acciones. Por ejemplo, los avestruces no pueden volar, pero sí ponen huevos. Por lo tanto, sería deseable tener interfaces más específicas para representar los comportamientos de las aves.

Segrega la interfaz Ave en varias interfaces más específicas y coherentes, utilizando el principio de segregación de interfaces. Cada nueva interfaz debe representar un comportamiento específico de las aves. Luego, define una clase Avestruz que implemente las interfaces necesarias que correspondan a los comportamientos que esta ave puede realizar. Genera una función main para instanciar objetos y probar el programa.

```
//Interfaz para aves que pueden volar
public interface AveVoladora {
 void volar();
}
```

```
//Interfaz para aves que pueden poner huevos
public interface AvePonedora {
 void ponerHuevos();
}
```

```
//Interfaz para aves que pueden cantar
public interface AveCantora {
 void cantar();
}
```

```
public class Avestruz implements AvePonedora {
 @Override
 public void ponerHuevos() {
     System.out.println("Soy un avestruz y estoy poniendo huevos");
 }
}
```

```
public class Ejercicio11_4 {
    public static void main(String[] args) {
        Avestruz avestruz = new Avestruz();

        avestruz.ponerHuevos(); // Output: Soy un avestruz y estoy poniendo
huevos
```

```
        // No se puede llamar a avestruz.volar() o avestruz.cantar() porque
Avestruz no implementa esas interfaces
    }
}
```

Ejercicio 11.5. Crea una jerarquía de clases para representar figuras geométricas. Debes implementar una clase abstracta *Figura* con un método abstracto calcularArea(). Luego, implementa las clases Circulo y Rectangulo que hereden de Figura y calculen su área correspondiente. Usa el método main para crear objetos de estas clases y mostrar el área.

```
abstract class Figura {
   abstract double calcularArea();
}
```

```
class Circulo extends Figura {
   double radio;
   Circulo(double radio) {
      this.radio = radio;
   }

   @Override
   double calcularArea() {
      return Math.PI * radio * radio;
   }
}
```

```
class Rectangulo extends Figura {
   double ancho, alto;

   Rectangulo(double ancho, double alto) {
      this.ancho = ancho;
      this.alto = alto;
   }

   @Override
   double calcularArea() {
      return ancho * alto;
   }
}
```

```
public class Ejercicio11_5 {
   public static void main(String[] args) {
      Figura f1 = new Circulo(3);
      Figura f2 = new Rectangulo(4, 5);

      System.out.println("Área del círculo: " + f1.calcularArea());
      System.out.println("Área del rectángulo: " + f2.calcularArea());
   }
}
```

Ejercicio 11.6. Crea una interfaz Sonido con el método hacerSonido(). Luego crea las clases Perro, Gato y Vaca que implementen esta interfaz. Cada clase debe tener su propia versión del sonido que emite. En el main, crea un array de Sonido y recórrelo para ejecutar hacerSonido() en cada objeto.

```
interface Sonido {
   void hacerSonido();
}
```

```
class Perro implements Sonido {
   public void hacerSonido() {
      System.out.println("El perro dice: ¡Guau!");
   }
}
```

```
class Gato implements Sonido {
   public void hacerSonido() {
      System.out.println("El gato dice: ¡Miau!");
   }
}
```

```
class Vaca implements Sonido {
   public void hacerSonido() {
      System.out.println("La vaca dice: ¡Muuu!");
   }
}
```

```
public class Ejercicio11_6 {
   public static void main(String[] args) {
      Sonido[] animales = {
         new Perro(),
         new Gato(),
         new Vaca()
      };

      for (Sonido animal : animales) {
         animal.hacerSonido();
      }
   }
}
```

Ejercicio 11.7. Crea una interfaz Conducible con los métodos acelerar() y frenar(). Define una clase abstracta Vehiculo con atributos marca, modelo y velocidad. Luego crea las clases Coche y Moto que hereden de Vehiculo e implementen la interfaz Conducible. En el main, crea instancias y simula una secuencia de acciones.

```
interface Conducible {
   void acelerar();
```

```
    void frenar();
}
```

```
abstract class Vehiculo {
    String marca;
    String modelo;
    int velocidad;

    Vehiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.velocidad = 0;
    }

    void mostrarEstado() {
        System.out.println(marca + " " + modelo + " va a " + velocidad + " km/h");
    }
}
```

```
class Coche extends Vehiculo implements Conducible {
    Coche(String marca, String modelo) {
        super(marca, modelo);
    }

    public void acelerar() {
        velocidad += 10;
    }

    public void frenar() {
        velocidad -= 5;
    }
}
```

```
class Moto extends Vehiculo implements Conducible {
    Moto(String marca, String modelo) {
        super(marca, modelo);
    }

    public void acelerar() {
        velocidad += 15;
    }

    public void frenar() {
        velocidad -= 10;
    }
}
```

```
public class Ejercicio11_7 {
    public static void main(String[] args) {
```

```
        Coche coche = new Coche("Toyota", "Corolla");
        Moto moto = new Moto("Yamaha", "FZ");

        coche.acelerar();
        coche.acelerar();
        coche.frenar();
        coche.mostrarEstado();

        moto.acelerar();
        moto.frenar();
        moto.mostrarEstado();
    }
}
```

Ejercicio 11.8. Define una interfaz MetodoPago con el método pagar(double monto). Crea las clases TarjetaCredito, Paypal y Efectivo que implementen esta interfaz con una lógica de ejemplo. Luego crea una clase Carrito con un método realizarPago(MetodoPago metodo, double monto). Prueba diferentes métodos de pago en el main.

```
interface MetodoPago {
    void pagar(double monto);
}
```

```
class TarjetaCredito implements MetodoPago {
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + "€ con tarjeta de crédito.");
    }
}
```

```
class Paypal implements MetodoPago {
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + "€ con PayPal.");
    }
}
```

```
class Efectivo implements MetodoPago {
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + "€ en efectivo.");
    }
}
```

```
class Carrito {
    void realizarPago(MetodoPago metodo, double monto) {
        metodo.pagar(monto);
    }
}
```

```
public class Ejercicio11_8 {
   public static void main(String[] args) {
      Carrito carrito = new Carrito();
      MetodoPago tarjeta = new TarjetaCredito();
      MetodoPago paypal = new Paypal();
      MetodoPago efectivo = new Efectivo();

      carrito.realizarPago(tarjeta, 50.0);
      carrito.realizarPago(paypal, 20.0);
      carrito.realizarPago(efectivo, 10.0);
   }
}
```

Ejercicio 11.9. Crea un sistema que distinga entre empleados administrativos y técnicos. Define:

- Una clase abstracta Empleado con nombre, id, y método calcularSalario().
- Una interfaz Reportable con el método generarReporte().
- Una interfaz Mantenible con el método realizarMantenimiento().

Luego la clase Administrativo hereda de Empleado e implementa Reportable y la clase Tecnico hereda de Empleado e implementa Mantenible.

Agrega un main que cree una lista de empleados y calcule sus salarios. Luego llamará a los métodos de mantenimiento o reporte según el tipo.

```
interface Reportable {
   void generarReporte();
}
```

```
interface Mantenible {
   void realizarMantenimiento();
}
```

```
interface Laboral {
   void realizarTarea();
}
```

```
abstract class Empleado implements Laboral {
   String nombre;
   int id;

   Empleado(String nombre, int id) {
      this.nombre = nombre;
      this.id = id;
   }

   abstract double calcularSalario();
}
```

```
class Administrativo extends Empleado implements Reportable {
   double sueldoBase;

   Administrativo(String nombre, int id, double sueldoBase) {
      super(nombre, id);
      this.sueldoBase = sueldoBase;
   }

   public double calcularSalario() {
      return sueldoBase;
   }

   public void generarReporte() {
      System.out.println(nombre + " (Administrativo) genera un reporte administrativo.");
   }

   public void realizarTarea() {
      generarReporte();
   }
}
```

```
class Tecnico extends Empleado implements Mantenible {
   double sueldoBase;
   int tareasRealizadas;

   Tecnico(String nombre, int id, double sueldoBase, int tareasRealizadas) {
      super(nombre, id);
      this.sueldoBase = sueldoBase;
      this.tareasRealizadas = tareasRealizadas;
   }

   public double calcularSalario() {
      return sueldoBase + (tareasRealizadas * 50);
   }

   public void realizarMantenimiento() {
      System.out.println(nombre + " (Técnico) realiza mantenimiento técnico.");
   }

   public void realizarTarea() {
      realizarMantenimiento();
   }
}
```

```
public class Ejercicio11_9 {
   public static void main(String[] args) {
      Empleado[] empleados = {
         new Administrativo("Ana", 101, 1500),
```

```
        new Tecnico("Luis", 102, 1200, 5),
        new Tecnico("Carlos", 103, 1200, 2),
        new Administrativo("Marta", 104, 1600)
    };

    for (Empleado e : empleados) {
        System.out.println(e.nombre + " tiene un salario de: " + e.calcularSalario() + "€");
        e.realizarTarea();
        System.out.println();
    }
  }
}
```

Ejercicio 11.10. Define una clase abstracta CuentaBancaria con atributos titular, saldo y métodos:

- depositar(double cantidad)
- retirar(double cantidad)
- consultarSaldo()

Crea la interfaz Transferible con método transferir(CuentaBancaria destino, double monto)

Implementa: CuentaCorriente y CuentaAhorro, que heredan de CuentaBancaria e implementan Transferible.

Simula en el main depósitos, retiros y transferencias entre cuentas. Incluye validaciones de saldo insuficiente y montos inválidos.

```
interface Transferible {
  void transferir(CuentaBancaria destino, double monto);
}
```

```
abstract class CuentaBancaria {
  String titular;
  double saldo;

  CuentaBancaria(String titular, double saldoInicial) {
    this.titular = titular;
    this.saldo = saldoInicial;
  }

  void depositar(double cantidad) {
    if (cantidad > 0) {
      saldo += cantidad;
      System.out.println(titular + " depositó " + cantidad + "€. Nuevo saldo: " + saldo);
    } else {
      System.out.println("Cantidad inválida para depositar.");
    }
  }
```

```
    void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            saldo -= cantidad;
            System.out.println(titular + " retiró " + cantidad + "€. Nuevo saldo: " + saldo);
        } else {
            System.out.println("Fondos insuficientes o cantidad inválida.");
        }
    }

    void consultarSaldo() {
        System.out.println("Saldo de " + titular + ": " + saldo + "€");
    }
}
```

```
class CuentaCorriente extends CuentaBancaria implements Transferible {
    CuentaCorriente(String titular, double saldoInicial) {
        super(titular, saldoInicial);
    }

    public void transferir(CuentaBancaria destino, double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            destino.saldo += monto;
            System.out.println(titular + " transfirió " + monto + "€ a " + destino.titular);
        } else {
            System.out.println("No se pudo transferir. Monto inválido o saldo insuficiente.");
        }
    }
}
```

```
class CuentaAhorro extends CuentaBancaria implements Transferible {
    CuentaAhorro(String titular, double saldoInicial) {
        super(titular, saldoInicial);
    }

    public void transferir(CuentaBancaria destino, double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            destino.saldo += monto;
            System.out.println(titular + " transfirió " + monto + "€ a " + destino.titular);
        } else {
            System.out.println("No se pudo transferir. Monto inválido o saldo insuficiente.");
        }
    }
}
```

```
class Ejercicio11_10 {
    public static void main(String[] args) {
        CuentaCorriente cc = new CuentaCorriente("Laura", 1000);
```

```
        CuentaAhorro ca = new CuentaAhorro("Pedro", 500);

        cc.consultarSaldo();
        ca.consultarSaldo();

        cc.depositar(200);
        ca.retirar(100);

        cc.transferir(ca, 300);

        cc.consultarSaldo();
        ca.consultarSaldo();
    }
}
```

EJERCICIOS RESUELTOS DEL TEMA 12

Ejercicio 12.1. Desarrolla un programa que guarde las frases que escriba el usuario en un fichero. El programa acabará cuando el usuario escriba "fin".

```
package DuodecimoTema;

import java.util.Scanner;
import java.io.*;

public class Ejercicio12_1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            FileWriter archivo = new FileWriter ("frases.txt");
            BufferedWriter escritor = new BufferedWriter(archivo);
            System.out.println("Escriba una frase para guardar en el
archivo: ");
            String frase = sc.nextLine();
            while (!frase.equals("fin")) {
                escritor.write(frase);
                escritor.newLine();
                frase = sc.nextLine();
            }
            escritor.close();
        } catch (IOException e) {
            System.out.println("Error al escribir en el archivo: " +
e.getMessage());
        }
    }
}
```

Ejercicio 12.2. Desarrolla un programa que lea el fichero escrito en el ejercicio anterior y lo muestre por consola.

```
package DuodecimoTema;

import java.io.*;

public class Ejercicio12_2 {
    public static void main(String[] args) {
        try {
            FileReader lector = new FileReader("frases.txt");
            BufferedReader buffer = new BufferedReader(lector);
            String line = buffer.readLine();
            while (line != null) {
                System.out.println(line);
                line = buffer.readLine();
            }
            buffer.close();
        } catch (IOException e) {
             System.out.println("Ha habido un error al leer el archivo: " +
e.getMessage());
        }
    }
}
```

Ejercicio 12.3. Desarrolla un programa que pida al usuario el nombre de un fichero de texto y un fragmento de un texto que quiera buscar. El programa mostrará por consola todas las líneas que contengan el texto que se está buscando.

```
package DuodecimoTema;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio12_3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Introduce el nombre del fichero: ");
        String nombreFichero = sc.nextLine();
        System.out.println("Introduce el fragmento de texto que quieres
buscar: ");
        String textoBuscado = sc.nextLine();

        try {
             FileReader lector = new FileReader(nombreFichero);
             BufferedReader buffer = new BufferedReader(lector);
             String linea;
            while ((linea = buffer.readLine()) != null) {
                if (linea.contains(textoBuscado)) {
                    System.out.println(linea);
                }
            }
        } catch (IOException e) {
            System.out.println("Error al abrir el fichero: " +
e.getMessage());
        }
    }
}
```

Ejercicio 12.4. Desarrolla un programa que cree un fichero donde se guardará una línea de texto. Luego se cerrará. Después, pedirá al usuario que introduzca frases para añadirlas al fichero creado. El proceso seguirá hasta que el usuario escriba “fin”. Utiliza funciones estáticas.

```
package DuodecimoTema;

import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio12_4 {
    public static void main(String[] args) {
        crearFichero();
        añadirFrases();
    }

    public static void crearFichero() {
      try {

            FileWriter escritor = new FileWriter("fichero.txt");
```

```
            escritor.write("Primera línea de texto");

            escritor.close();
        } catch (IOException e) {
            System.out.println("Ocurrió un error de entrada o salida del
archivo: " + e.getMessage());
        }
    }

    public static void añadirFrases() {
      try {
             Scanner sc = new Scanner(System.in);
            FileWriter escritor = new FileWriter("fichero.txt", true);
            String frase;

            while (true) {
                System.out.print("Introduce una frase (escribe 'fin' para
terminar): ");
                frase = sc.nextLine();
                if (frase.equals("fin")) {
                    break;
                }
                escritor.write("\n" + frase);
            }
            escritor.close();
            System.out.print("Fin del programa.");
        } catch (IOException e) {
            System.out.println("Ocurrió un error de entrada o salida del
archivo: " + e.getMessage());
        }
    }
}
```

Ejercicio 12.5. Desarrolla un programa que pida al usuario dos nombres de ficheros. El primero existirá y tendrá contenido, y el segundo se creará y estará vacío. Después, volcará todo el contenido del primer fichero al segundo, pasando todo el texto a mayúsculas. Además, se controlarán todas las posibles excepciones del programa.

```
package DuodecimoTema;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio12_5 {
    public static void main(String[] args) {
             Scanner sc = new Scanner(System.in);
             System.out.print("Introduce el nombre de un fichero: ");
             String nombreFichero1 = sc.nextLine();
        crearPrimerFichero(nombreFichero1);

             System.out.print("Introduce el nombre de otro fichero: ");
             String nombreFichero2 = sc.nextLine();
        crearSegundoFicheroYVolcarDatos(nombreFichero1, nombreFichero2);

        System.out.println("Fin del programa.");
```

```
    }

    public static void crearPrimerFichero(String nombreFichero1) {
      try {

            FileWriter escritor = new FileWriter(nombreFichero1);

            escritor.write("Primera línea de texto");
            escritor.write("\nSegunda línea de texto");
            escritor.write("\nTercera línea de texto");

            escritor.close();
        } catch (IOException e) {
            System.out.println("Ocurrió un error de entrada o salida del
archivo: " + e.getMessage());
        }
    }

    public static void crearSegundoFicheroYVolcarDatos(String
nombreFichero1, String nombreFichero2) {
      try {

            FileWriter escritor = new FileWriter(nombreFichero2);
            FileReader lector = new FileReader(nombreFichero1);
             BufferedReader buffer = new BufferedReader(lector);
             String linea;
            while ((linea = buffer.readLine()) != null) {
             escritor.write(linea.toUpperCase() + "\n");
            }
            escritor.close();
            lector.close();
            buffer.close();
        } catch (IOException e) {
            System.out.println("Ocurrió un error de entrada o salida del
archivo: " + e.getMessage());
        }
    }
}
```

Ejercicio 12.6. Desarrolla un programa que pida al usuario dos números y la operación que quiere realizar con ellos. Este proceso se seguirá realizando hasta que el usuario escriba en la operación "fin". Todos los cálculos se guardarán en un fichero. Además, el programa controlará todas las posibles excepciones, no solo las referidas a ficheros, sino de los propios datos (se introducen caracteres en lugar de números, se divide entre cero, etcétera). En el caso de saltar alguna excepción, no se debe interrumpir el programa.

```
package DuodecimoTema;

import java.util.Scanner;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio12_6 {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    String operacion = "";
    try {
```

```
      FileWriter writer = new FileWriter("resultados.txt");
      while (!operacion.equals("fin")) {
        System.out.print("Introduce el primer número: ");
        int num1 = scanner.nextInt();
        System.out.print("Introduce el segundo número: ");
        int num2 = scanner.nextInt();
        System.out.print("Introduce la operación (+, -, *, /) o escribe fin
para salir: ");
        operacion = scanner.next();
        switch (operacion) {
          case "+":
            int suma = num1 + num2;
            writer.write(num1 + " + " + num2 + " = " + suma + "\n");
            System.out.println("El resultado de la suma es " + suma);
            break;
          case "-":
            int resta = num1 - num2;
            writer.write(num1 + " - " + num2 + " = " + resta + "\n");
            System.out.println("El resultado de la resta es " + resta);
            break;
          case "*":
            int multiplicacion = num1 * num2;
            writer.write(num1 + " * " + num2 + " = " + multiplicacion +
"\n");
            System.out.println("El resultado de la multiplicación es " +
multiplicacion);
            break;
          case "/":
            if (num2 == 0) {
              System.out.println("No se puede dividir por cero");
              writer.write(num1 + " / " + num2 + " = error (división por
cero)\n");
            } else {
              double division = (double) num1 / num2;
              writer.write(num1 + " / " + num2 + " = " + division + "\n");
              System.out.println("El resultado de la división es " +
division);
            }
            break;
          case "fin":
            break;
          default:
            System.out.println("Operación no válida");
            break;
        }
      }
      writer.close();
      System.out.println("Fin del programa.");
    } catch (NumberFormatException e) {
       System.out.println("Error de formato: " + e.getMessage());
    } catch (IOException e2) {
      System.out.println("Error al guardar el archivo: " +
e2.getMessage());
    } catch (Exception e3) {
        System.out.println("Se ha producido un error: " + e3.getMessage());
    }
  }
}
```

Ejercicio 12.7. Desarrolla un programa que pida una palabra al usuario. Después tendrá que leer un archivo de texto creado previamente y contar cuántas veces aparece la palabra. Utiliza el método Split() para dividir las frases en palabras. El resultado lo escribirá en un nuevo fichero de texto. El programa debe gestionar las posibles excepciones.

NOTA: Cuando usas throws IOException en la declaración de un método, delegas la responsabilidad de manejar la excepción a quien llame al método. En este caso, debe ser el propio método quien gestione las posibles excepciones, es decir, debes usar bloques de try-catch.

```
package DuodecimoTema;

import java.io.*;
import java.util.Scanner;

public class Ejercicio12_7 {

   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      System.out.print("Ingresa la palabra que deseas contar: ");
      String palabraBuscada = scanner.nextLine();

      try {
         contarPalabraEnArchivo(palabraBuscada);
      } catch (IOException e) {
         System.out.println("Ocurrió un error al procesar el archivo: " + e.getMessage());
      }
   }

   public static void contarPalabraEnArchivo(String palabra) throws IOException {
      FileReader fr = new FileReader("archivo.txt");
      BufferedReader br = new BufferedReader(fr);

      FileWriter fw = new FileWriter("resultado.txt");
      BufferedWriter bw = new BufferedWriter(fw);

      String linea;
      int contador = 0;

      while ((linea = br.readLine()) != null) {
         String[] palabras = linea.split(" ");

         for (String palabraLinea : palabras) {
            if (palabraLinea.equals(palabra)) {
               contador++;
            }
         }
      }

      bw.write("La palabra '" + palabra + "' aparece " + contador + " veces en el archivo.");
      System.out.println("El resultado ha sido escrito en 'resultado.txt'.");
```

```
        br.close();//así se cierra el fileReader también
        bw.close();//así se cierra el fileWriter también
    }
}
```

Ejercicio 12.8. Desarrolla un programa que lea un archivo de texto y escriba todo el contenido en otro fichero de texto en mayúsculas (método toUpperCase). El programa debe gestionar las posibles excepciones. Además, cada línea en el archivo de salida debe comenzar con su número de línea.

```
package DuodecimoTema;

import java.io.*;

public class Ejercicio12_8 {

    public static void main(String[] args) {
        try {
            convertirArchivoAMayusculas("archivoEntrada.txt", "archivoSalida.txt");
        } catch (IOException e) {
            System.out.println("Ocurrió un error al procesar el archivo: " + e.getMessage());
        }
    }

    public static void convertirArchivoAMayusculas(String archivoEntrada, String archivoSalida)
throws IOException {
        FileReader fr = new FileReader(archivoEntrada);
        BufferedReader br = new BufferedReader(fr);

        FileWriter fw = new FileWriter(archivoSalida);
        BufferedWriter bw = new BufferedWriter(fw);

        String linea;
        int numeroLinea = 1;

        while ((linea = br.readLine()) != null) {
            String lineaEnMayusculas = linea.toUpperCase();
            String lineaNumerada = numeroLinea + ": " + lineaEnMayusculas;
            bw.write(lineaNumerada);
            bw.newLine();//esto es como presionar intro
            numeroLinea++;
        }

        br.close();//así se cierra el fileReader también
        bw.close();//así se cierra el fileWriter también
    }
}
```

Ejercicio 12.9. Desarrolla un programa que permita al usuario escribir mensajes por consola. Estos mensajes se añadirán al final de un archivo de texto existente junto con la fecha en cada línea (utiliza el método now() de la clase LocalDateTime). El proceso termina cuando el usuario escriba la palabra "salir". Gestiona las posibles excepciones.

```
package DuodecimoTema;

import java.io.*;
import java.util.Scanner;
import java.time.LocalDateTime;

public class Ejercicio12_9 {

   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);
      String mensaje;

      try {
         FileWriter fw = new FileWriter("registro.txt", true);
         BufferedWriter bw = new BufferedWriter(fw);

         System.out.println("Añade mensajes. Escribe 'salir' para terminar.");

         while (true) {
            mensaje = scanner.nextLine();

            if (mensaje.equals("salir")) {
               break;
            }

            String linea = "[" + LocalDateTime.now() + "] " + mensaje;

            bw.write(linea);
            bw.newLine();//esto es como un intro
         }

         bw.close();//al cerrar este objeto se cierra el FileWriter
      } catch (IOException e) {
         System.out.println("Error al escribir en el archivo: " + e.getMessage());
      }
   }
}
```

Ejercicio 12.10. Desarrolla un programa que lea un archivo de texto que contiene una lista de estudiantes y sus calificaciones. El formato del archivo es el siguiente:

```
Juan Pérez, 8.5
María García, 7.0
Carlos López, 9.2+
Ana Sánchez, 6.8
```

El programa debe:

1. Leer el archivo de texto y almacenar la información de las calificaciones en una lista.
2. Calcular la **nota media** de todos los estudiantes.
3. Crear un **nuevo archivo** con las calificaciones de los estudiantes ordenadas de menor a mayor.
4. Añadir al final del archivo una línea que indique la **nota media** de todos los estudiantes.

El programa debe gestionar las excepciones que puedan ocurrir durante la lectura y escritura de archivos.

```
package DuodecimoTema;

import java.io.*;
import java.util.*;

public class Ejercicio12_10 {

   public static void main(String[] args) {
      List<Double> calificaciones = new ArrayList<>();
      String archivoEntrada = "calificaciones.txt";
      String archivoSalida = "calificaciones_ordenadas.txt";

      try {
         BufferedReader br = new BufferedReader(new FileReader(archivoEntrada));
         String linea;

         while ((linea = br.readLine()) != null) {
            String[] partes = linea.split(", ");
            double calificacion = Double.parseDouble(partes[1]);
            calificaciones.add(calificacion);
         }

         br.close();

         double suma = 0;
         for (Double calificacion : calificaciones) {
            suma += calificacion;
         }
         double media = suma / calificaciones.size();

         calificaciones.sort();

         BufferedWriter bw = new BufferedWriter(new FileWriter(archivoSalida));

         for (Double calificacion : calificaciones) {
            bw.write(calificacion.toString());
            bw.newLine();
         }
```

```
            bw.write("Nota media: " + media);
            bw.close();

            System.out.println("Proceso completado. El archivo de salida es: " + archivoSalida);

        } catch (IOException e) {
            System.out.println("Error al procesar el archivo: " + e.getMessage());
        }
    }
}
```

Ejercicio 12.11. Desarrolla un programa que lea un archivo y calcule el número total de caracteres, de palabras y de líneas que tiene. Escribe los resultados en un nuevo archivo de texto.

```
package DuodecimoTema;

import java.io.*;

public class Ejercicio12_11 {

    public static void main(String[] args) {
        String archivoEntrada = "entrada.txt";
        String archivoSalida = "resumen.txt";

        int totalLineas = 0;
        int totalPalabras = 0;
        int totalCaracteres = 0;

        try {
            BufferedReader br = new BufferedReader(new FileReader(archivoEntrada));
            String linea;

            while ((linea = br.readLine()) != null) {
                totalLineas++;
                totalCaracteres += linea.length();
                String[] palabras = linea.split(" ");
                totalPalabras += palabras.length;
            }

            br.close();

            BufferedWriter bw = new BufferedWriter(new FileWriter(archivoSalida));
            bw.write("Líneas: " + totalLineas);
            bw.newLine();
            bw.write("Palabras: " + totalPalabras);
            bw.newLine();
            bw.write("Caracteres: " + totalCaracteres);
            bw.close();

            System.out.println("Resumen generado en: " + archivoSalida);
```

```
        } catch (IOException e) {
            System.out.println("Error al procesar el archivo: " + e.getMessage());
        }
    }
}
```

Ejercicio 12.12. Desarrolla un programa que lea un archivo. Luego, tiene que escribir esas líneas en un archivo nuevo en orden inverso (última línea primero).

```
package DuodecimoTema;

import java.io.*;
import java.util.Stack;

public class Ejercicio12_12 {

    public static void main(String[] args) {
        String archivoEntrada = "entrada.txt";
        String archivoSalida = "invertido.txt";

        Stack<String> pilaLineas = new Stack<>();

        try {
            BufferedReader br = new BufferedReader(new FileReader(archivoEntrada));
            String linea = "";

            while (true) {
                linea = br.readLine();
                if (linea == null) break;

                pilaLineas.push(linea);
            }

            br.close();

            BufferedWriter bw = new BufferedWriter(new FileWriter(archivoSalida));

            while (!pilaLineas.isEmpty()) {
                String lineaOriginal = pilaLineas.pop();
                StringBuilder sb = new StringBuilder(lineaOriginal);
                sb.reverse();
                String lineaInvertida = sb.toString();

                bw.write(lineaInvertida);
                bw.newLine();
            }

            bw.close();

            System.out.println("Archivo invertido creado: " + archivoSalida);
```

```
        } catch (IOException e) {
            System.out.println("Error al procesar el archivo: " + e.getMessage());
        }
    }
}
```